

6.

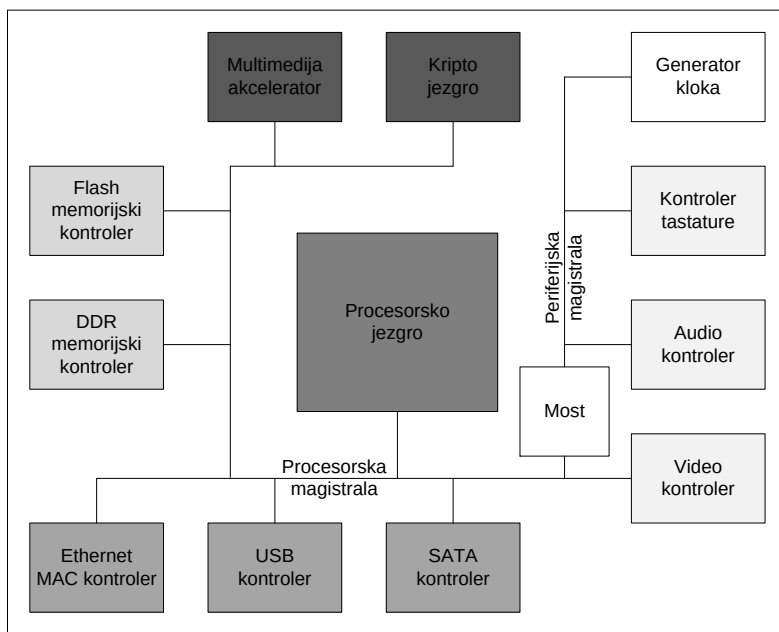
Projektovanje IP jezgara

Digitalni sistemi projektovani korišćenjem RT metodologije obično predstavljaju gradivne blokove većih, složenijih sistema na čipu (*System on Chip, SoC*). SoC sistemi se zatim implementiraju korišćenjem ASIC ili FPGA tehnologije. Gradivni blokovi od kojih se sastoje savremeni SoC sistemi nazivaju se i IP jezgrima (*IP Cores*). Da bi se olakšalo projektovanje SoC sistema i povećavala mogućnost ponovnog korišćenja prethodno razvijenih IP jezgara, IP jezgra se projektuju tako da koriste standardizovane interfejse, koji zatim olakšavaju njihovu integraciju u nove SoC sisteme. U ovoj glavi biće reči o standardnim interfejsima koji se koriste unutar SoC sistema, sa naglaskom na AXI familiju interfejsa, kao i potrebnim modifikacijama i doradama digitalnih sistema projektovanih pomoću RT metodologije koje je neophodno izvršiti kako bi oni mogli efikasno da se koriste unutar savremenih SoC sistema. Takođe će biti prikazan polu-automatizovani postupak pakovanja sistema projektovanog pomoću RT metodologije u IP jezgro, baziran na Xilinx IP Packager alatu, koje se zatim može koristiti unutar Xilinx Vivado Design Suite alata.

6.1. SoC systemske magistrale

Digitalni sistemi koji implementiraju odabrani algoritam obrade podataka, projektovani korišćenjem RT metodologije, retko kada predstavljaju najviši hijerarhijski nivo dizajna. Najčešće su oni zapravo gradivni blokovi nekog većeg, složenijeg dizajna, koji se sastoje od velikog broja modula specijalizovanih za izvršavanje specifičnih zadataka

unutar sistema. Ovakav složeni sistem naziva se sistem na čipu (SoC). SoC sistemi po pravilu su bazirani na jednom ili više procesorskih jezgara, koji su putem jedne ili više sistemskih magistrala povezani sa memorijskim i komunikacionim modulima, kao i sa modulima za specijalizovanu obradu podataka, kao što je prikazano na slici 6.1. Moduli iz poslednje dve grupe (komunikacioni moduli i moduli za specijalizovanu obradu podataka) tipično se projektuju korišćenjem RT metodologije, diskutovane u glavi 4.



Slika 6.1. Tipična struktura SoC sistema

Da bi se olakšalo projektovanje SoC sistema i povećala mogućnost ponovnog korišćenja postojećih IP jezgara, standardizuje se način na koji se podaci, komande i statusne informacije razmenjuju između različitih modula unutar SoC sistema. Za razmenu ovih informacija koriste se standardne SoC sistemske magistrale, sa jasno definisanim protokolima za razmenu podataka. Na žalost, ne postoji samo jedna standardizovana grupa sistemskih magistrala, već se vremenom pojavio veći broj različitih rešenja, od kojih su najpoznatija:

- ARM AMBA magistrale
- IBM CoreConnect magistrala
- Altera AVALON magistrala
- Silicore Corporation WISHBONE magistrala
- STMicroelectronics STBus magistrala

- CoreFrame magistrala
- MARBLE magistrala
- PI magistrala
- Open Core Protocol
- Virtual Component Interface

U konkretnom SoC projektu, odluka koja će se sistemska magistrala koristiti u najvećoj meri zavisi od izbora procesorskog jezgra. Većina procesorskih jezgara direktno podržava samo jedan tip sistemske magistrale, tako da ova činjenica zapravo predstavlja glavno ograničenje prilikom izbora sistemske magistrale.

Kako u savremenim SoC sistemima trenutno dominiraju ARM procesori, koji direktno podržavaju AMBA ili AXI sistemske magistrale, od kojih je AXI magistrala novija, u nastavku će detaljnije biti diskutovana AXI sistemska magistrala.

6.1.1. AXI interfejs

AXI (*Advanced Extensible Interface*) magistrale predstavljaju poslednju evoluciju ARM AMBA (*Advanced Microcontroller Bus Architecture*) magistrale. Trenutno je aktuelna četvrta verzija AXI protokola, AXI4, standardizovana 2010. godine. Kao što je već rečeno, AXI protokol pripada grupi ARM AMBA familije SoC magistrala. AMBA protokol je specifikacija otvorenog tipa koja omogućava povezivanje i kontrolu velikog broja kontrolera i periferijskih jedinica unutar multi-master sistema. Kao i većina ostalih članova AMBA familije, AXI je namenjena za korišćenje unutar SoC sistema visokih performansi, koji rade na visokim učestanostima. AXI komunikacioni protokol ima sledeće karakteristike:

- Adresne i kontrolne faze odvojene su od faza prenosa podataka
- Kontrolni signali validnosti pojedinačnih bajtova prilikom prenosa podataka (*Byte Strokes*) omogućavaju neporavnate prenose podataka
- Podržan je prenos podataka u blokovima (*Burst-based Transaction*), pri čemu je neophodno specificirati samo početnu adresu bloka
- Kanali za upis i čitanje podataka su razdvojeni, omogućavajući jednostavnu implementaciju DMA prenosa
- Inicirane transakcije mogu se kompletirati u redosledu koji je različit od redosleda u kom su zadate (*Out-of-order Completion*)
- Moguće je jednostavno uvođenje protočne obrade, u cilju zadovoljavanja zahtevanih vremenskih karakteristika sistema

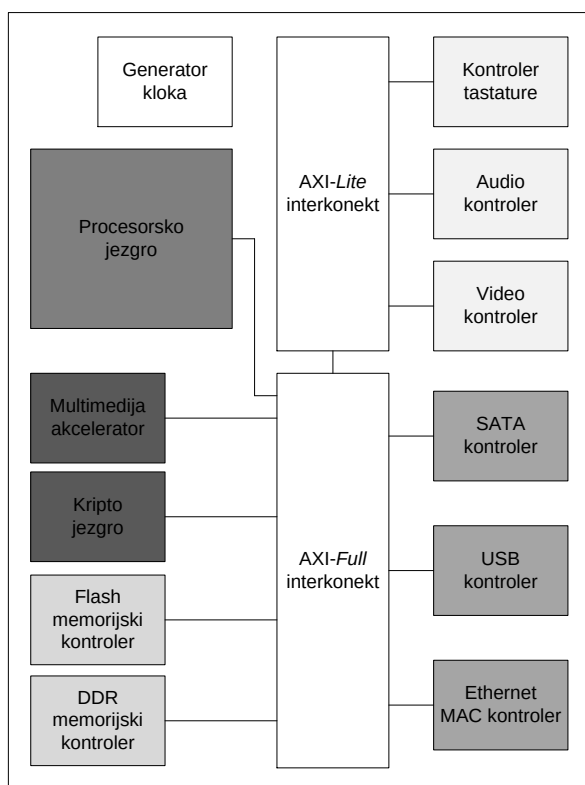
Postoje ukupno tri varijante AXI4 interfejsa:

- **AXI4** – poznat i pod nazivom *AXI4-Full* interfejs. Ovo je AXI interfejs visokih performansi, prikladan za komunikaciju sa memorijski mapiranim modulima kod kojih je podržan prenos podataka u blokovima od maksimalno 256 transfer ciklusa.
- **AXI4-Lite** – pojednostavljena verzija *AXI4-Full* interfejsa, koja se takođe koristi za komunikaciju sa memorijski mapiranim modulima, ali kod kojih ne postoji mogućnost prenosa podataka u blokovima, već je moguće prenositi samo jedan podatak po transakciji. Usled toga su hardverski resursi neophodni za implementaciju *AXI4-Lite* kontrolera znatno manji od resursa potrebnih za implementaciju *AXI4-Full* kontrolera. Takođe je pojednostavljen i sam interfejs, jer je dobar deo signala iz *AXI4-Full* interfejsa nepotreban.
- **AXI4-Stream** – ova varijanta AXI4 protokola ne sadrži adresnu fazu prilikom transfera podataka. Koristi se u aplikacijama gde koncept adrese nije prisutan ili potreban (kao što su *dataflow* DSP aplikacije: kompresija zvuka, slike, videa, filtriranje, enkripcija, itd.). Kako nema adresne faze, ovaj interfejs je vrlo jednostavan i sastoji se iz minimalnog broja signala, ali se ne može koristiti za komunikaciju sa memorijski mapiranim modulima. Sa druge strane ne postoji nikakvo ograničenje na broj podataka koji se mogu preneti pomoću jedne transakcije. Ova vrsta interfejsa je unidirekciona, što znači da se podaci mogu prenositi samo u jednom smeru, od mastera ka slejvu. Ukoliko je potrebno ostvariti bidirekcioni prenos podataka, moraju se koristiti dva *AXI4-Stream* interfejsa i svaki od učesnika u komunikaciji mora implementirati i logiku *AXI4-Stream* mastera kao i logiku *AXI4-Stream* slejva.

Bez obzira na varijantu AXI interfejsa, sve AXI kompatibilne periferije dele se u dve grupe: AXI mastere i AXI slejvove. AXI masteri iniciraju transakciju, dok AXI slejvovi mogu samo da odgovaraju na započetu transakciju. U svakoj AXI transakciji mora da učestvuje tačno jedan AXI master (*AXI Master*) i jedan AXI slejv (*AXI Slave*). U sistemu može biti više AXI mastera i više AXI slejvova. Za ostvarivanje potrebnih veza između AXI mastera i AXI slejvova koriste se posebni moduli, AXI interkonekti (*AXI Interconnect*). U slučaju da u sistemu postoji samo jedan AXI master i jedan AXI slejv, AXI interkonekt nije potreban. AXI master inicira transakciju sa željenim AXI slejvom preko odgovarajućeg AXI interkonekt modula, na koji su povezani AXI master i AXI slejv. Analizirajući ciljnu adresu transakcije, koja je deo transakcije, AXI interkonekt odlučuje kojem od priključenih AXI slejvova je namenjena transakcija i prosleđuje je na odgovarajući port.

Gornja priča odnosi se na način povezivanja modula koji komuniciraju korišćenjem *AXI-Full* ili *AXI-Lite* interfejsa. U slučaju *AXI-Stream* interfejsa i dalje postoje AXI master i AXI slejv moduli, ali se povezivanje većeg broja master i slejvova ovog puta mora izvršiti pomoću odvojenih *Point-to-point* konekcija. Kako u okviru *AXI-Stream* interfejsa ne postoji koncept adrese, nije moguće koristiti AXI interkonekt modul, jer on ne bi imao načina kako da rutira transakcije ka odgovarajućim slejvovima.

Na primer, na slici 6.2 prikazana je moguća arhitektura SoC sistema sa slike 6.1, u slučaju korišćenja AXI interfejsa za ostvarivanje veza između različitih IP jezgara.

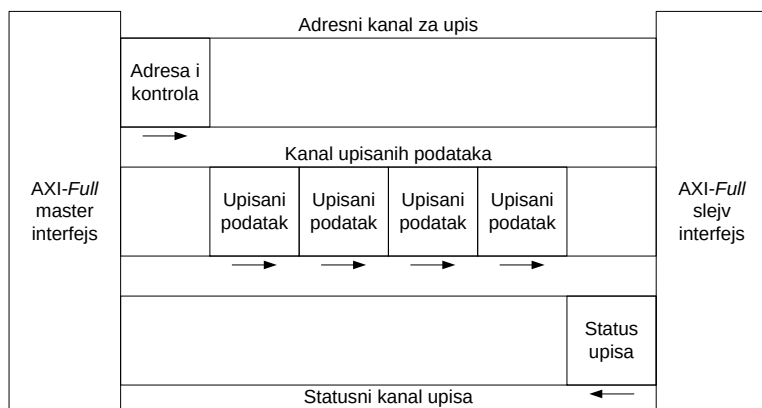


Slika 6.2. Tipična struktura SoC sistema baziranog na korišćenju AXI interfejsa

Arhitektura AXI-Full i AXI-Lite interfejsa je vrlo slična, pri čemu je AXI-Lite interfejs jednostavniji i ne sadrži neke od signala koji su prisutni u AXI-Full varijanti. AXI-Full i AXI-Lite interfejsi sastoje se iz sledećih pet kanala:

- Adresnog kanala za čitanje (*Read Address Channel*)
- Adresnog kanala za upis (*Write Address Channel*)
- Kanala pročitanih podataka (*Read Data Channel*)
- Kanala upisanih podataka (*Write Data Channel*)
- Statusnog kanala upisa (*Write Response Channel*)

Na slici 6.3 prikazan je tok transakcije upisa podataka korišćenjem AXI-Full interfejsa.



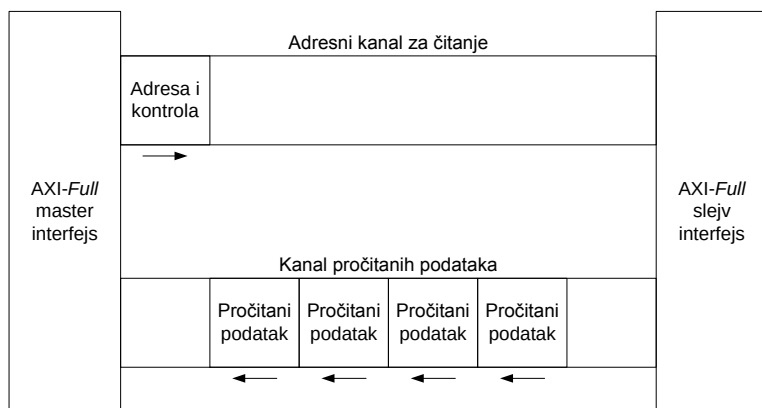
Slika 6.3. Transakcija upisa podataka korišćenjem AXI-Full interfejsa

U slučaju upisa podataka aktivna su tri kanala, adresni kanal za upis, kanal upisanih podataka i statusni kanal upisa. AXI master inicira transakciju upisa bloka podataka šaljući početnu adresu bloka podataka koji će biti upisan, veličinu bloka, i ostale neophodne informacije preko adresnog kanala za upis ka AXI slejvu. Kada AXI slejv prihvati ove informacije, AXI master počinje da šalje blok podataka koji je potrebno upisati unutar AXI slejva, preko kanala upisanih podataka. Nakon prenetog poslednjeg podatka iz bloka, i nakon što AXI slejv završi njihov upis, AXI slejv šalje informaciju o završnom stanju operacije upisa AXI masteru preko statusnog kanala upisa. Ovaj dodatni kanal je neophodan, jer kako podaci teku od AXI mastera ka AXI slejvu, AXI master nema načina da sazna kada je AXI slejv zapravo završio sa procesom upisa podataka. Tek nakon ovog trenutka AXI master sme inicirati novu transakciju. Upravo iz ovog razloga AXI slejv mora da signalizira uspešan ili neuspešan proces upisa podataka ka AXI masteru, koristeći statusni kanal upisa.

Obratite pažnju da prilikom upisa bloka podataka pomoću AXI-Full interfejsa veličina bloka može biti jednaka i samo jednom podatku.

NAPOMENA: Transakcija upisa podataka pomoću AXI-Lite interfejsa je identična sa onom prikazanom na slici 6.3, pri čemu je u svakoj transakciji moguće upisati **samo jedan** podatak.

Na slici 6.4 prikazan je tok transakcije upisa podataka korišćenjem AXI-Full interfejsa.



Slika 6.4. Transakcija čitanja podataka korišćenjem AXI-Full interfejsa

Prilikom čitanja podataka aktivna su samo dva kanala, adresni kanal za čitanje i kanal pročitanih podataka. Ne postoji statusni kanal za čitanje jer je on suvišan. AXI master zna koliko podataka želi da pročita i kada dobije poslednji podatak, može smatrati tekuću transakciju čitanja kompletiranom. AXI master inicira transakciju čitanja bloka podataka šaljući početnu adresu bloka podataka koji će biti pročitani, veličinu bloka, i ostale neophodne informacije preko adresnog kanala za upis ka AXI slejvu. Kada AXI slejv prihvati ove informacije, počinje sa slanjem podataka ka AXI masteru, koristeći kanal pročitanih podataka. Kada AXI master prihvati poslednji pročitani podatak, transakcija čitanja podataka je kompletirana.

NAPOMENA: Kao i u slučaju transakcije upisa, transakcija čitanja podataka pomoću AXI-Lite interfejsa je identična sa onom prikazanom na slici 6.4, pri čemu je u svakoj transakciji moguće pročitati **samo jedan** podatak.

Sledeća tabela prikazuje spisak svih signala koji čine AXI-Full i AXI-Lite interfejs, u slučaju AXI slejv komponente (što je najčešći slučaj, jer su IP moduli uglavnom AXI slejv komponente u SoC sistemu) zajedno sa kratkim objašnjenjem svakog od signala.

Naziv signala	Opis signala	AXI Lite
<i>s_axi_aclk</i>	Sinhronizacioni signal AXI interfejsa.	X
<i>s_axi_aresetn</i>	Reset signal AXI interfejsa.	X
Adresni kanal za upis		
<i>s_axi_awaddr</i>	Početna adresa bloka od koje će biti upisivani podaci.	X
<i>s_axi_awlen</i>	Veličina bloka podataka (1-256 podataka). Vrednost 0 odgovara veličini bloka od 1 podatka, a vrednost 255 odgovara bloku veličine 256 podataka.	-

<i>s_axi_awsiz</i>	Veličina individualnog podatka. Moguće vrednosti: 1, 2, 4, 8, 16, 32, 64, 128 bita.	-
<i>s_axi_awburst</i>	Tip blok transfera. Mogući tipovi su: fixed , increment i wrap . Prilikom fixed transfera (kod 0b00) svi podaci iz bloka upisuju se na istu adresu, koja je prenet na <i>s_axi_awaddr</i> magistrali. Prilikom increment transfera (kod 0b01) adresa naredne lokacije na koju se upisuje podatak dobija se inkrementacijom prethodne adrese (gde je korak inkrementacije određen tekućom vrednošću <i>s_axi_awsiz</i> magistrale). Prilikom wrap transfera (kod 0b10) adresa naredne lokacije se dobija inkrementacijom prethodne adrese, kao u slučaju <i>increment</i> tipa transfera, ali ako se prilikom inkrementacije dostigne gornja granična adresa, adresa se incijalizuje na početnu.	-
<i>s_axi_awlock</i>	Vrsta zaključavanja. Retko se koristi.	-
<i>s_axi_awcache</i>	Vrsta memorije sa kojom se komunicira. Retko se koristi.	-
<i>s_axi_awprot</i>	Vrsta pristupa. Retko se koristi.	X
<i>s_axi_awregion</i>	Broj regiona u memoriji kojem se pristupa. Retko se koristi.	-
<i>s_axi_awqos</i>	Identifikacija kvaliteta usluge. Retko se koristi.	-
<i>s_axi_awvalid</i>	Indikacija validnosti adrese. Kada je visok, trenutna adresa je validna.	X
<i>s_axi_awready</i>	Indikacija da je AXI slejv spreman da prihvati adresu i ostale kontrolne signale iz kanala.	X
Kanal upisanih podataka		
<i>s_axi_wdata</i>	Podatak koji je potrebno upisati.	X
<i>s_axi_wstrb</i>	Signal validnosti pojedinačnih bajtova unutar podatka koji se upisuje. Svaki bit koji je setovan znači da je njemu pridružen bajt u podatku validan.	X
<i>s_axi_wlast</i>	Indikacija da je tekući podatak poslednji koji je potrebno upisati.	-
<i>s_axi_wvalid</i>	Indikacija validnosti podatka koji se upisuje. Kada je visok, trenutni podatak je validan.	X
<i>s_axi_wready</i>	Indikacija da je AXI slejv spreman da prihvati podatak.	X
Statusni kanal upisa		
<i>s_axi_bresp</i>	Status kompletirane transakcije upisa.	X
<i>s_axi_bvalid</i>	Indikacija da je podatak u statusu transakcije validan.	X
<i>s_axi_bready</i>	Indikacija da je AXI master spreman da prihvati podatak o statusu.	X
Adresni kanal za čitanje		
<i>s_axi_araddr</i>	Početna adresa bloka od koje će biti čitani podaci.	X
<i>s_axi_arlen</i>	Veličina bloka podataka (1-256 podataka).	-

<i>s_axi_arsize</i>	Veličina individualnog podatka. Moguće vrednosti: 1, 2, 4, 8, 16, 32, 64, 128 bita.	-
<i>s_axi_arburst</i>	Tip blok transfera. Mogući tipovi su: <i>fixed</i> , <i>increment</i> i <i>wrap</i> . Značenje je isto kao u slučaju <i>s_axi_awburst</i> signala.	-
<i>s_axi_arlock</i>	Vrsta zaključavanja. Retko se koristi.	-
<i>s_axi_arcache</i>	Vrsta memorije sa kojom se komunicira. Retko se koristi.	-
<i>s_axi_arprot</i>	Vrsta pristupa. Retko se koristi.	X
<i>s_axi_arregion</i>	Broj regiona u memoriji kojem se pristupa. Retko se koristi.	-
<i>s_axi_arqos</i>	Identifikacija kvaliteta usluge. Retko se koristi.	-
<i>s_axi_arvalid</i>	Indikacija validnosti adrese. Kada je visok, trenutna adresa je validna.	X
<i>s_axi_arready</i>	Indikacija da je AXI slejv spreman da prihvati adresu i ostale kontrolne signale iz kanala.	X
Kanal pročitanih podataka		
<i>s_axi_rdata</i>	Podatak koji je pročitao.	X
<i>s_axi_rresp</i>	Status kompletirane transakcije čitanja.	X
<i>s_axi_rlast</i>	Indikacija da je tekući podatak poslednji koji je trebalo pročitati.	-
<i>s_axi_rvalid</i>	Indikacija validnosti podatka koji se čita. Kada je visok, trenutni podatak je validan.	X
<i>s_axi_rready</i>	Indikacija da je AXI mater spreman da prihvati podatak.	X

Tabela 6.1. Signali AXI-Full i AXI-Lite interfejsa u slučaju AXI slejv komponente

NAPOMENA: U koloni „AXI-Lite“ simbolom X označeni su signali koji su prisutni u AXI-Lite interfejsu.

U nastavku će biti prikazani i komentarisani tipični talasni oblici signala AXI-Full i AXI-Lite interfejsa u slučaju transakcija upisa i čitanja podataka.

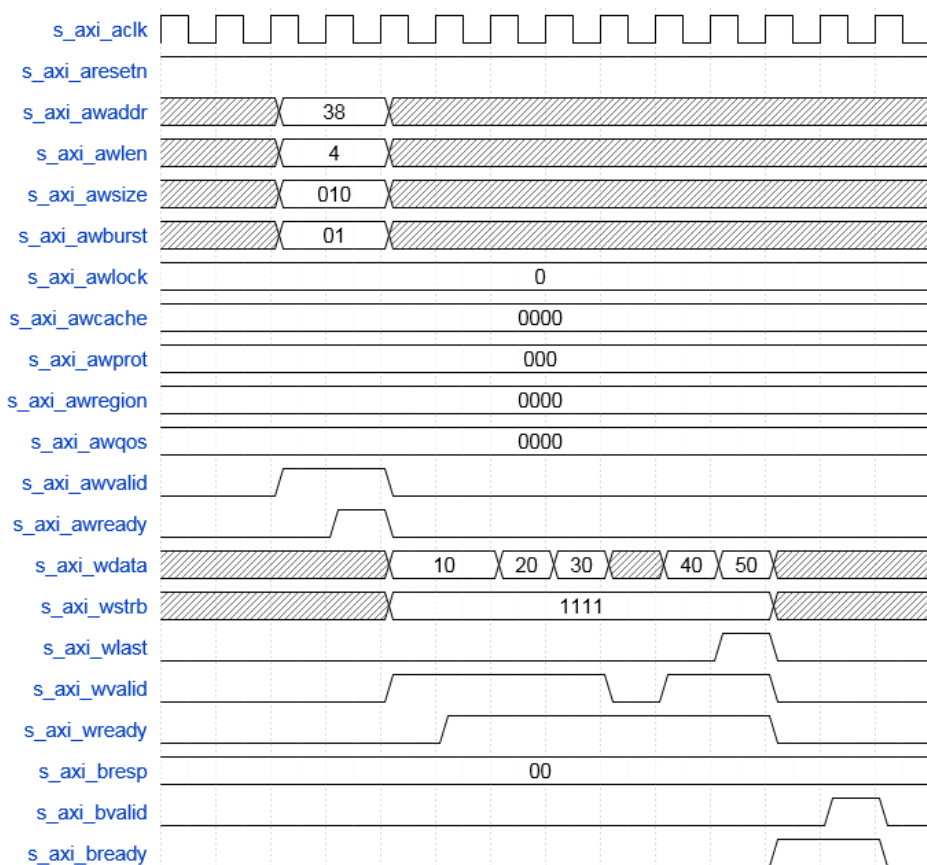
Talasni oblici AXI-Full transakcija

Na slici 6.5 prikazani su talasni oblici relevantnih signala AXI-Full protokola hipotetičkog IP jezgra u slučaju transakcije upisa bloka podataka. Transakcija upisa bloka podataka započinje tako što AXI master postavlja *s_axi_awaddr* magistralu na vrednost adrese lokacije unutar IP jezgra na koju želi upisati prvi podatak. U primeru sa slike 6.5, adresa lokacije na koju se želi upisati podatak je 0x38. Istovremeno AXI master postavlja i signale *s_axi_awlen*, *s_axi_awsz*, *s_axi_awburst*, *s_axi_awlock*, *s_axi_awcache*, *s_axi_awprot*, *s_axi_awregion* i *s_axi_awqos* na potrebne vrednosti. U našem primeru signal *s_axi_awlen* ima vrednost 0x4, što znači da AXI master želi da započne transakciju upisa bloka od 5 podataka. Signal *s_axi_awsz* ima vrednost

„010“, što znači da je svaki od podataka veličine 32 bita. Signal *s_axi_awburst* postavljen je na vrednost „01“, što znači da će prilikom blok transfera biti korišćen *increment* mod uvećavanja adrese. Preostali signali se obično ne koriste, te stoga imaju vrednost nula. AXI master takođe postavlja i signal *s_axi_awvalid* na jedinicu, signalizirajući IP jezgru da je nova validna adresa postavljena na *s_axi_awaddr* magistrali. Kada je IP jezgro spremno da prihvati adresu, postavlja *s_axi_awready* signal na jedinicu. U sledećem taktu AXI master postavlja *s_axi_awvalid* signal na nulu i uklanja adresu sa *s_axi_awaddr* magistrale, kao i sve ostale signale koji čine adresni kanal za čitanje.

Istovremeno ili kasnije, kao što je slučaj na slici 6.5, AXI master postavlja prvi podatak koji želi da upiše na adresiranu lokaciju unutar IP jezgra na *s_axi_wdata* magistralu (na slici 6.5 reč je o vrednosti 0x10). Takođe postavlja i indikaciju koji bajtovi unutar podatka su validni i trebaju da se upišu u adresiranu lokaciju, postavljajući odgovarajuće bitove *s_axi_wstrb* magistrale na jedinicu. AXI master postavlja i *s_axi_wvalid* signal na jedinicu, kako bi signalizirao IP jezgru da je na *s_axi_wdata* magistrali validni podatak. Kada je IP jezgro spremno da upiše podatak u adresiranu lokaciju, postavlja *s_axi_wready* signal na jedinicu. Sve dok se ovo ne desi AXI master drži tekući podatak na *s_axi_wdata* magistrali. Nakon detekcije visokog nivoa na *s_axi_wready* signalu, AXI master postavlja sledeći podatak iz bloka na *s_axi_wdata* magistralu i podiže *s_axi_wready* signal na jedinicu. Ova procedura se ponavlja sve do trenutka dok nije potrebno upisati poslednji podatak iz bloka. Ukoliko tokom upisa podataka iz bloka AXI master ili AXI slejv nisu u stanju da upišu sledeći podatak, oni će zaustaviti tekući proces upisa spuštanjem *s_axi_wvalid*, odnosno *s_axi_wready* signala na nulu, što se takođe može videti na slici 6.5, u trenutku koji sledi nakon upisa podatka 0x30. Kada AXI master postavi poslednji podatak iz bloka koji je potrebno upisati na *s_axi_wdata* magistralu (u primeru sa slike 6.5 to je peti podatak 0x50), on istovremeno postavlja i *s_axi_wlast* signal na jedinicu, kako bi IP jezgro znalo da je u toku upis poslednjeg podatka iz bloka. Nakon detekcije visokog nivoa na *s_axi_wready* signalu, AXI master uklanja podatke sa *s_axi_wdata* i *s_axi_wstrb* magistrala, i postavlja *s_axi_wvalid* i *s_axi_wlast* signale na nulu. Istovremeno i IP jezgro postavlja signal *s_axi_wready* na nulu.

Poslednji deo transakcije upisa bloka podataka predstavlja indikacija o finalnom statusu transakcije. Ovu informaciju generiše IP jezgro, preko statusnog kanala upisa. IP jezgro postavlja potrebni kod statusa na *s_axi_bresp* magistralu, istovremeno postavljajući i *s_axi_bvalid* signal na jedinicu i zatim čeka da *s_axi_bready* signal postane visok. AXI master signalizira da je spreman za prihvatanje finalnog statusa transakcije upisa postavljajući *s_axi_bready* signal na jedinicu. Kada, nakon postavljanja validnog finalnog statusa transakcije upisa, IP jezgro detektuje da je *s_axi_bready* signal visok, u narednom taktu uklanja ove informacije sa statusnog kanala upisa, na taj način kompletirajući AXI-Full transakciju upisa bloka podataka.

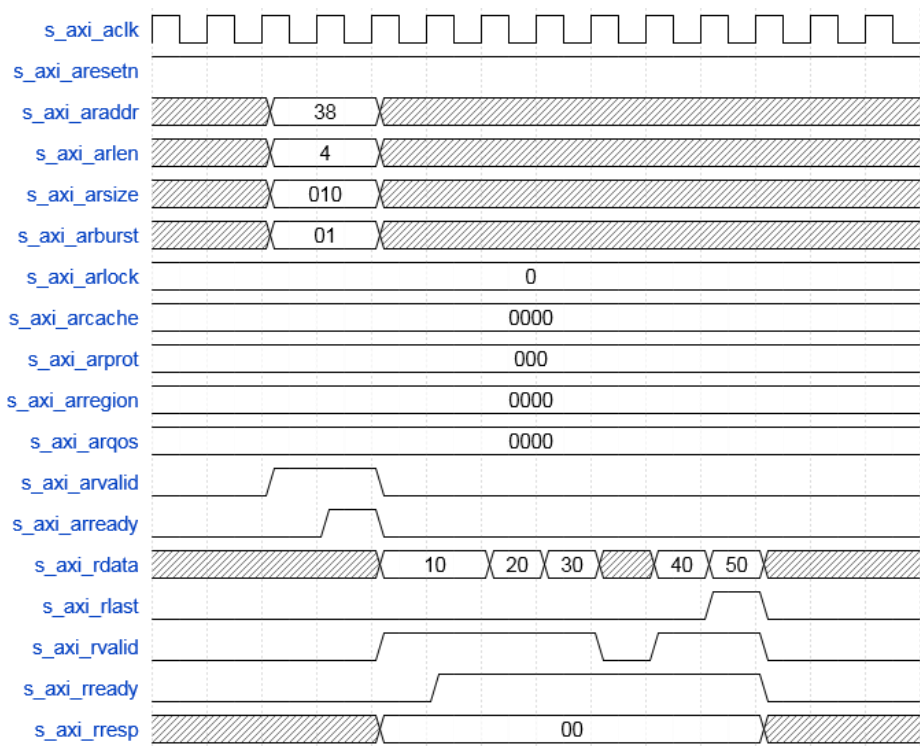


Slika 6.5. Transakcija upisa bloka podataka korišćenjem AXI-Full interfejsa

Na slici 6.6 prikazani su talasni oblici relevantnih signala AXI-Full protokola hipotetičkog IP jezgra u slučaju transakcije čitanja bloka podataka. Transakcija čitanja bloka podataka započinje tako što AXI master postavlja *s_axi_araddr* magistralu na vrednost adrese lokacije unutar IP jezgra sa koje želi pročitati prvi podatak. U primeru sa slike 6.6, adresa lokacije na koju se želi upisati podatak je 0x38. Istovremeno AXI master postavlja i signale *s_axi_arlen*, *s_axi_arsize*, *s_axi_arburst*, *s_axi_arlock*, *s_axi_arcache*, *s_axi_arprot*, *s_axi_arregion* i *s_axi_arqos* na potrebne vrednosti. U našem primeru signal *s_axi_arlen* ima vrednost 0x4, što znači da AXI master želi da započne transakciju čitanja bloka od 5 podataka. Signal *s_axi_arsize* ima vrednost „010“, što znači da je svaki od podataka veličine 32 bita. Signal *s_axi_arburst* postavljen je na vrednost „01“, što znači da će prilikom blok transfera biti korišćen *increment* mod uvećavanja adrese. Preostali signali se obično ne koriste, te stoga imaju vrednost nula. AXI master takođe postavlja i signal *s_axi_arvalid* na jedinicu, signalizirajući IP jezgru da je nova validna adresa postavljena na *s_axi_araddr* magistrali. Kada je IP jezgro spremno da prihvati adresu, postavlja *s_axi_arready* signal

na jedinicu. U sledećem taktu AXI master postavlja *s_axi_arvalid* signal na nulu i uklanja adresu sa *s_axi_araddr* magistrale, kao i sve ostale signale koji čine adresni kanal za čitanje.

Kada je IP jezgro spremno da prosledi prvi pročitani podatak sa adresirane lokacije AXI masteru postavlja ga na *s_axi_rdata* magistralu (na slici 6.6 prvi pročitani podatak ima vrednost 0x10). Istovremeno IP jezgro postavlja i *s_axi_rvalid* signal na jedinicu, signalizirajući AXI masteru da se na *s_axi_rdata* magistrali nalazi validan podatak. Ovaj podatak ostaje na *s_axi_rdata* magistrali sve dok AXI master ne potvrdi da ga je uspešno primio, postavljajući *s_axi_rready* signal na jedinicu. Nakon ove signalizacije IP jezgro u sledećem taktu postavlja sledeći podatak iz bloka na *s_axi_rdata* magistralu (na slici 6.6 to je podatak 0x20) i opet podiže *s_axi_rvalid* signal na jedinicu. Ova procedura se ponavlja sve do trenutka dok IP jezgro nije spremno poslati poslednji podatak iz bloka. Ukoliko tokom čitanja podataka iz bloka AXI master ili AXI slejv nisu u stanju da prihvate, odnosno pošalju, sledeći podatak, oni će zaustaviti tekući proces upisa spuštanjem *s_axi_rvalid*, odnosno *s_axi_rready* signala na nulu, što se takođe može videti na slici 6.6, u trenutku koji sledi nakon čitanja podatka 0x30. Kada IP jezgro postavi poslednji podatak iz bloka koji je potrebno pročitati na *s_axi_rdata* magistralu (u primeru sa slike 6.6 to je peti podatak 0x50), ono istovremeno postavlja i *s_axi_rlast* signal na jedinicu, kako bi AXI master znao da je u toku čitanje poslednjeg podatka iz bloka. Nakon detekcije visokog nivoa na *s_axi_rready* signalu, IP jezgro uklanja podatak sa *s_axi_rdata* magistrale, i postavlja *s_axi_rvalid* i *s_axi_rlast* signale na nulu. Istovremeno i AXI master postavlja signal *s_axi_rready* na nulu. Ovim je kompletirana AXI-Full transakcija čitanja bloka podatka.

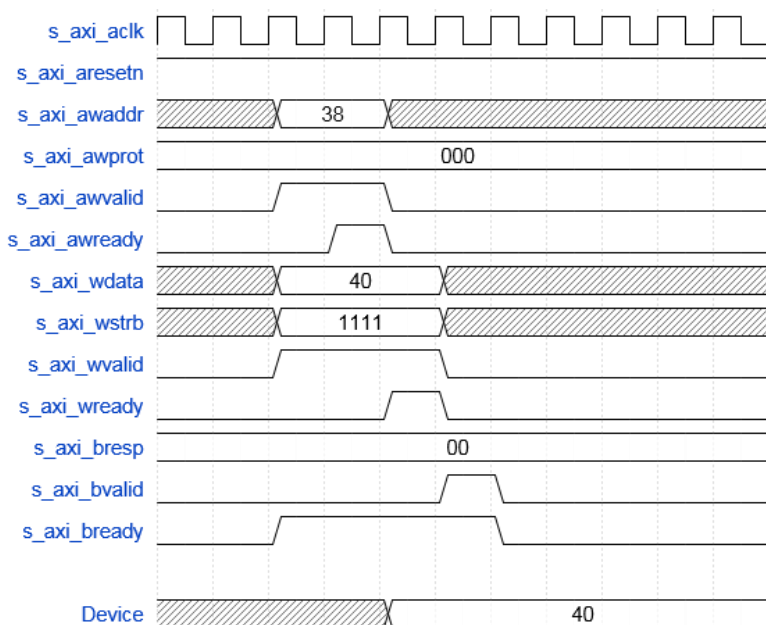


Slika 6.6. Transakcija čitanja bloka podataka korišćenjem AXI-Full interfejsa

Talasni oblici AXI-Lite transakcija

Na slici 6.7 prikazani su talasni oblici relevantnih signala slejv AXI-Lite protokola hipotetičkog IP jezgra, u slučaju transakcije upisa podatka. Transakcija upisa podatka započinje tako što AXI master postavlja `s_axi_awaddr` magistralu na vrednost adrese lokacije unutar IP jezgra na koje želi upisati podatak. U primeru sa slike 6.7, adresa lokacije na koju se želi upisati podatak je 0x38. Istovremeno AXI master postavlja i signal `s_axi_awvalid` na jedinicu, signalizirajući IP jezgru da je nova validna adresa postavljena na `s_axi_awaddr` magistrali. Kada je IP jezgro spremno da prihvati adresu, postavlja `s_axi_awready` signal na jedinicu. U sledećem taktu AXI master postavlja `s_axi_awvalid` signal na nulu i uklanja adresu sa `s_axi_awaddr` magistrale. Istovremeno, ili i ranije kao što je slučaj na slici 6.7, AXI master postavlja podatak koji želi da upiše na adresiranu lokaciju unutar IP jezgra na `s_axi_wdata` magistralu (na slici 6.7 reč je o vrednosti 0x40). Takođe postavlja i indikaciju koji bajtovi unutar podatka su validni i trebaju da se upišu u adresiranu lokaciju, postavljajući odgovarajuće bitove `s_axi_wstrb` magistrale na jedinicu. AXI master postavlja i `s_axi_wvalid` signal na jedinicu, kako bi signalizirao IP jezgru da je na `s_axi_wdata` magistrali validni podatak. Kada je IP jezgro spremno da upiše podatak u adresiranu lokaciju, postavlja `s_axi_wready` signal na jedinicu, i istovremeno upisuje podatak (na slici 6.7 podatak 0x40 se upisuje u hipotetički *Device* registar, jer se on nalazi na adresi 0x38). Nakon detekcije visokog

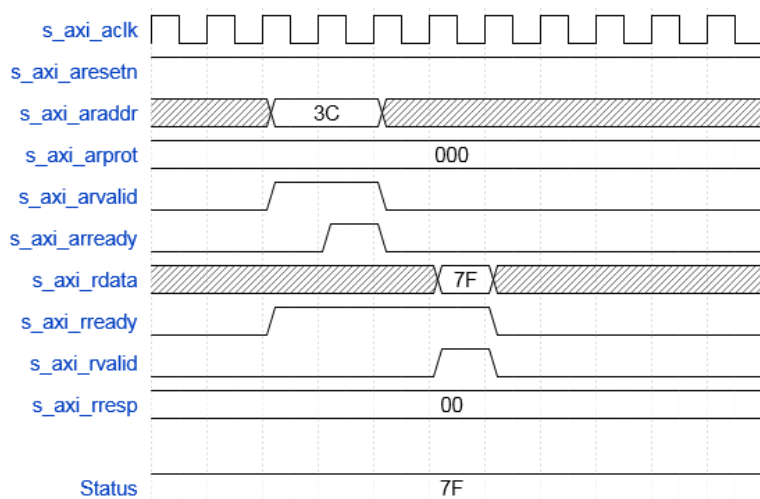
nivoa na `s_axi_wready` signalu, AXI master uklanja podatke sa `s_axi_wdata` i `s_axi_wstrb` magistrala, i postavlja `s_axi_wvalid` signal na nulu. Istovremeno i IP jezgro postavlja signal `s_axi_wready` na nulu. Poslednji deo transakcije upisa podatka predstavlja indikacija o finalnom statusu transakcije. Ovu informaciju generiše IP jezgro, preko statusnog kanala upisa. IP jezgro postavlja potrebni kod statusa na `s_axi_bresp` magistralu, istovremeno postavljajući i `s_axi_bvalid` signal na jedinicu i zatim čeka da `s_axi_bready` signal postane visok. AXI master signalizira da je spreman za prihvatanje finalnog statusa transakcije upisa, postavljajući `s_axi_bready` signal na jedinicu. Kada, nakon postavljanja validnog finalnog statusa transakcije upisa, IP jezgro detektuje da je `s_axi_bready` signal visok, u narednom taktu uklanja ove informacije sa statusnog kanala upisa, na taj način kompletirajući AXI-Lite transakciju upisa podatka.



Slika 6.7. Transakcija upisa podatka korišćenjem AXI-Lite interfejsa

Na slici 6.8 prikazani su talasni oblici relevantnih signala slejv AXI-Lite protokola hipotetičkog IP jezgra, u slučaju transakcije čitanja podatka. Transakcija čitanja podatka započinje tako što AXI master postavlja `s_axi_araddr` magistralu na vrednost adrese lokacije unutar IP jezgra sa koje želi pročitati podatak. U primeru sa slike 6.8, adresa lokacije sa koje se želi pročitati podatak je 0x3C. Istovremeno AXI master postavlja i signal `s_axi_arvalid` na jedinicu, signalizirajući IP jezgru da je nova validna adresa postavljena na `s_axi_araddr` magistrali. Kada je IP jezgro spremno da prihvati adresu, postavlja `s_axi_arready` signal na jedinicu. U sledećem taktu AXI master postavlja `s_axi_arvalid` signal na nulu i uklanja adresu sa `s_axi_araddr` magistrale. Istovremeno, ili i ranije kao što je slučaj na slici 6.8, AXI master postavlja i `s_axi_rready` signal na

jedinicu, kako bi signalizirao IP jezgru da je spreman za prihvrat podatka. Kada je IP jezgro spremno da prosledi pročitani podatak sa adresirane lokacije AXI materu, postavlja ga na *s_axi_rdata* magistralu (na slici 6.8 pročitani podatak predstavlja sadržaj hipotetičkog *Status* registra, i ima vrednost 0x7F). Istovremeno IP jezgro postavlja i *s_axi_rvalid* signal na jedinicu, signalizirajući AXI masteru da se na *s_axi_rdata* magistrali nalazi validan podatak. Nakon detekcije visokog nivoa na *s_axi_rready* signalu, IP jezgro uklanja podatke sa *s_axi_rdata* magistrale i postavlja *s_axi_rvalid* signal na nulu. Ovim je kompletirana AXI-Lite transakcija čitanja podatka.



Slika 6.8. Transakcija čitanja podatka korišćenjem AXI-Lite interfejsa

Poslednja varijanta AXI4 interfejsa koju ćemo predstaviti je *AXI-Stream* protokol. Kao što je već rečeno, *AXI-Stream* protokol ne koristi adresne informacije kao indikaciju na koju lokaciju treba smestiti ili pročitati podatak. Umesto toga *AXI-Stream* jednostavno prenosi podatke kao jedan kontinualni tok (otuda i njegovo ime, *AXI-Stream*). Podaci se prenose u blokovima i kao takvi se zatim i obrađuju. Ovo je vrlo čest slučaj u audio, video aplikacijama, prenosu podataka, itd. Sa druge strane, nepostojanje adresne informacije omogućava da *AXI-Stream* protokol bude daleko jednostavniji od *AXI-Full*, pa čak i od *AXI-Lite* protokola. Obratite pažnju da u slučaju *AXI-Stream* protokola nema smisla govoriti o operaciji čitanja ili upisa podataka. U ovom slučaju podaci se prosto prenose od *AXI-Stream* mastera ka *AXI-Stream* slejvu, i to je jedina moguća transakcija u ovom protokolu.

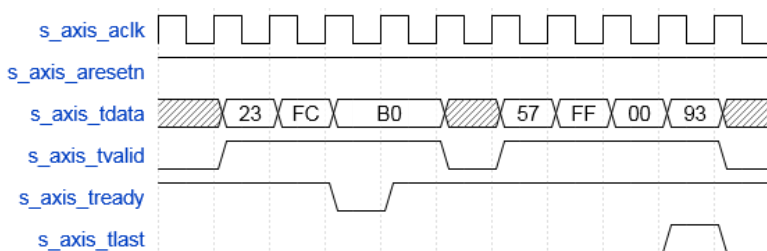
Tabela 6.2 prikazuje spisak svih signala koji čine *AXI-Stream* interfejs, u slučaju AXI master komponente zajedno sa kratkim objašnjenjem svakog od signala. U slučaju AXI slejv komponente *AXI-Stream* interfejs bi se sastojao iz istih signala, samo bi smer svakog od njih bio obrnut.

Naziv signala	Opis signala
<i>m_axis_aclk</i>	Sinhronizacioni signal AXI interfejsa.
<i>m_axis_aresetn</i>	Reset signal AXI interfejsa.
<i>m_axis_tdata</i>	Magistrala podataka.
<i>m_axis_tvalid</i>	Indikacija da je tekući podatak validan.
<i>m_axis_tready</i>	Indikacija da je AXI slejv spreman da primi sledeći podatak.
<i>m_axis_tlast</i>	Indikacija da je tekući podatak ujedno i poslednji podatak koji je potrebno preneti.

Tabela 6.2. Signali AXI-Stream interfejsa u slučaju AXI master komponente

Talasni oblici AXI-Stream transakcija

Kako se u slučaju AXI-Stream interfejsa podaci prenose samo u jednom smeru, od AXI-Stream mastera ka AXI-Stream slejvu, na slici 6.9 prikazani su talasni oblici signala AXI-Stream interfejsa AXI-Stream mastera prilikom transakcije prenosa bloka podataka. Proces prenosa započinje kada AXI-Stream master postavi prvi podatak na *m_axis_tdata* magistralu i podigne *m_axis_tvalid* signal na jedinicu. Postavljeni podatak ostaje na *m_axis_tdata* magistrali sve dok AXI-Stream slejv modul ne signalizira AXI-Stream masteru da je spreman za prihvatanje sledećeg podatka, postavljajući *m_axis_tready* signal na jedinicu, što se lepo može videti na slici 6.9. Kada AXI-Stream master uoči da je AXI-Stream slejv spreman za prijem sledećeg podatka, on u sledećem taktu na *m_axis_tdata* magistralu postavlja sledeći podatak. Ukoliko iz nekog razloga AXI-Stream master nije u stanju da postavi sledeći podatak na *m_axis_tdata* magistralu, on ovu činjenicu signalizira AXI-Stream slejvu spuštanjem *m_axis_tvalid* podatka. Opisani postupak prenosa podataka nastavlja se dok se ne dođe do poslednjeg podatka koji je potrebno preneti. Kada AXI-Stream master postavi poslednji podatak na *m_axis_tdata* magistralu i označi ga kao validan, postavljajući *m_axis_tvalid* signal na jedinicu, istovremeno postavlja i *m_axis_tlast* signal na jedinicu, signalizirajući da je reč o poslednjem podatku iz tekućeg paketa. Nakon što AXI-Stream slejv preuzme i ovaj podatak, postavljajući *m_axis_tready* signal na jedinicu, tekuća transakcija prenosa bloka podataka je završena.



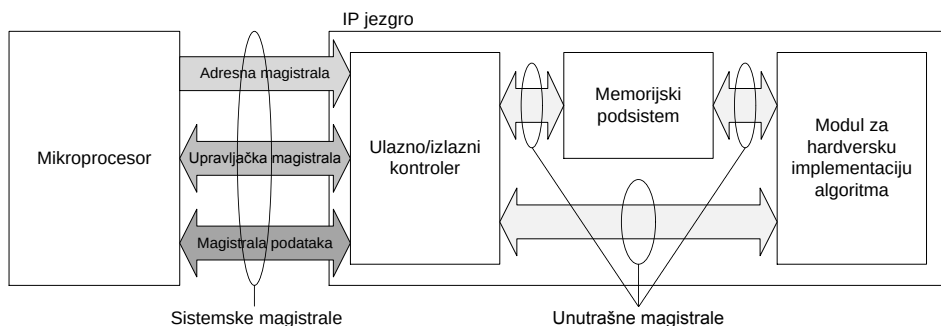
Slika 6.9. Prenos bloka podatka korišćenjem AXI-Stream interfejsa

6.2. Tipična struktura IP jezgra

Kao što je ranije već rečeno, u cilju povećavanja mogućnosti za ponovno korišćenje IP jezgra poželjno je da IP jezgro koristi neke od standardnih SoC interfejsa. Iako bi bilo moguće uključiti protokol odabrane SoC magistrale u proces projektovanja digitalnog sistema pomoću RT metodologije (dodajući odgovarajuće portove u interfejs sistema i modifikujući ASMD dijagram i *datapath*) ovaj pristup se uglavnom izbegava. Glavni razlog leži u činjenici da ovako projektovan digitalni sistem vrlo teško može da se prilagodi karakteristikama nekog drugog protokola SoC magistrale, limitirajući na taj način univerzalnost razvijenog sistema.

Takođe, pošto će IP jezgro biti korišćeno unutar SoC sistema, koji su po pravilu centrirani oko jednog ili više procesora, IP jezgro će zapravo predstavljati periferijsku jedinicu za procesor. Kako procesor periferijske jedinice „vidi“ kao skup registara, poželjno bi bilo da se i komunikacija sa IP jezgrom odvija na isti način, preko skupa registara, ili memorijskih bafera.

Način da se ovi problemi adekvatno reše jeste da se oko digitalnog sistema projektovanog pomoću RT metodologije „obavi“ još jedan nivo hijerarhije, koji će sadržati memorijski podsistem (koji će biti dostupan procesoru) i posebno projektovan kontroler odabrane SoC magistrale (preko koje će procesor moći da komunicira sa IP jezgrom). Standardna struktura IP jezgra projektovanog na ovaj način, prikazana je na slici 6.10.



Slika 6.10. Tipična struktura IP jezgra

Sa slike 6.10 može se videti da se na najvišem nivou hijerarhije IP jezgro obično sastoji iz tri modula:

- **Modula za hardversku implementaciju algoritma** – u okviru kojega je implementiran odabrani algoritam obrade podataka primenom RT metodologije. Ovaj modul projektuje se tako da koristi interfejs za komunikaciju sa okruženjem koji je najprikladniji za korišćenje u sprezi sa algoritmom koji se implementira. Obično je reč o potpuno nestandardnom tipu

interfejsa, kao što je bio slučaj u svim primerima iz glava 4 i 5. Projektant ima potpunu slobodu da odabere tip interfejsa koji mu najviše odgovara.

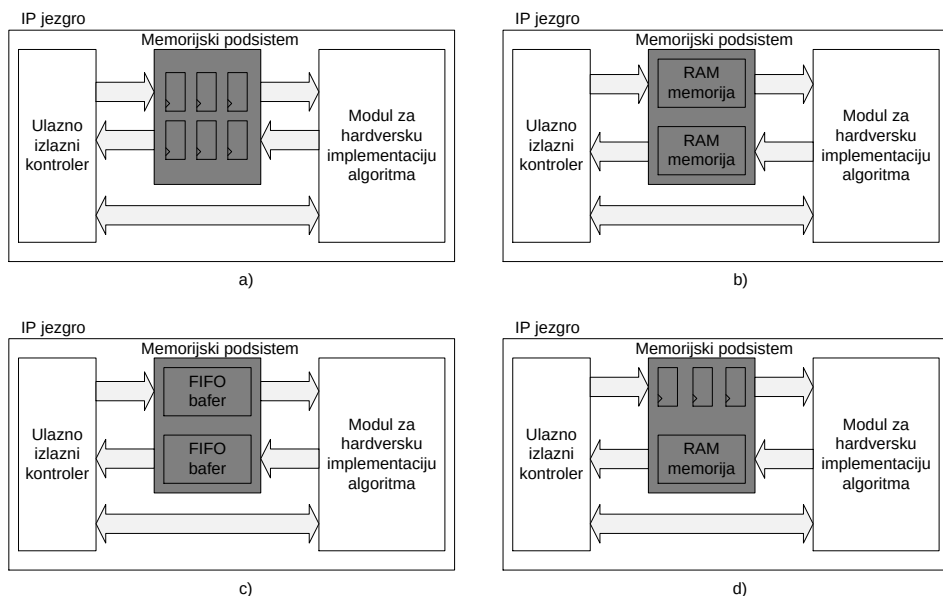
- **Memorijskog podsistema** – u okviru kojega su implementirani svi potrebni memorijski moduli neophodni za smeštanje ulaznih i izlaznih podataka koji se obrađuju ili predstavljaju rezultat obrade pomoću hardverski implementiranog algoritma, kao i za kontrolu sistema koji implementira odabrani algoritam. Ovaj memorijski sistem može biti organizovan na različite načine, o čemu će biti više reči kasnije.
- **Ulazno/izlaznog kontrolera** – koji implementira protokol odabrane SoC magistrale.

Ovakvom strukturom IP jezgra postiže se njegova velika fleksibilnost. Modul za hardversku implementaciju algoritma može se projektovati potpuno nezavisno od protokola SoC magistrale. Ulazno/izlazni kontroler predstavlja neku vrstu „mosta“ između nekog od standardnih protokola SoC magistrale sa jedne strane (na primer *AXI-Full*, *AXI-Lite*, *AXI-Stream*, itd.), i specifičnog protokola koji koristi modul za hardversku implementaciju algoritma. Projektovanjem različitih ulazno/izlaznih kontrolera moguće je prostim uključivanjem potrebnog kontrolera obezbediti veliku fleksibilnost razvijenog IP jezgra u pogledu korišćene SoC magistrale. Ovaj pristup u znatnoj meri povećava šanse za ponovno korišćenje razvijenog IP jezgra. HDL model u kojem je definisana struktura sistema sa slike 6.10 naziva se i „omotačem“ (*Wrapper*) oko modula za hardversku implementaciju algoritma. Korišćenjem različitih „omotača“ (koji se razlikuju u korišćenom ulazno/izlaznom kontroleru) razvijeni modul za hardversku implementaciju algoritma se uz minimalne modifikacije može prilagoditi specifičnostima potrebne SoC magistrale. Na ovaj način prilikom migracije sa jednog SoC protokola na drugi nije potrebno modifikovati sam RT dizajn modula za hardversku implementaciju algoritma, pa ga samim tim nije potrebno ni ponovo verifikovati, što sve štedi dragoceno vreme. Sve što treba uraditi jeste projektovati odgovarajući ulazno/izlazni kontroler, što najčešće predstavlja daleko manje zahtevan zadatak.

Kao što je rečeno ranije, memorijski podsistem sa slike 6.10 ima zadatak da implementira memorijske resurse neophodne za čuvanje vrednosti ulaznih i izlaznih argumenata algoritma koji je implementiran u hardveru. Kako ulazno/izlazni argumenti mogu biti različite strukture, postoji veliki broj načina na koji se može projektovati memorijski podsistem, od kojih su najčešći prikazani na slici 6.11.

U slučaju kada su ulazni i izlazni argumenti algoritma skalarni tipovi, najjednostavniji način za implementaciju memorijskog podsistema je korišćenjem odgovarajućeg broja individualnih registara, kao što je prikazano na slici 6.11a. Svaki registar odgovara jednom ulaznom, odnosno izlaznom argumentu algoritma. Na primer, ukoliko bismo želeli da „*Repetitive-Addition*“ množač razvijen u primeri 4.1 zapakujemo u formi IP jezgra, najjednostavniji način za smeštanje vrednosti ulaznih i izlaznih argumenata bio bi baziran na korišćenju registara. „*Repetitive-Addition*“ množač ima dva skalarna ulaza, *a_in* i *b_in*, preko kojih se specificiraju vrednosti operanada u operaciji množenja. Množač ima jedan skalarni izlaz, *r_out*, čija vrednost predstavlja rezultat množenja

operanada, zadatih na portovima a_in i b_in . Korišćenjem tri odvojena registra može se projektovati potrebiti memorijski podsistem. Dodavanjem još dva registra može se obezbediti jednostavna kontrola *start* i *ready* signala. U ovom primeru kompletni memorijski podsistem sastojao bi se od ukupno pet registra.



Slika 6.11. Neke od mogućih organizacija memorijskog podsistema unutar IP jezgra: a) korišćenjem individualnih registara, b) korišćenjem memorija, c) korišćenjem FIFO bafera, d) korišćenjem kombinovanih tipova memorijskih resursa

U slučaju kada su ulazni ili izlazni argumenti implementiranog algoritma složeni tipovi podataka (nizovi, matrice, strukture, itd.), prirodan način za implementaciju memorijskog podsistema bio bi baziran na korišćenju odgovarajućeg broja RAM memorija sa odgovarajućim brojem pristupa za čitanje i upis, kao što je prikazano na slici 6.11b. Na primer, u slučaju sistema za implementaciju „Naive“ algoritma množenja dve matrice, matrice A , B i C predstavljaju ulazne, odnosno izlazne argumente složenog tipa, pa je najjednostavniji način za njihovu hardversku implementaciju baziran na korišćenju odgovarajućih memorija. U ovom slučaju imamo veći broj mogućih konfiguracija, od kojih su neke diskutovane u okviru primera 4.3 (videti slike 4.19 i 4.21).

U slučaju kada su ulazni ili izlazni argumenti implementiranog algoritma zapravo tokovi podataka, gde se podaci obrađuju u redosledu u kojem dospevaju do sistema, najprirodniji način za njihovu hardversku implementaciju bio bi pomoću FIFO bafera, kao što je prikazano na slici 6.11c.

Na kraju, što je i najčešći slučaj u praksi, ulazni i izlazni argumenti implementiranog algoritma mogu biti različite prirode (neki mogu biti skalarni, neki složeni, neki mogu biti tokovi podataka, ...). U ovom slučaju memorijski podsistem projektuje se tako da sadrži različite tipove memorijskih resursa (registre, RAM memorije, FIFO bafere, ...), birajući optimalni resurs za svaki od ulaznih i izlaznih argumenata implementiranog algoritma pojedinačno, kao što je prikazano na slici 6.11d. Na primer, u slučaju „Naive“ množača matrica, projektovanog u primeru 4.3, najpogodnija struktura memorijskog podsistema uključivala bi sledeće memorijske resurse:

- Tri jednodostupne RAM memorije za smeštanje matrica A , B i C
- Tri registra za smeštanje dimenzija matrica A , B , i C (parametri n_{in} , p_{in} i m_{in})
- Jedan registar za smeštanje vrednosti ulaznog *start* signala
- Jedan registar za smeštanje vrednosti izlaznog *ready* signala

6.3. Kreiranje IP jezgra pomoću Xilinx IP Packager alata

Xilinx Vivado Design Suite alat pruža mogućnost automatskog generisanja ulazno/izlaznih kontrolera za AXI-Full, AXI-Lite i AXI-Stream protokole. Takođe, Vivado Design Suite alat pruža i mogućnost pakovanja svih potrebnih fajlova koji čine jedno IP jezgro u standardni format koji Vivado Design Suite alat prepoznaje. Na ovaj način korisnik može generisati nova IP jezgra, koja zatim može dodati u IP katalog Vivado Design Suite alata i koristiti ih po potrebi. Reč je o IP Packager alatu. Način rada sa IP Packager alatom biće ilustrovan na primeru pakovanja „Naive“ množača matrica, projektovanog u primeru 4.3.

Primer 6.1: Kreiranje IP jezgra „Naive“ množača matrica

Prvi korak prilikom kreiranja IP jezgra jeste da se definišu njegovi interfejsi i projektuje ulazno/izlazni kontroler (ili kontroleri) u zavisnosti od odabrane SoC magistrale koja se planira koristiti, organizuje struktura memorijskog podsistema i formira strukturni model koji povezuje tri komponente u jednu celinu.

Kako ćemo mi koristiti IP Packager alat, koji ima mogućnost automatskog generisanja AXI kompatibilnih ulaznog/izlaznih kontrolera, projektovaćemo IP jezgro „Naive“ množača sa AXI interfejsima.

Što se tiče strukture memorijskog podsistema, u slučaju „Naive“ množača matrica, projektovanog u primeru 4.3, najpogodnija struktura uključivala bi sledeće memorijske resurse:

- Tri jednodostupne RAM memorije za smeštanje matrica A , B i C

- Tri registra (n , m i p) za smeštanje dimenzija matrica A , B , i C (parametri n_in , p_in i m_in)
- Jedan registar (cmd) za smeštanje vrednosti ulaznog $start$ signala
- Jedan registar ($status$) za smeštanje vrednosti izlaznog $ready$ signala

NAPOMENA: Radi olakšanog razvoja sprežne logike između različitih modula IP jezgra „Naive“ množača matrica, memorije A , B i C će biti projektovane tako da budu dvopristupne. Na ovaj način se jedan pristup može iskoristiti za vezu između ulazno/izlaznog kontrolera a drugi pristup za vezu sa modulom za hadrversku implementaciju „Naive“ algoritma množenja dve matrice.

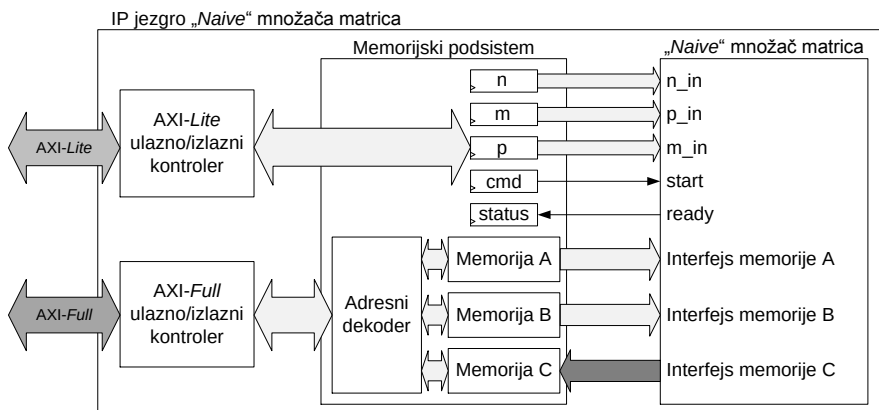
Kako memorijski podsistem u ovom slučaju uključuje i registre i RAM memorije, najlogičniji način za njegovo povezivanje na AXI magistrale je da se koriste dva AXI protokola:

- *AXI-Lite* protokol – ovaj protokol namenjen je za komunikaciju sa individualnim registrima unutar IP jezgra, pa će u ovom slučaju biti iskorišćen za pristup n , m , p , cmd i $status$ registrima unutar memorijskog podsistema IP jezgra „Naive“ množača matrica
- *AXI-Full* protokol – ovaj protokol namenjen je za komunikaciju sa memorijama unutar IP jezgra, pa će u ovom slučaju biti iskorišćen za pristup memorijama A , B i C unutar memorijskog podsistema IP jezgra „Naive“ množača matrica.

Imajući ovo i vidu, IP jezgro „Naive“ množača matrica sadržaće dva ulazno/izlazna kontrolera:

- *AXI-Lite* kontroler, za pristup registrima i
- *AXI-Full* kontroler, za pristup podacima unutar memorija.

Na osnovu svega izloženog projektovano IP jezgro „Naive“ množača matrica imaće strukturu prikazanu na slici 6.12.



Slika 6.12. Struktura IP jezgra „Naive“ množača matrica sa AXI interfejsima

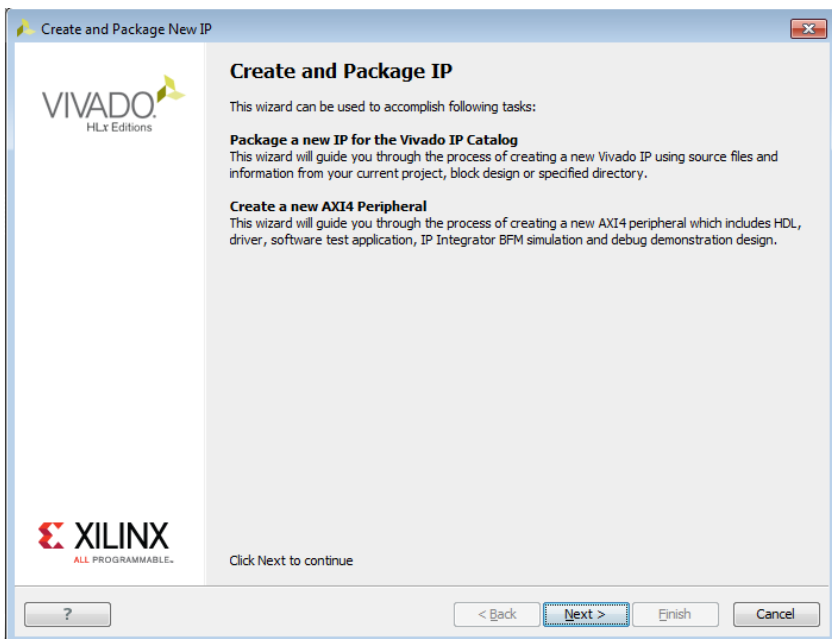
Kao što je već rečeno, IP Packager alat može automatski generisati AXI-Lite i AXI-Full kontrolere, a takođe može generisati i kostur strukturnog HDL modela IP jezgra na najvišem nivou hijerarhije.

Prvi korak u postupku automatskog generisanja potrebnih AXI kontrolera jeste da se kreira novi projekat u okviru Vivado alata, ponavljajući korake iz poglavlja 1.3.

NAPOMENA: Prilikom kreiranja projekta, ime projekta treba da bude *matrix_multiply*.

Kreiranje potrebnih AXI interfejsa

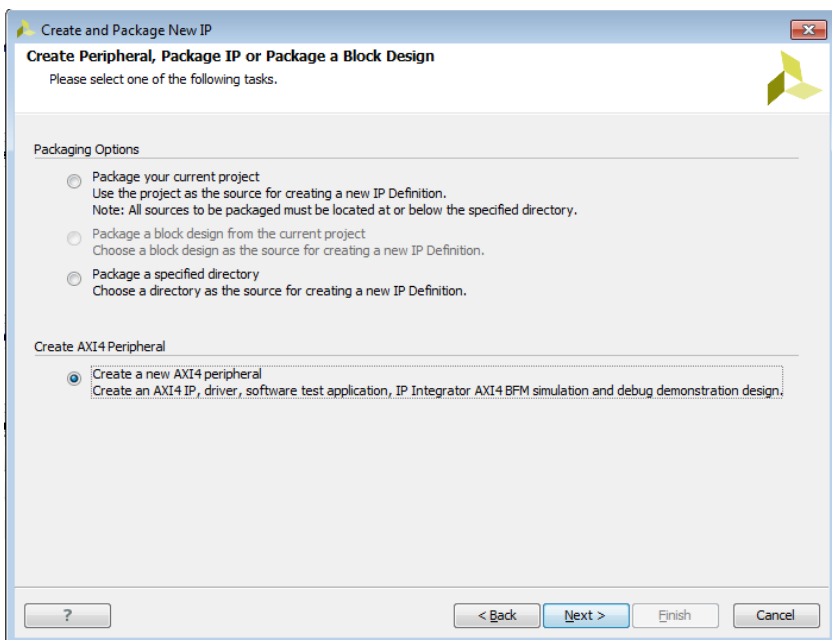
Nakon što se kreira novi Vivado projekat, pokretanjem opcije Tools → Create and Package IP ..., pokreće se Vivado IP Packager alat i otvara se prozor prikazan na slici 6.13.



Slika 6.13. Početni prozor IP Packager alata

Pritiskom na dugme *Next* prelazi se na sledeći prozor, prikazan na slici 6.14, gde možemo izabrati jednu od tri moguće akcije:

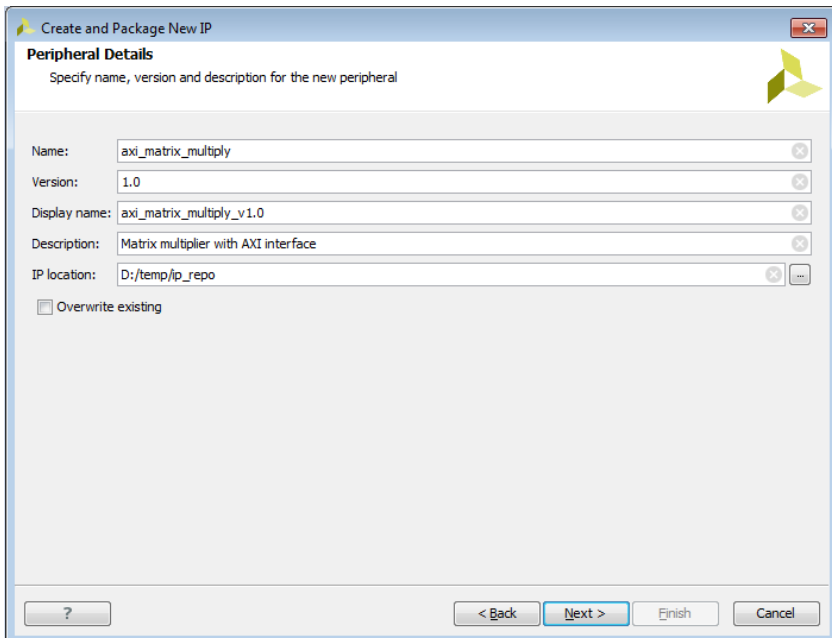
- *Package your current project* – pomoću ove akcije tekući projekat otvoren u Vivado Design Suite alatu pakuje se u formu IP jezgra
- *Package a specified directory* – pomoću ove akcije može se specificirati direktorijum koji sadrži sve fajlove koje želimo da zapakujemo u formu IP jezgra
- *Create a new AXI4 peripheral* – započinje se razvoj novog IP jezgra koje će koristiti jedan ili više AXI interfejsa



Slika 6.14. Moguće akcije IP Packager alata

Prve dve opcije korisne su kada već imamo sve izvorne fajlove koje želimo da zapakujemo u novo IP jezgro. Takođe, ukoliko ne želimo da koristimo mogućnost automatskog generisanja AXI kontrolera, treba odabrati neku od prve dve opcije.

U našem slučaju mi želimo da iskoristimo IP Packager alat da nam automatski generiše jedan AXI-*Lite* i jedan AXI-*Full* kontroler, pa ćemo izabrati poslednju opciju, kao što je prikazano na slici 6.14. Nakon pritiska dugmeta *Next*, prelazimo na „*Peripheral Details*“ prozor, prikazan na slici 6.15.



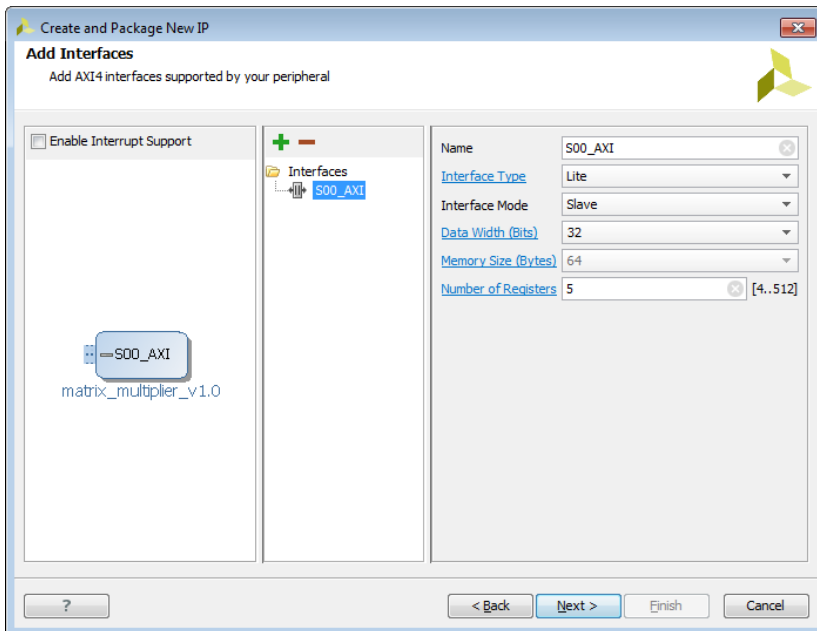
Slika 6.15. „Peripheral Details“ prozor IP Packager alata

„Peripheral Details“ prozor omogućava nam da definišemo osnovne podatke o IP jezgru koje razvijamo:

- *Name* – predstavlja ime IP jezgra koje razvijamo
- *Version* – predstavlja verziju IP jezgra koje razvijamo
- *Display name* – predstavlja ime IP jezgra koje će biti prikazivano unutar ostalih Vivado alata
- *Description* – predstavlja kratak opis funkcionalnosti IP jezgra koje razvijamo. Ovaj opis će se pojaviti kada selektujemo IP jezgro unutar IP kataloga Vivado Design Suite alata
- *IP location* – mesto na disku gde će biti smešteno zapakovano jezgro

U našem primeru polja unutar „Peripheral Details“ prozora treba da imaju vrednosti kao na slici 6.15, sa izuzetkom polja „IP Location“. Mesto na disku gde će biti smešteno konačno zapakovano IP jezgro može se odabrati proizvoljno, po sopstvenom izboru.

Pritiskom na dugme *Next* prelazimo na „Add Interfaces“ prozor, prikazan na slici 6.16.



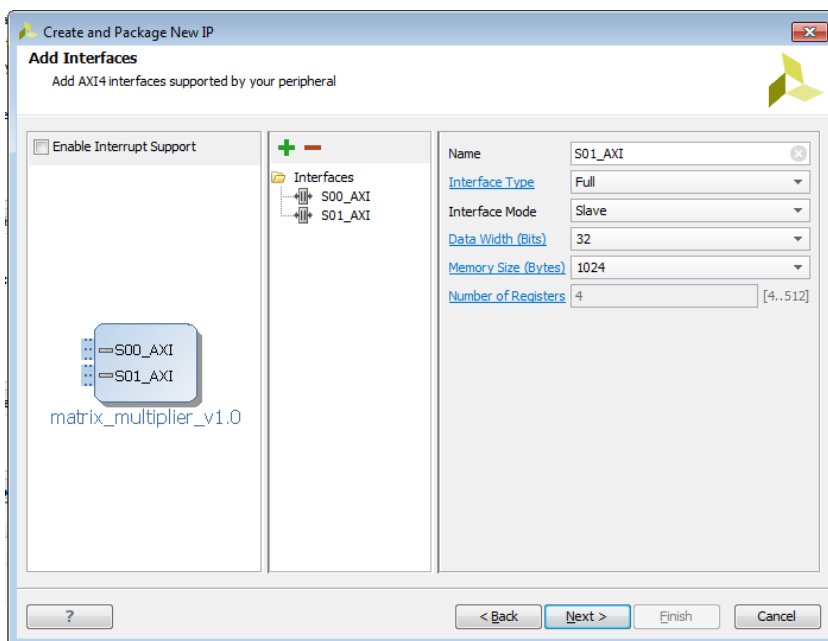
Slika 6.16. „Add Interfaces“ prozor IP Packager alata, sa potrebnim podešavanjima za definisanje AXI-Lite interfejsa „Naive“ množača matrica

„Add Interfaces“ prozor služi za kreiranje novih AXI interfejsa i definisanje njihovih parametara. Lista parametara koje je potrebno definisati zavisi od tipa odabranog AXI protokola (AXI-Lite, AXI-Full ili AXI-Stream). U našem primeru potrebno je da kreiramo jedan AXI-Lite i jedan AXI-Full interfejs. Kada se „Add Interfaces“ prozor prvi put otvori, u njemu se već nalazi kreiran jedan interfejs, AXI-Lite, sa inicijalnim podešavanjima. U slučaju kreiranja AXI-Lite ili AXI-Full interfejsa potrebno je definisati sledeće parametre:

- *Name* – ime koje će biti korišćeno kao prefiks u imenima svih signala koji čine interfejs
- *Interface Type* – tip AXI interfejsa koji se kreira
- *Interface Mode* – mod AXI interfejsa, master ili slejv. U slučaju IP jezgara većina AXI interfejsa će biti slejv tipa.
- *Data Width* – širina magistrale podataka AXI interfejsa
- *Memory Size* – veličina memorijskog prostora koji je alocirao IP jezgru. U slučaju AXI-Lite interfejsa ova opcija je neaktivna.
- *Number of Registers* – broj individualnih registara kojima se može pristupiti preko AXI interfejsa. U slučaju AXI-Full interfejsa ova opcija je neaktivna.

Prvo ćemo kreirati AXI-Lite interfejs. Potrebna podešavanja AXI-Lite interfejsa prikazana su na slici 6.16. Obratite pažnju da smo parametar „Number of Registers“ postavili na vrednost pet, jer u našem primeru želimo da preko AXI-Lite interfejsa pristupimo registrima n , m , p , cmd i $status$ (vidi sliku 6.12).

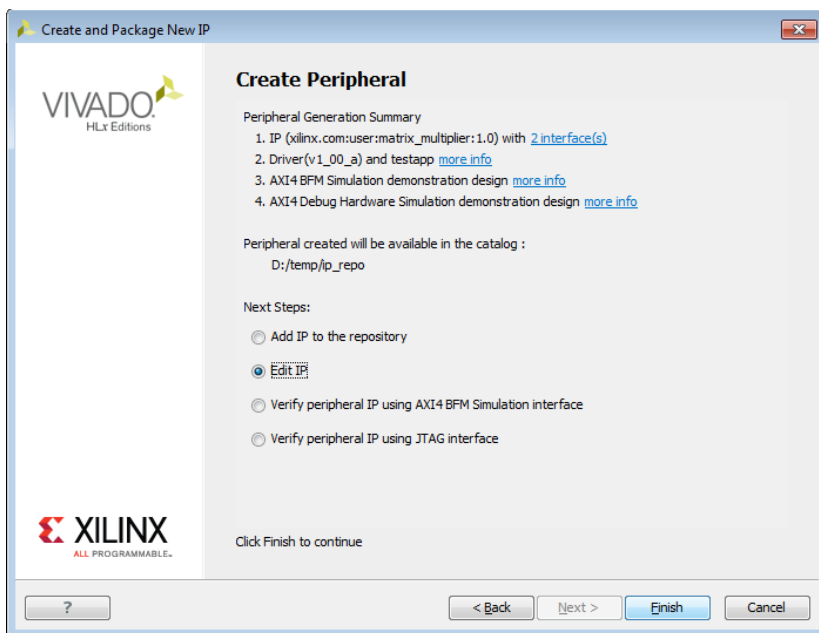
Nakon što smo podesili parametre AXI-Lite interfejsa, moramo dodati i definisati još jedan, AXI-Full, interfejs. Pritiskom na zeleni plus (u gornjem levom delu prozora) dodaje se sledeći interfejs. Ovaj interfejs treba da bude AXI-Full tipa, sa podešavanjima prikazanim na slici 6.17.



Slika 6.17. „Add Interfaces“ prozor IP Packager alata, sa potrebnim podešavanjima za definisanje AXI-Full interfejsa „Naive“ množača matrica

Sa slike 6.17 možemo primetiti da smo izabrali maksimalnu vrednost za veličinu adresnog prostora asociranog IP jezgra, od ukupno 1024 lokacije. Ovaj podatak predstavlja gornju granicu postavljenu na zbir veličina memorija za smeštanje matrica A , B i C . Radi lakšeg projektovanja adresnog dekodera, potrebnog unutar memorijskog podsistema (vidi sliku 6.12), usvojimo pretpostavku da je veličina svake od ove tri memorije jednaka i iznosi 256 lokacija. Ovaj podatak biće potreban kasnije, prilikom definisanja konfiguracionih parametara IP jezgra.

Nakon što smo definisali dva neophodna AXI interfejsa, pritiskom na dugme *Next* prelazimo na poslednji, „Create Peripheral“ prozor IP Packager alata.



Slika 6.18. „Create Peripheral“ prozor IP Packager alata

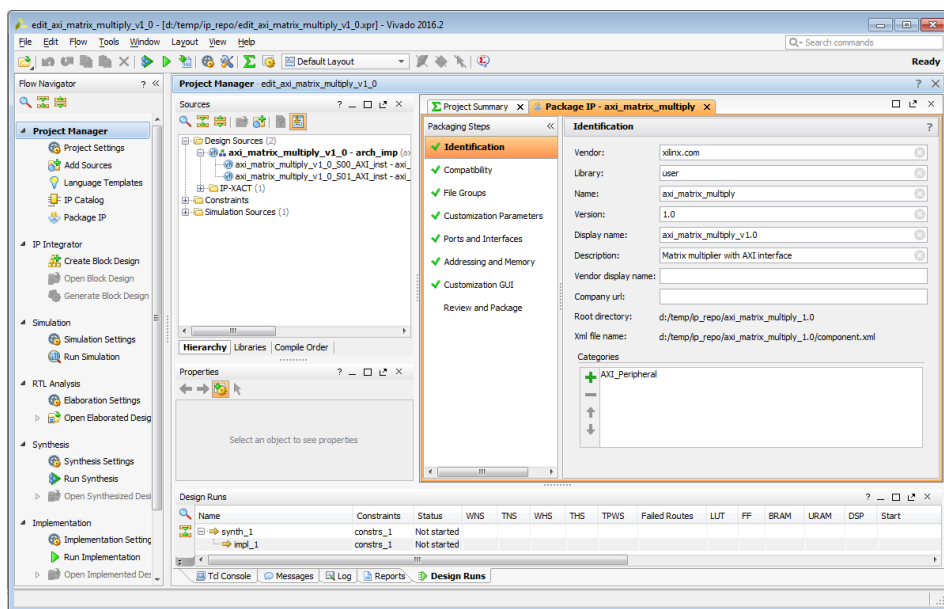
„Create Peripheral“ prozor sumira podešavanja i interfejsa koje smo definisali za IP jezgro koje razvijamo i nudi na sledeće opcije:

- *Add IP to the repository* – završava se proces pakovanja IP jezgra i ono se dodaje u IP repozitorijum Vivado Design Suite alata
- *Edit IP* – prelazi se u režim izmene karakteristika IP jezgra. Ovo je najčešće korišćena opcija, jer IP jezgro projektovano do sada još uvek nije kompletno.
- *Verify peripheral IP using AXI4 BFM simulation interface* – omogućava verifikaciju ispravnog rada projektovanih AXI interfejsa pomoću simulacije
- *Verify peripheral IP using JTAG interface* – omogućava hardversku proveru ispravnosti rada projektovanih AXI interfejsa

U našem slučaju potrebno je izvršiti značajne modifikacije kreiranog IP jezgra. Na primer, potrebno je dodati memorijski podsistem i povezati ga sa automatski generisanim AXI-Lite i AXI-Full kontrolerima. Takođe, potrebno je dodati i sam „Naive“ množač i povezati ga sa ostalim modulima. Zbog je potrebno izabrati opciju „Edit IP“ pre nego što se proces kreiranja IP jezgra pomoću IP Packager alata završi, pritiskom na dugme *Finish*.

Pritiskom na dugme *Finish*, Vivado Design Suite alat generiše privremeni projekat u koji smešta sve automatski generisane fajlove, prema podešavanjima definisanim od

strane projektanta prilikom kreiranja IP jezgra i takođe otvara novi Vivado glavni prozor, čiji je sadržaj prikazan na slici 6.19.



Slika 6.19. Glavni prozor Vivado Design Suite alata koji prikazuje početni sadržaj automatski generisanog projekta kao rezultat kreiranja novog IP jezgra

Organizacija glavnog prozora Vivado alata ostala je ista kao i u slučaju ručno kreiranog projekta, sa jednom bitnom razlikom. U glavnom delu prozora pojavila se nova kartica, „Package IP“. Pomoću ove kartice definiše se proces pakovanja novog IP jezgra koje razvijamo. Pre nego što pređemo na detaljno objašnjenje ovog procesa pakovanja, moramo završiti proces projektovanja IP jezgra, modifikacijom postojećih i dodavanjem novih VHDL izvornih fajlova u privremeni projekat.

Analizom trenutnog sadržaja privremenog projekta možemo primetiti da se unutar njega već nalaze tri izvorna VHDL fajla:

- *matrix_multiplier_v1_0* – ovo je zapravo strukturni model IP jezgra na najvišem nivou hijerarhije. Reč je o nekompletnom modelu, automatski generisanom od strane Vivado alata na osnovu informacija sakupljenih prilikom kreiranja IP jezgra. Ovaj model će morati ručno da se dopuni kako bi na kraju modelovao sistem sa slike 6.12.
- *matrix_multiplier_v1_0_S00_AXI_inst* – VHDL model ulazno/izlaznog kontrolera koji implementira AXI-Lite protokol. I ovaj model je nepotpun, trenutno sadrži logiku samog AXI-Lite protokola, ali nedostaju informacije o

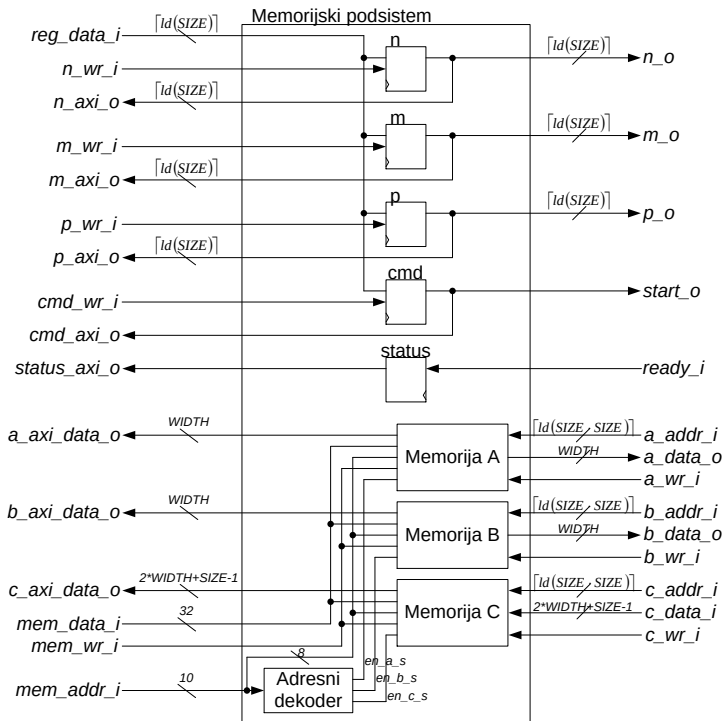
vezama sa unutrašnjim registrima (n , m , p , cmd i $status$) iz memorijskog podsistema.

- *matrix_multiplier_v1_0_S01_AXI_inst* – VHDL model ulazno/izlaznog kontrolera koji implementira AXI-Full protokol. I ovaj model je nepotpun, trenutno sadrži logiku samog AXI-Full protokola, ali nedostaju informacije o vezama sa unutrašnjim memorijama (A, B i C) iz memorijskog podsistema.

Da bismo kompletirali model IP jezgra „Naive“ množača matrica, osim gore navedenih izmena automatski generisanih VHDL modela, potrebno je u projekat dodati i VHDL modele „Naive“ množača matrica (ovaj model razvijen je u primeru 4.3) i memorijskog podsistema.

Kreiranje VHDL modela memorijskog podsistema

Napišimo prvo VHDL model memorijskog podsistema. Struktura memorijskog podsistema, zajedno sa svim portovima koji čine njegov interfejs prikazana je na slici 6.20.



Slika 6.20. Blok dijagram i interfejs memorijskog podsistema IP jezgra „Naive“ množača matrica

Na osnovu blok dijagrama sa slike 6.20, može se napisati sledeći mešoviti VHDL model memorijskog podsistema.

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

use work.utils_pkg.all;

entity mem_subsystem is
  generic (
    WIDTH      : integer := 8;
    SIZE       : integer := 3
  );
  port(
    clk        : in std_logic;
    reset      : in std_logic;

    -- Interface to the AXI controllers
    reg_data_i : in std_logic_vector(log2c(SIZE)-1 downto 0);
    n_wr_i     : in std_logic;
    m_wr_i     : in std_logic;
    p_wr_i     : in std_logic;
    cmd_wr_i   : in std_logic;

    n_axi_o    : out std_logic_vector(log2c(SIZE)-1 downto 0);
    m_axi_o    : out std_logic_vector(log2c(SIZE)-1 downto 0);
    p_axi_o    : out std_logic_vector(log2c(SIZE)-1 downto 0);
    cmd_axi_o  : out std_logic;
    status_axi_o : out std_logic;

    mem_addr_i : in std_logic_vector(10-1 downto 0);
    mem_data_i : in std_logic_vector(32-1 downto 0);
    mem_wr_i   : in std_logic;

    a_axi_data_o : out std_logic_vector(WIDTH-1 downto 0);
    b_axi_data_o : out std_logic_vector(WIDTH-1 downto 0);
    c_axi_data_o : out std_logic_vector(2*WIDTH+SIZE-1 downto 0);

    -- Interface to the matrix multiply module
    n_o          : out std_logic_vector(log2c(SIZE)-1 downto 0);
    m_o          : out std_logic_vector(log2c(SIZE)-1 downto 0);
    p_o          : out std_logic_vector(log2c(SIZE)-1 downto 0);
    start_o      : out std_logic;
    ready_i      : in std_logic;

    a_addr_i     : in std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
    a_wr_i       : in std_logic;
    a_data_o     : out std_logic_vector(WIDTH-1 downto 0);

    b_addr_i     : in std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
    b_wr_i       : in std_logic;
    b_data_o     : out std_logic_vector(WIDTH-1 downto 0);

    c_addr_i     : in std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
    c_wr_i       : in std_logic;
    c_data_i     : in std_logic_vector(2*WIDTH+SIZE-1 downto 0)

```

```

);
end mem_subsystem;

architecture struct of mem_subsystem is
  signal n_s, m_s, p_s: std_logic_vector(log2c(SIZE)-1 downto 0);
  signal cmd_s, status_s: std_logic;

  signal en_a_s, en_b_s, en_c_s: std_logic;
begin
  n_o    <= n_s;
  m_o    <= m_s;
  p_o    <= p_s;
  start_o <= cmd_s;

  ----- REGISTERS -----
  n_axi_o    <= n_s;
  m_axi_o    <= m_s;
  p_axi_o    <= p_s;
  cmd_axi_o  <= cmd_s;
  status_axi_o <= status_s;

  -- N register
  process(clk)
  begin
    if clk'event and clk = '1' then
      if reset = '1' then
        n_s <= (others => '0');
      elsif n_wr_i = '1' then
        n_s <= reg_data_i;
      end if;
    end if;
  end process;

  -- M register
  process(clk)
  begin
    if clk'event and clk = '1' then
      if reset = '1' then
        m_s <= (others => '0');
      elsif m_wr_i = '1' then
        m_s <= reg_data_i;
      end if;
    end if;
  end process;

  -- P register
  process(clk)
  begin
    if clk'event and clk = '1' then
      if reset = '1' then
        p_s <= (others => '0');
      elsif p_wr_i = '1' then
        p_s <= reg_data_i;
      end if;
    end if;
  end process;

  -- CMD register
  process(clk)

```



```

begin
  if clk'event and clk = '1' then
    if reset = '1' then
      cmd_s <= '0';
    elsif cmd_wr_i = '1' then
      cmd_s <= reg_data_i(0);
    end if;
  end if;
end process;

-- STATUS register
process(clk)
begin
  if clk'event and clk = '1' then
    if reset = '1' then
      status_s <= '0';
    else
      status_s <= ready_i;
    end if;
  end if;
end process;

----- MEMORIES -----
-- Address decoder
addr_dec: process (mem_addr_i)
begin
  -- Default assignments
  en_a_s <= '0';
  en_b_s <= '0';
  en_c_s <= '0';
  case mem_addr_i(9 downto 8) is
    when "00" =>
      en_a_s <= '1';
    when "01" =>
      en_b_s <= '1';
    when others =>
      en_c_s <= '1';
  end case;
end process;

-- Memory for storing the elements of matrix A
a_memory: entity work.dp_bram(beh)
generic map (
  WIDTH => WIDTH,
  SIZE   => SIZE
)
port map (
  ckla  => clk,
  ena   => en_a_s,
  wea   => mem_wr_i,
  addra => mem_addr_i(log2c(SIZE*SIZE)-1 downto 0),
  dia   => mem_data_i(WIDTH-1 downto 0),
  doa   => a_axi_data_o,

  clkb  => clk,
  enb   => '1',
  web   => a_wr_i,
  addrb => a_addr_i,
  dib   => (others => '0'),

```

```

    dob    => a_data_o
);

-- Memory for storing the elements of matrix B
b_memory: entity work.dp_bram(beh)
generic map (
    WIDTH    => WIDTH,
    SIZE    => SIZE
)
port map (
    ckla    => clk,
    ena     => en_b_s,
    wea     => mem_wr_i,
    addra   => mem_addr_i(log2c(SIZE*SIZE)-1 downto 0),
    dia     => mem_data_i(WIDTH-1 downto 0),
    doa     => b_axi_data_o,

    clkb    => clk,
    enb     => '1',
    web     => b_wr_i,
    addrb   => b_addr_i,
    dib     => (others => '0'),
    dob     => b_data_o
);

-- Memory for storing the elements of matrix C
c_memory: entity work.dp_bram(beh)
generic map (
    WIDTH => 2*WIDTH+SIZE,
    SIZE  => SIZE
)
port map (
    ckla    => clk,
    ena     => en_c_s,
    wea     => mem_wr_i,
    addra   => mem_addr_i(log2c(SIZE*SIZE)-1 downto 0),
    dia     => mem_data_i(2*WIDTH+SIZE-1 downto 0),
    doa     => c_axi_data_o,

    clkb    => clk,
    enb     => '1',
    web     => c_wr_i,
    addrb   => c_addr_i,
    dib     => c_data_i,
    dob     => open
);
end struct;

```

Model memorijskog podsistema koristii tri instance modela dvoprístupne memorije, potrebnih za smeštanje vrednosti elemenata matrica *A*, *B* i *C*. Radi kompletnosti u nastavku je prikazan i VHDL model dvoprístupne memorije.

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

```

```

use work.utils_pkg.all;

entity dp_bram is
  generic (
    WIDTH: integer := 8;
    SIZE   : integer := 3
  );
  port (
    ckla   : in std_logic;
    clk_b  : in std_logic;
    ena    : in std_logic;
    enb    : in std_logic;
    wea    : in std_logic;
    web    : in std_logic;
    addra  : in std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
    addrb  : in std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
    dia    : in std_logic_vector(WIDTH-1 downto 0);
    dib    : in std_logic_vector(WIDTH-1 downto 0);
    doa    : out std_logic_vector(WIDTH-1 downto 0);
    dob    : out std_logic_vector(WIDTH-1 downto 0)
  );
end dp_bram;

architecture beh of dp_bram is
  type ram_type is array (2**log2c(SIZE*SIZE)-1 downto 0)
    of std_logic_vector(WIDTH-1 downto 0);

  shared variable RAM: ram_type;
begin
  process(ckla)
  begin
    if ckla'event and ckla = '1' then
      if ena = '1' then
        doa <= RAM(conv_integer(addra));
        if wea = '1' then
          RAM(conv_integer(addra)) := dia;
        end if;
      end if;
    end if;
  end process;

  process(clk_b)
  begin
    if clk_b'event and clk_b = '1' then
      if enb = '1' then
        dob <= RAM(conv_integer(addrb));
        if web = '1' then
          RAM(conv_integer(addrb)) := dib;
        end if;
      end if;
    end if;
  end process;
end beh;

```

Modifikacija automatski generisanog VHDL modela AXI-Lite kontrolera

Sada imamo VHDL modele svih podsistema IP jezgra „Naive“ množača, prikazanih na slici 6.12. Kao što je ranije već rečeno VHDL modeli AXI-Lite i AXI-Full kontrolera, kao i VHDL model strukturnom modela IP jezgra „Naive“ množača sa slike 6.12 generisani su automatski, ali su nepotpuni.

VHDL model AXI-Lite kontrolera (sadržan u *axi_matrix_multiply_v1_0_S00_AXI.vhd* fajlu) potrebno je modifikovati na sledećim mestima:

- Potrebno je uključiti dodatne biblioteke (*std_logic_arith* i *utils_pkg*)
- Potrebno je definisati dodatne generičke konstante (*WIDTH* i *SIZE*)
- Potrebno je definisati dodatne ulazne i izlazne portove (*reg_data_o*, *n_wr_o*, *m_wr_o*, *p_wr_o*, *cmd_wr_o*, *n_axi_i*, *m_axi_i*, *p_axi_i*, *cmd_axi_i*, *status_axi_i*)
- Potrebno je izbaciti korišćenje *slv_reg1* - *slv_reg6* signala unutar procesa koji modeluje postupak upisa u unutrašnje registre i modifikovati proces tako da se generišu potrebni signali dozvole upisa *n_wr_o*, *m_wr_o*, *p_wr_o*, *cmd_wr_o*
- Potrebno je izbaciti korišćenje *slv_reg1* - *slv_reg6* signala unutar procesa koji modeluje postupak čitanja sadržaja unutrašnjih registara i modifikovati proces tako da se prilikom čitanja koriste vrednosti ulaznih portova *n_axi_i*, *m_axi_i*, *p_axi_i*, *cmd_axi_i*, *status_axi_i*
- Potrebno je dodati registar, na izaznom portu *reg_data_o*, radi pravilnog usklađivanja podatka koji se upisuje u selektovani registar i signala dozvole upisa u registar

Nakon navedenih modifikacija VHDL model AXI-Lite kontrolera ima sledeći izgled. Mesta na kojima su izvršene modifikacije osnovnog, automatski generisanog VHDL modela, označena su crvenom bojom.

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.std_logic_arith.all;  
use ieee.numeric_std.all;
```

```
use work.utils_pkg.all;
```

```
entity axi_matrix_multiply_v1_0_S00_AXI is  
  generic (  
    -- Users to add parameters here  
    WIDTH      : integer := 8;  
    SIZE       : integer := 3;  
    -- User parameters ends  
    -- Do not modify the parameters beyond this line  
    -- Width of S_AXI data bus  
    C_S_AXI_DATA_WIDTH : integer := 32;  
    -- Width of S_AXI address bus
```

```

C_S_AXI_ADDR_WIDTH : integer := 5);
port (
-- Users to add ports here
reg_data_o          : out std_logic_vector(log2c(SIZE)-1 downto 0);
n_wr_o              : out std_logic;
m_wr_o              : out std_logic;
p_wr_o              : out std_logic;
cmd_wr_o            : out std_logic;

n_axi_i              : in std_logic_vector(log2c(SIZE)-1 downto 0);
m_axi_i              : in std_logic_vector(log2c(SIZE)-1 downto 0);
p_axi_i              : in std_logic_vector(log2c(SIZE)-1 downto 0);
cmd_axi_i            : in std_logic;
status_axi_i         : in std_logic;
-- User ports ends
-- Do not modify the ports beyond this line
S_AXI_ACLK           : in std_logic;
S_AXI_ARESETN        : in std_logic;
S_AXI_AWADDR          : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
S_AXI_AWPROT          : in std_logic_vector(2 downto 0);
S_AXI_AWVALID         : in std_logic;
S_AXI_AWREADY         : out std_logic;
S_AXI_WDATA           : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
S_AXI_WSTRB           : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto 0);
S_AXI_WVALID         : in std_logic;
S_AXI_WREADY         : out std_logic;
S_AXI_BRESP           : out std_logic_vector(1 downto 0);
S_AXI_BVALID         : out std_logic;
S_AXI_BREADY         : in std_logic;
S_AXI_ARADDR          : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
S_AXI_ARPROT          : in std_logic_vector(2 downto 0);
S_AXI_ARVALID         : in std_logic;
S_AXI_ARREADY         : out std_logic;
S_AXI_RDATA           : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
S_AXI_RRESP           : out std_logic_vector(1 downto 0);
S_AXI_RVALID         : out std_logic;
S_AXI_RREADY         : in std_logic;
end axi_matrix_multiply_v1_0_S00_AXI;

architecture arch_imp of axi_matrix_multiply_v1_0_S00_AXI is
-- AXI4LITE signals
signal axi_awaddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
signal axi_awready     : std_logic;
signal axi_wready      : std_logic;
signal axi_bresp        : std_logic_vector(1 downto 0);
signal axi_bvalid       : std_logic;
signal axi_araddr       : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
signal axi_arready     : std_logic;
signal axi_rdata        : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal axi_rresp        : std_logic_vector(1 downto 0);
signal axi_rvalid       : std_logic;

-- Example-specific design signals
-- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
-- ADDR_LSB is used for addressing 32/64 bit registers/memories
-- ADDR_LSB = 2 for 32 bits (n downto 2)
-- ADDR_LSB = 3 for 64 bits (n downto 3)
constant ADDR_LSB      : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
constant OPT_MEM_ADDR_BITS : integer := 2;

```

---- Signals for user logic register space example

---- Number of Slave Registers 5

signal slv_reg_rden : std_logic;

signal slv_reg_wren : std_logic;

signal reg_data_out : std_logic_vector(C_S_AXI_DATA_WIDTH-1 **downto** 0);

signal byte_index : integer;

begin

-- I/O Connections assignments

S_AXI_AWREADY <= axi_awready;

S_AXI_WREADY <= axi_wready;

S_AXI_BRESP <= axi_bresp;

S_AXI_BVALID <= axi_bvalid;

S_AXI_ARREADY <= axi_arready;

S_AXI_RDATA <= axi_rdata;

S_AXI_RRESP <= axi_rresp;

S_AXI_RVALID <= axi_rvalid;

-- Implement axi_awready generation

-- axi_awready is asserted for one S_AXI_ACLK clock cycle when both S_AXI_AWVALID and

-- S_AXI_WVALID are asserted. axi_awready is de-asserted when reset is low.

process (S_AXI_ACLK)

begin

if rising_edge(S_AXI_ACLK) **then**

if S_AXI_ARESETN = '0' **then**

axi_awready <= '0';

else

if (axi_awready = '0' **and** S_AXI_AWVALID = '1' **and** S_AXI_WVALID = '1') **then**

-- Slave is ready to accept write address when there is a valid write address

-- and write data on the write address and data bus. This design expects no outstanding transactions.

axi_awready <= '1';

else

axi_awready <= '0';

end if;

end if;

end if;

end process;

-- Implement axi_awaddr latching

-- This process is used to latch the address when both S_AXI_AWVALID and S_AXI_WVALID are valid.

process (S_AXI_ACLK)

begin

if rising_edge(S_AXI_ACLK) **then**

if S_AXI_ARESETN = '0' **then**

axi_awaddr <= (others => '0');

else

if (axi_awready = '0' **and** S_AXI_AWVALID = '1' **and** S_AXI_WVALID = '1') **then**

-- Write Address latching

axi_awaddr <= S_AXI_AWADDR;

end if;

end if;

end if;

end process;

-- Implement axi_wready generation

-- axi_wready is asserted for one S_AXI_ACLK clock cycle when both S_AXI_AWVALID and

```

-- S_AXI_WVALID are asserted. axi_wready is de-asserted when reset is low.
process (S_AXI_ACLK)
begin
  if rising_edge(S_AXI_ACLK) then
    if S_AXI_ARESETN = '0' then
      axi_wready <= '0';
    else
      if (axi_wready = '0' and S_AXI_WVALID = '1' and S_AXI_AWVALID = '1') then
        -- Slave is ready to accept write data when there is a valid write address and
        -- write data on the write address and data bus. This design expects no outstanding transactions.
        axi_wready <= '1';
      else
        axi_wready <= '0';
      end if;
    end if;
  end if;
end process;

-- Implement memory mapped register select and write logic generation
-- The write data is accepted and written to memory mapped registers when axi_awready, S_AXI_WVALID,
-- axi_wready and S_AXI_WVALID are asserted. Write strobes are used to select byte enables of slave
-- registers while writing. These registers are cleared when reset (active low) is applied. Slave register write
-- enable is asserted when valid address and data are available and the slave is ready to accept the write
-- address and write data.
slv_reg_wren <= axi_wready and S_AXI_WVALID and axi_awready and S_AXI_AWVALID;
process (S_AXI_ACLK)
  variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
  if rising_edge(S_AXI_ACLK) then
    if S_AXI_ARESETN = '0' then
      n_wr_o    <= '0';
      m_wr_o    <= '0';
      p_wr_o    <= '0';
      cmd_wr_o  <= '0';
    else
      -- Default assignments
      n_wr_o    <= '0';
      m_wr_o    <= '0';
      p_wr_o    <= '0';
      cmd_wr_o  <= '0';

      loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
      if (slv_reg_wren = '1') then
        case loc_addr is
          when b"000" =>
            n_wr_o <= '1';
          when b"001" =>
            m_wr_o <= '1';
          when b"010" =>
            p_wr_o <= '1';
          when b"011" =>
            cmd_wr_o <= '1';
          when others =>
            end case;
        end if;
      end if;
    end if;
  end if;
end process;

```

```

-- Implement write response logic generation
-- The write response and response valid signals are asserted by the slave when
-- axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
-- This marks the acceptance of address and indicates the status of write transaction.
process (S_AXI_ACLK)
begin
  if rising_edge(S_AXI_ACLK) then
    if S_AXI_ARESETN = '0' then
      axi_bvalid <= '0';
      axi_bresp <= "00"; --need to work more on the responses
    else
      if (axi_awready = '1' and S_AXI_AWVALID = '1' and axi_wready = '1'
        and S_AXI_WVALID = '1' and axi_bvalid = '0' ) then
        axi_bvalid <= '1';
        axi_bresp <= "00";
      elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
        --check if bready is asserted while bvalid is high
        axi_bvalid <= '0'; -- (there is a possibility that bready is always asserted high)
      end if;
    end if;
  end if;
end process;

-- Implement axi_arready generation
-- axi_arready is asserted for one S_AXI_ACLK clock cycle when S_AXI_ARVALID
-- is asserted. axi_arready is de-asserted when reset (active low) is asserted.
-- The read address is also latched when S_AXI_ARVALID is asserted.
-- axi_araddr is reset to zero on reset assertion.
process (S_AXI_ACLK)
begin
  if rising_edge(S_AXI_ACLK) then
    if S_AXI_ARESETN = '0' then
      axi_arready <= '0';
      axi_araddr <= (others => '1');
    else
      if (axi_arready = '0' and S_AXI_ARVALID = '1') then
        -- indicates that the slave has accepted the valid read address
        axi_arready <= '1';
        -- Read Address latching
        axi_araddr <= S_AXI_ARADDR;
      else
        axi_arready <= '0';
      end if;
    end if;
  end if;
end process;

-- Implement axi_rvalid generation
-- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_ARVALID and axi_arready are asserted. The slave registers data are
-- available on the axi_rdata bus at this instance. The assertion of axi_rvalid marks
-- the validity of read data on the bus and axi_rresp indicates the status of read
-- transaction. axi_rvalid is deasserted on reset (active low).
-- axi_rresp and axi_rdata are cleared to zero on reset (active low).
process (S_AXI_ACLK)
begin
  if rising_edge(S_AXI_ACLK) then
    if S_AXI_ARESETN = '0' then
      axi_rvalid <= '0';

```



```

    axi_rresp <= "00";
else
    if (axi_arready = '1' and S_AXI_ARVALID = '1' and axi_rvalid = '0') then
        -- Valid read data is available at the read data bus
        axi_rvalid <= '1';
        axi_rresp <= "00"; -- 'OKAY' response
    elsif (axi_rvalid = '1' and S_AXI_RREADY = '1') then
        -- Read data is accepted by the master
        axi_rvalid <= '0';
    end if;
end if;
end if;
end process;

-- Implement memory mapped register select and read logic generation
-- Slave register read enable is asserted when valid address is available
-- and the slave is ready to accept the read address.
slv_reg_rden <= axi_arready and S_AXI_ARVALID and (not axi_rvalid);
process (n_axi_i, m_axi_i, p_axi_i, cmd_axi_i, status_axi_i, axi_araddr,
         S_AXI_ARESETN, slv_reg_rden)
    variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    -- Address decoding for reading registers
    loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
    case loc_addr is
        when b"000" =>
            reg_data_out <= conv_std_logic_vector(0, C_S_AXI_DATA_WIDTH-log2c(SIZE))&n_axi_i;
        when b"001" =>
            reg_data_out <= conv_std_logic_vector(0, C_S_AXI_DATA_WIDTH-log2c(SIZE))&m_axi_i;
        when b"010" =>
            reg_data_out <= conv_std_logic_vector(0, C_S_AXI_DATA_WIDTH-log2c(SIZE))&p_axi_i;
        when b"011" =>
            reg_data_out <= conv_std_logic_vector(0, C_S_AXI_DATA_WIDTH-1)&cmd_axi_i;
        when b"100" =>
            reg_data_out <= conv_std_logic_vector(0, C_S_AXI_DATA_WIDTH-1)&status_axi_i;
        when others =>
            reg_data_out <= (others => '0');
    end case;
end process;

-- Output register or memory read data
process( S_AXI_ACLK) is
begin
    if (rising_edge (S_AXI_ACLK)) then
        if ( S_AXI_ARESETN = '0' ) then
            axi_rdata <= (others => '0');
        else
            if (slv_reg_rden = '1') then
                -- When there is a valid read address (S_AXI_ARVALID) with acceptance of
                -- read address by the slave (axi_arready), output the read data
                -- Read address mux
                axi_rdata <= reg_data_out;    -- register read data
            end if;
        end if;
    end if;
end process;

-- Add user logic here
process (S_AXI_ACLK)

```

```

begin
  if (S_AXI_ACLK'event and S_AXI_ACLK = '1') then
    reg_data_o <= S_AXI_WDATA(log2c(SIZE)-1 downto 0);
  end if;
end process;
-- User logic ends
end arch_imp;

```

Modifikacija automatski generisanog VHDL modela AXI-Full kontrolera

Slično, kao i u slučaju automatski generisanog VHDL modela AXI-Lite kontrolera, i automatski generisani VHDL model AXI-Full kontrolera (koji se nalazi unutar *axi_matrix_multiply_v1_0_S01_AXI.vhd* fajla) potrebno se modifikovati na sledećim mestima:

- Potrebno je uključiti dodatne biblioteke (*utils_pkg*)
- Potrebno je definisati dodatne generičke konstante (WIDTH i SIZE)
- Potrebno je definisati dodatne ulazne i izlazne portove (*mem_addr_o*, *mem_data_o*, *mem_wr_o*, *a_axi_data_i*, *b_axi_data_i*, *c_axi_data_i*)
- Potrebno je modelovati generisanje potrebnih signala memorijskog interfejsa koji spaja AXI-Full kontroler sa memorijskim podsistemom

Nakon navedenih modifikacija VHDL model AXI-Full kontrolera ima sledeći izgled. Mesta na kojima su izvršene modifikacije osnovnog, automatski generisanog VHDL modela, označena su crvenom bojom.

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

use work.utils_pkg.all;

entity axi_matrix_multiply_v1_0_S01_AXI is
  generic (
    -- Users to add parameters here
    WIDTH : integer      := 8;
    SIZE   : integer     := 3;
    -- User parameters ends
    -- Do not modify the parameters beyond this line
    -- Width of ID for for write address, write data, read address and read data
    C_S_AXI_ID_WIDTH      : integer := 1;
    -- Width of S_AXI data bus
    C_S_AXI_DATA_WIDTH   : integer := 32;
    -- Width of S_AXI address bus
    C_S_AXI_ADDR_WIDTH   : integer := 12;
    -- Width of optional user defined signal in write address channel
    C_S_AXI_AWUSER_WIDTH : integer := 0;
    -- Width of optional user defined signal in read address channel
    C_S_AXI_ARUSER_WIDTH : integer := 0;
    -- Width of optional user defined signal in write data channel

```

```

C_S_AXI_WUSER_WIDTH : integer := 0;
-- Width of optional user defined signal in read data channel
C_S_AXI_RUSER_WIDTH : integer := 0;
-- Width of optional user defined signal in write response channel
C_S_AXI_BUSER_WIDTH : integer := 0;
port (
-- Users to add ports here
mem_addr_o : out std_logic_vector(C_S_AXI_ADDR_WIDTH- ((C_S_AXI_DATA_WIDTH/32)+ 1)-1
downto 0);
mem_data_o : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
mem_wr_o : out std_logic;

a_axi_data_i: in std_logic_vector(WIDTH-1 downto 0);
b_axi_data_i: in std_logic_vector(WIDTH-1 downto 0);
c_axi_data_i: in std_logic_vector(2*WIDTH+SIZE-1 downto 0);
-- User ports ends
-- Do not modify the ports beyond this line
S_AXI_ACLK                : in std_logic;
S_AXI_ARESETN             : in std_logic;
S_AXI_AWID                : in std_logic_vector(C_S_AXI_ID_WIDTH-1 downto 0);
S_AXI_AWADDR              : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
S_AXI_AWLEN               : in std_logic_vector(7 downto 0);
S_AXI_AWSIZE              : in std_logic_vector(2 downto 0);
S_AXI_AWBURST             : in std_logic_vector(1 downto 0);
S_AXI_AWLOCK              : in std_logic;
S_AXI_AWCACHE             : in std_logic_vector(3 downto 0);
S_AXI_AWPROT              : in std_logic_vector(2 downto 0);
S_AXI_AWQOS               : in std_logic_vector(3 downto 0);
S_AXI_AWREGION            : in std_logic_vector(3 downto 0);
S_AXI_AWUSER              : in std_logic_vector(C_S_AXI_AWUSER_WIDTH-1 downto 0);
S_AXI_AWVALID             : in std_logic;
S_AXI_AWREADY            : out std_logic;
S_AXI_WDATA               : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
S_AXI_WSTRB               : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto 0);
S_AXI_WLAST               : in std_logic;
S_AXI_WUSER               : in std_logic_vector(C_S_AXI_WUSER_WIDTH-1 downto 0);
S_AXI_WVALID              : in std_logic;
S_AXI_WREADY             : out std_logic;
S_AXI_BID                 : out std_logic_vector(C_S_AXI_ID_WIDTH-1 downto 0);
S_AXI_BRESP               : out std_logic_vector(1 downto 0);
S_AXI_BUSER               : out std_logic_vector(C_S_AXI_BUSER_WIDTH-1 downto 0);
S_AXI_BVALID              : out std_logic;
S_AXI_BREADY             : in std_logic;
S_AXI_ARID                : in std_logic_vector(C_S_AXI_ID_WIDTH-1 downto 0);
S_AXI_ARADDR              : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
S_AXI_ARLEN               : in std_logic_vector(7 downto 0);
S_AXI_ARSIZE              : in std_logic_vector(2 downto 0);
S_AXI_ARBURST             : in std_logic_vector(1 downto 0);
S_AXI_ARLOCK              : in std_logic;
S_AXI_ARCACHE             : in std_logic_vector(3 downto 0);
S_AXI_ARPROT              : in std_logic_vector(2 downto 0);
S_AXI_ARQOS               : in std_logic_vector(3 downto 0);
S_AXI_ARREGION            : in std_logic_vector(3 downto 0);
S_AXI_ARUSER              : in std_logic_vector(C_S_AXI_ARUSER_WIDTH-1 downto 0);
S_AXI_ARVALID             : in std_logic;
S_AXI_ARREADY            : out std_logic;
S_AXI_RID                 : out std_logic_vector(C_S_AXI_ID_WIDTH-1 downto 0);
S_AXI_RDATA               : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
S_AXI_RRESP               : out std_logic_vector(1 downto 0);

```

```

S_AXI_RLAST      : out std_logic;
S_AXI_RUSER      : out std_logic_vector(C_S_AXI_RUSER_WIDTH-1 downto 0);
S_AXI_RVALID     : out std_logic;
S_AXI_RREADY     : in std_logic;
end axi_matrix_multiply_v1_0_S01_AXI;

architecture arch_imp of axi_matrix_multiply_v1_0_S01_AXI is
-- AXI4FULL signals
signal axi_awaddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
signal axi_awready     : std_logic;
signal axi_wready      : std_logic;
signal axi_bresp       : std_logic_vector(1 downto 0);
signal axi_buser       : std_logic_vector(C_S_AXI_BUSER_WIDTH-1 downto 0);
signal axi_bvalid      : std_logic;
signal axi_araddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
signal axi_arready     : std_logic;
signal axi_rdata       : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
signal axi_rresp       : std_logic_vector(1 downto 0);
signal axi_rlast       : std_logic;
signal axi_ruser       : std_logic_vector(C_S_AXI_RUSER_WIDTH-1 downto 0);
signal axi_rvalid      : std_logic;
signal aw_wrap_en      : std_logic;
signal ar_wrap_en      : std_logic;
signal aw_wrap_size    : integer;
signal ar_wrap_size    : integer;
signal axi_awv_awr_flag : std_logic;
signal axi_arv_arr_flag : std_logic;
signal axi_awlen_cntr  : std_logic_vector(7 downto 0);
signal axi_arlen_cntr  : std_logic_vector(7 downto 0);
signal axi_arburst     : std_logic_vector(2-1 downto 0);
signal axi_awburst     : std_logic_vector(2-1 downto 0);
signal axi_arlen       : std_logic_vector(8-1 downto 0);
signal axi_awlen       : std_logic_vector(8-1 downto 0);

--local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
--ADDR_LSB is used for addressing 32/64 bit registers/memories
--ADDR_LSB = 2 for 32 bits (n downto 2)
--ADDR_LSB = 3 for 42 bits (n downto 3)
constant ADDR_LSB      : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
constant OPT_MEM_ADDR_BITS : integer := 7;
constant USER_NUM_MEM   : integer := 1;
constant low            : std_logic_vector (C_S_AXI_ADDR_WIDTH - 1 downto 0) := (others => '0');
signal addr_field_s    : std_logic_vector(1 downto 0);
begin
-- I/O Connections assignments
S_AXI_AWREADY <= axi_awready;
S_AXI_WREADY  <= axi_wready;
S_AXI_BRESP   <= axi_bresp;
S_AXI_BUSER   <= axi_buser;
S_AXI_BVALID  <= axi_bvalid;
S_AXI_ARREADY <= axi_arready;
S_AXI_RDATA   <= axi_rdata;
S_AXI_RRESP   <= axi_rresp;
S_AXI_RLAST   <= axi_rlast;
S_AXI_RUSER   <= axi_ruser;
S_AXI_RVALID  <= axi_rvalid;
S_AXI_BID     <= S_AXI_AWID;
S_AXI_RID     <= S_AXI_ARID;

```

```

aw_wrap_size <= ((C_S_AXI_DATA_WIDTH)/8 * to_integer(unsigned(axi_awlen)));
ar_wrap_size <= ((C_S_AXI_DATA_WIDTH)/8 * to_integer(unsigned(axi_arlen)));
aw_wrap_en <= '1' when (((axi_awaddr
    AND std_logic_vector(to_unsigned(aw_wrap_size,C_S_AXI_ADDR_WIDTH)))
    XOR std_logic_vector(to_unsigned(aw_wrap_size,C_S_AXI_ADDR_WIDTH))) = low)
    else '0';
ar_wrap_en <= '1' when (((axi_araddr
    AND std_logic_vector(to_unsigned(ar_wrap_size,C_S_AXI_ADDR_WIDTH)))
    XOR std_logic_vector(to_unsigned(ar_wrap_size,C_S_AXI_ADDR_WIDTH))) = low)
    else '0';
S_AXI_BUSER <= (others => '0');

```

-- Implement axi_awready generation
-- axi_awready is asserted for one S_AXI_ACLK clock cycle when both S_AXI_AWVALID and
-- S_AXI_WVALID are asserted. axi_awready is de-asserted when reset is low.

```

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_awready <= '0';
            axi_awv_awr_flag <= '0';
        else
            if (axi_awready = '0' and S_AXI_AWVALID = '1' and axi_awv_awr_flag = '0'
                and axi_arv_arr_flag = '0') then
                -- slave is ready to accept an address and associated control signals
                axi_awv_awr_flag <= '1'; -- used for generation of bresp() and bvalid
                axi_awready <= '1';
            elsif (S_AXI_WLAST = '1' and axi_wready = '1') then
                -- preparing to accept next address after current write burst tx completion
                axi_awv_awr_flag <= '0';
            else
                axi_awready <= '0';
            end if;
        end if;
    end if;
end process;

```

-- Implement axi_awaddr latching
-- This process is used to latch the address when both
-- S_AXI_AWVALID and S_AXI_WVALID are valid.

```

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_awaddr <= (others => '0');
            axi_awburst <= (others => '0');
            axi_awlen <= (others => '0');
            axi_awlen_cntr <= (others => '0');
        else
            if (axi_awready = '0' and S_AXI_AWVALID = '1' and axi_awv_awr_flag = '0')
                then
                    -- address latching
                    -- start address of transfer
                    axi_awaddr <= S_AXI_AWADDR(C_S_AXI_ADDR_WIDTH - 1 downto 0);
                    axi_awlen_cntr <= (others => '0');
                    axi_awburst <= S_AXI_AWBURST;
                    axi_awlen <= S_AXI_AWLEN;
                elsif ((axi_awlen_cntr <= axi_awlen) and axi_wready = '1' and S_AXI_WVALID = '1') then
                    axi_awlen_cntr <= std_logic_vector(unsigned(axi_awlen_cntr) + 1);

```

```

case (axi_awburst) is
when "00" => -- fixed burst
    -- The write address for all the beats in the transaction are fixed
    axi_awaddr <= axi_awaddr; ----for awsize = 4 bytes (010)
when "01" => --incremental burst
    -- The write address for all the beats in the transaction are increments by awsize
    axi_awaddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
        <= std_logic_vector (unsigned(axi_awaddr(C_S_AXI_ADDR_WIDTH - 1
            downto ADDR_LSB)) + 1); --awaddr aligned to 4 byte boundary
    axi_awaddr(ADDR_LSB-1 downto 0) <= (others => '0'); -- for awsize = 4 bytes (010)
when "10" => --Wrapping burst
    -- The write address wraps when the address reaches wrap boundary
    if (aw_wrap_en = '1') then
        axi_awaddr <= std_logic_vector (unsigned(axi_awaddr
            - (to_unsigned(aw_wrap_size,C_S_AXI_ADDR_WIDTH)));
    else
        axi_awaddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
            <= std_logic_vector (unsigned(axi_awaddr(C_S_AXI_ADDR_WIDTH - 1
                downto ADDR_LSB)) + 1);--awaddr aligned to 4 byte boundary
        axi_awaddr(ADDR_LSB-1 downto 0) <= (others => '0'); -- for awsize = 4 bytes (010)
    end if;
when others => --reserved (incremental burst for example)
    axi_awaddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
        <= std_logic_vector (unsigned(axi_awaddr(C_S_AXI_ADDR_WIDTH - 1
            downto ADDR_LSB)) + 1);--for awsize = 4 bytes (010)
    axi_awaddr(ADDR_LSB-1 downto 0) <= (others => '0');
end case;
end if;
end if;
end if;
end process;

-- Implement axi_wready generation
-- axi_wready is asserted for one S_AXI_ACLK clock cycle when both S_AXI_AWVALID and
-- S_AXI_WVALID are asserted. axi_wready is de-asserted when reset is low.
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_wready <= '0';
        else
            if (axi_wready = '0' and S_AXI_WVALID = '1' and axi_awv_awr_flag = '1') then
                axi_wready <= '1';
            elsif (S_AXI_WLAST = '1' and axi_wready = '1') then
                axi_wready <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement write response logic generation
-- The write response and response valid signals are asserted by the slave when
-- axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
-- This marks the acceptance of address and indicates the status of write transaction.
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_bvalid <= '0';

```

```

    axi_bresp <= "00"; --need to work more on the responses
else
    if (axi_awv_awr_flag = '1' and axi_wready = '1' and S_AXI_WVALID = '1'
        and axi_bvalid = '0' and S_AXI_WLAST = '1') then
        axi_bvalid <= '1';
        axi_bresp <= "00";
    elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
        --check if bready is asserted while bvalid is high)
        axi_bvalid <= '0';
    end if;
end if;
end if;
end process;

-- Implement axi_arready generation
-- axi_arready is asserted for one S_AXI_ACLK clock cycle when S_AXI_ARVALID is asserted.
-- axi_arready is de-asserted when reset (active low) is asserted. The read address is also latched when
-- S_AXI_ARVALID is asserted. axi_araddr is reset to zero on reset assertion.
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_arready <= '0';
            axi_arv_arr_flag <= '0';
        else
            if (axi_arready = '0' and S_AXI_ARVALID = '1' and axi_awv_awr_flag = '0'
                and axi_arv_arr_flag = '0') then
                axi_arready <= '1';
                axi_arv_arr_flag <= '1';
            elsif (axi_rvalid = '1' and S_AXI_RREADY = '1' and (axi_arlen_cntr = axi_arlen)) then
                -- preparing to accept next address after current read completion
                axi_arv_arr_flag <= '0';
            else
                axi_arready <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement axi_araddr latching
--This process is used to latch the address when both S_AXI_ARVALID and S_AXI_RVALID are valid.
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_araddr <= (others => '0');
            axi_arburst <= (others => '0');
            axi_arlen <= (others => '0');
            axi_arlen_cntr <= (others => '0');
            axi_rl原因 <= '0';
        else
            if (axi_arready = '0' and S_AXI_ARVALID = '1' and axi_arv_arr_flag = '0') then
                -- address latching
                -- start address of transfer
                axi_araddr <= S_AXI_ARADDR(C_S_AXI_ADDR_WIDTH - 1 downto 0);
                axi_arlen_cntr <= (others => '0');
                axi_rl原因 <= '0';
                axi_arburst <= S_AXI_ARBURST;
                axi_arlen <= S_AXI_ARLEN;
            end if;
        end if;
    end if;
end process;

```

```

elsif((axi_arlen_cntr <= axi_arlen) and axi_rvalid = '1'
    and S_AXI_RREADY = '1') then
    axi_arlen_cntr <= std_logic_vector (unsigned(axi_arlen_cntr) + 1);
    axi_rlast <= '0';
    case (axi_arburst) is
        when "00" => -- fixed burst
            -- The read address for all the beats in the transaction are fixed
            axi_araddr <= axi_araddr; ----for arsize = 4 bytes (010)
        when "01" => --incremental burst
            -- The read address for all the beats in the transaction are increments by awsize
            axi_araddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
                <= std_logic_vector (unsigned(axi_araddr(C_S_AXI_ADDR_WIDTH - 1
                    downto ADDR_LSB)) + 1); --araddr aligned to 4 byte boundary
        when "10" => --Wrapping burst
            -- The read address wraps when the address reaches wrap boundary
            if (ar_wrap_en = '1') then
                axi_araddr <= std_logic_vector (unsigned(axi_araddr)
                    - (to_unsigned(ar_wrap_size,C_S_AXI_ADDR_WIDTH)));
            else
                axi_araddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
                    <= std_logic_vector (unsigned(axi_araddr(C_S_AXI_ADDR_WIDTH - 1
                        downto ADDR_LSB)) + 1); --araddr aligned to 4 byte boundary
                axi_araddr(ADDR_LSB-1 downto 0) <= (others => '0'); -- for awsize = 4 bytes (010)
            end if;
        when others => --reserved (incremental burst for example)
            axi_araddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
                <= std_logic_vector (unsigned(axi_araddr(C_S_AXI_ADDR_WIDTH - 1
                    downto ADDR_LSB)) + 1);--for arsize = 4 bytes (010)
            axi_araddr(ADDR_LSB-1 downto 0) <= (others => '0');
    end case;
elsif((axi_arlen_cntr = axi_arlen) and axi_rlast = '0' and axi_arv_arr_flag = '1') then
    axi_rlast <= '1';
    elsif (S_AXI_RREADY = '1') then
        axi_rlast <= '0';
    end if;
end if;
end if;
end process;

-- Implement axi_rvalid generation
-- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both S_AXI_ARVALID and
-- axi_rready are asserted. The slave registers data are available on the axi_rdata bus at this instance.
-- The assertion of axi_rvalid marks the validity of read data on the bus and axi_rresp indicates
-- the status of read transaction.axi_rvalid is deasserted on reset (active low).
-- axi_rresp and axi_rdata are cleared to zero on reset (active low).
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_rvalid <= '0';
            axi_rresp <= "00";
        else
            if (axi_arv_arr_flag = '1' and axi_rvalid = '0') then
                axi_rvalid <= '1';
                axi_rresp <= "00"; -- 'OKAY' response
            elsif (axi_rvalid = '1' and S_AXI_RREADY = '1') then
                axi_rvalid <= '0';
            end if;
        end if;
    end if;

```



```

    end if;
  end if;
end process;

-- Add user logic here
mem_wr_o <= axi_wready and S_AXI_WVALID;
mem_addr_o <= axi_araddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
    when axi_arv_arr_flag = '1' else
    axi_awaddr(C_S_AXI_ADDR_WIDTH - 1 downto ADDR_LSB)
    when axi_awv_awr_flag = '1' else
    (others => '0');
mem_data_o <= S_AXI_WDATA;

--Output memory read data
addr_field_s <= axi_araddr(C_S_AXI_ADDR_WIDTH - 1
    downto C_S_AXI_ADDR_WIDTH - 2);
process(a_axi_data_i, b_axi_data_i, c_axi_data_i, axi_rvalid, addr_field_s) is
begin
  if (axi_rvalid = '1') then
    -- When there is a valid read address (S_AXI_ARVALID) with
    -- acceptance of read address by the slave (axi_arready), output the read data
    case addr_field_s is
      when "00" =>
        axi_rdata <= std_logic_vector(to_unsigned(0, C_S_AXI_DATA_WIDTH-WIDTH))& a_axi_data_i;
      when "01" =>
        axi_rdata <= std_logic_vector(to_unsigned(0, C_S_AXI_DATA_WIDTH-WIDTH))&b_axi_data_i;
      when others =>
        axi_rdata <= std_logic_vector(to_unsigned(0, C_S_AXI_DATA_WIDTH-(2*WIDTH+SIZE)))&
        c_axi_data_i;
    end case;
  else
    axi_rdata <= (others => '0');
  end if;
end process;
-- User logic ends
end arch_imp;
```

Modifikacija automatski generisanog VHDL modela celokupnog sistema

Poslednji VHDL model koji je potrebno modifikovati je VHDL model celokupnog sistema (koji se nalazi unutar *axi_matrix_multiply_v1_0.vhd* fajla). Mesta na kojima je neophodno modifikovati ovaj model su:

- Potrebno je uključiti dodatne biblioteke (*utils_pkg*)
- Potrebno je definisati dodatne generičke konstante (*WIDTH* i *SIZE*)
- Potrebno je modifikovati postojeće generičke konstante (*C_S01_AXI_ADDR_WIDTH*, *C_S01_AXI_AWUSER_WIDTH*, *C_S01_AXI_ARUSER_WIDTH*, *C_S01_AXI_WUSER_WIDTH*, *C_S01_AXI_RUSER_WIDTH*, *C_S01_AXI_BUSER_WIDTH*)

- Potrebno je modifikovati postojeće instance *axi_matrix_multiply_v1_0_S00_AXI* i *axi_matrix_multiply_v1_0_S01_AXI* komponenti
- Potrebno je instancionirati i pravilno povezati *mem_subsystem* i *matrix_mult* komponente

Nakon navedenih modifikacija VHDL model kompletnog IP jezgra „Naive“ množača matrica sa AXI interfejsom ima sledeći izgled. Mesta na kojima su izvršene modifikacije osnovnog, automatski generisanog VHDL modela, označena su crvenom bojom.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

use work.utils_pkg.all;

entity axi_matrix_multiply_v1_0 is
generic (
  -- Users to add parameters here
  WIDTH : integer := 8;
  SIZE   : integer := 3;
  -- User parameters ends
  -- Do not modify the parameters beyond this line
  -- Parameters of Axi Slave Bus Interface S00_AXI
  C_S00_AXI_DATA_WIDTH      : integer := 32;
  C_S00_AXI_ADDR_WIDTH      : integer := 5;
  -- Parameters of Axi Slave Bus Interface S01_AXI
  C_S01_AXI_ID_WIDTH        : integer := 1;
  C_S01_AXI_DATA_WIDTH      : integer := 32;
  C_S01_AXI_ADDR_WIDTH      : integer := 12;
  C_S01_AXI_AWUSER_WIDTH    : integer := 1;
  C_S01_AXI_ARUSER_WIDTH    : integer := 1;
  C_S01_AXI_WUSER_WIDTH     : integer := 1;
  C_S01_AXI_RUSER_WIDTH     : integer := 1;
  C_S01_AXI_BUSER_WIDTH     : integer := 1);
port (
  -- Users to add ports here
  -- User ports ends
  -- Do not modify the ports beyond this line
  -- Ports of Axi Slave Bus Interface S00_AXI
  s00_axi_aclk      : in std_logic;
  s00_axi_aresetn    : in std_logic;
  s00_axi_awaddr     : in std_logic_vector(C_S00_AXI_ADDR_WIDTH-1 downto 0);
  s00_axi_awprot     : in std_logic_vector(2 downto 0);
  s00_axi_awvalid    : in std_logic;
  s00_axi_awready    : out std_logic;
  s00_axi_wdata      : in std_logic_vector(C_S00_AXI_DATA_WIDTH-1 downto 0);
  s00_axi_wstrb      : in std_logic_vector((C_S00_AXI_DATA_WIDTH/8)-1 downto 0);
  s00_axi_wvalid     : in std_logic;
  s00_axi_wready     : out std_logic;
  s00_axi_bresp      : out std_logic_vector(1 downto 0);
  s00_axi_bvalid     : out std_logic;
```

```

s00_axi_bready      : in std_logic;
s00_axi_araddr      : in std_logic_vector(C_S00_AXI_ADDR_WIDTH-1 downto 0);
s00_axi_arprot      : in std_logic_vector(2 downto 0);
s00_axi_arvalid     : in std_logic;
s00_axi_arready     : out std_logic;
s00_axi_rdata       : out std_logic_vector(C_S00_AXI_DATA_WIDTH-1 downto 0);
s00_axi_rresp       : out std_logic_vector(1 downto 0);
s00_axi_rvalid      : out std_logic;
s00_axi_ready       : in std_logic;

```

-- Ports of Axi Slave Bus Interface S01_AXI

```

s01_axi_aclk        : in std_logic;
s01_axi_aresetn     : in std_logic;
s01_axi_awid        : in std_logic_vector(C_S01_AXI_ID_WIDTH-1 downto 0);
s01_axi_awaddr      : in std_logic_vector(C_S01_AXI_ADDR_WIDTH-1 downto 0);
s01_axi_awlen       : in std_logic_vector(7 downto 0);
s01_axi_awsiz       : in std_logic_vector(2 downto 0);
s01_axi_awburst     : in std_logic_vector(1 downto 0);
s01_axi_awlock      : in std_logic;
s01_axi_awcache     : in std_logic_vector(3 downto 0);
s01_axi_awprot      : in std_logic_vector(2 downto 0);
s01_axi_awqos       : in std_logic_vector(3 downto 0);
s01_axi_awregion    : in std_logic_vector(3 downto 0);
s01_axi_awuser      : in std_logic_vector(C_S01_AXI_AWUSER_WIDTH-1 downto 0);
s01_axi_awvalid     : in std_logic;
s01_axi_awready     : out std_logic;
s01_axi_wdata       : in std_logic_vector(C_S01_AXI_DATA_WIDTH-1 downto 0);
s01_axi_wstrb       : in std_logic_vector((C_S01_AXI_DATA_WIDTH/8)-1 downto 0);
s01_axi_wlast       : in std_logic;
s01_axi_wuser       : in std_logic_vector(C_S01_AXI_WUSER_WIDTH-1 downto 0);
s01_axi_wvalid      : in std_logic;
s01_axi_wready     : out std_logic;
s01_axi_bid         : out std_logic_vector(C_S01_AXI_ID_WIDTH-1 downto 0);
s01_axi_bresp       : out std_logic_vector(1 downto 0);
s01_axi_buser       : out std_logic_vector(C_S01_AXI_BUSER_WIDTH-1 downto 0);
s01_axi_bvalid      : out std_logic;
s01_axi_bready     : in std_logic;
s01_axi_arid        : in std_logic_vector(C_S01_AXI_ID_WIDTH-1 downto 0);
s01_axi_araddr      : in std_logic_vector(C_S01_AXI_ADDR_WIDTH-1 downto 0);
s01_axi_arlen       : in std_logic_vector(7 downto 0);
s01_axi_arsize      : in std_logic_vector(2 downto 0);
s01_axi_arburst     : in std_logic_vector(1 downto 0);
s01_axi_arlock      : in std_logic;
s01_axi_arcache     : in std_logic_vector(3 downto 0);
s01_axi_arprot      : in std_logic_vector(2 downto 0);
s01_axi_arqos       : in std_logic_vector(3 downto 0);
s01_axi_arregion    : in std_logic_vector(3 downto 0);
s01_axi_aruser      : in std_logic_vector(C_S01_AXI_ARUSER_WIDTH-1 downto 0);
s01_axi_arvalid     : in std_logic;
s01_axi_arready     : out std_logic;
s01_axi_rid         : out std_logic_vector(C_S01_AXI_ID_WIDTH-1 downto 0);
s01_axi_rdata       : out std_logic_vector(C_S01_AXI_DATA_WIDTH-1 downto 0);
s01_axi_rresp       : out std_logic_vector(1 downto 0);
s01_axi_rlast       : out std_logic;
s01_axi_ruser       : out std_logic_vector(C_S01_AXI_RUSER_WIDTH-1 downto 0);
s01_axi_rvalid      : out std_logic;
s01_axi_ready       : in std_logic;
end axi_matrix_multiply_v1_0;

```

```

architecture arch_imp of axi_matrix_multiply_v1_0 is
  -- signal declarations
  signal reset_s      : std_logic;
  -- Interface to the AXI controllers
  signal reg_data_s    : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal n_wr_s        : std_logic;
  signal m_wr_s        : std_logic;
  signal p_wr_s        : std_logic;
  signal cmd_wr_s      : std_logic;

  signal n_axi_s       : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal m_axi_s       : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal p_axi_s       : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal cmd_axi_s     : std_logic;
  signal status_axi_s  : std_logic;

  signal mem_addr_s    : std_logic_vector(10-1 downto 0);
  signal mem_data_s    : std_logic_vector(32-1 downto 0);
  signal mem_wr_s      : std_logic;

  signal a_axi_data_s  : std_logic_vector(WIDTH-1 downto 0);
  signal b_axi_data_s  : std_logic_vector(WIDTH-1 downto 0);
  signal c_axi_data_s  : std_logic_vector(2*WIDTH+SIZE-1 downto 0);

  -- Interface to the matrix multiply module
  signal n_s           : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal m_s           : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal p_s           : std_logic_vector(log2c(SIZE)-1 downto 0);
  signal start_s       : std_logic;
  signal ready_s       : std_logic;

  signal a_addr_s      : std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
  signal a_wr_s        : std_logic;
  signal a_data_s      : std_logic_vector(WIDTH-1 downto 0);

  signal b_addr_s      : std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
  signal b_wr_s        : std_logic;
  signal b_data_s      : std_logic_vector(WIDTH-1 downto 0);

  signal c_addr_s      : std_logic_vector(log2c(SIZE*SIZE)-1 downto 0);
  signal c_wr_s        : std_logic;
  signal c_data_s      : std_logic_vector(2*WIDTH+SIZE-1 downto 0);
begin

  -- Instantiation of Axi Bus Interface S00_AXI
  axi_matrix_multiply_v1_0_S00_AXI_inst : entity work.axi_matrix_multiply_v1_0_S00_AXI(arch_imp)
    generic map (
      WIDTH           => WIDTH,
      SIZE            => SIZE,
      C_S_AXI_DATA_WIDTH  => C_S00_AXI_DATA_WIDTH,
      C_S_AXI_ADDR_WIDTH  => C_S00_AXI_ADDR_WIDTH)
    port map (
      -- Users to add ports here
      reg_data_o       => reg_data_s,
      n_wr_o           => n_wr_s,
      m_wr_o           => m_wr_s,
      p_wr_o           => p_wr_s,
      cmd_wr_o         => cmd_wr_s,

```

```

n_axi_i      => n_axi_s,
m_axi_i      => m_axi_s,
p_axi_i      => p_axi_s,
cmd_axi_i    => cmd_axi_s,
status_axi_i => status_axi_s,
-- User ports ends

S_AXI_ACLK      => s00_axi_aclk,
S_AXI_ARESETN  => s00_axi_aresetn,
S_AXI_AWADDR    => s00_axi_awaddr,
S_AXI_AWPROT    => s00_axi_awprot,
S_AXI_AWVALID   => s00_axi_awvalid,
S_AXI_AWREADY   => s00_axi_awready,
S_AXI_WDATA     => s00_axi_wdata,
S_AXI_WSTRB     => s00_axi_wstrb,
S_AXI_WVALID    => s00_axi_wvalid,
S_AXI_WREADY    => s00_axi_wready,
S_AXI_BRESP     => s00_axi_bresp,
S_AXI_BVALID    => s00_axi_bvalid,
S_AXI_BREADY    => s00_axi_bready,
S_AXI_ARADDR    => s00_axi_araddr,
S_AXI_ARPROT    => s00_axi_arprot,
S_AXI_ARVALID   => s00_axi_arvalid,
S_AXI_ARREADY   => s00_axi_arready,
S_AXI_RDATA     => s00_axi_rdata,
S_AXI_RRESP     => s00_axi_rresp,
S_AXI_RVALID    => s00_axi_rvalid,
S_AXI_RREADY    => s00_axi_rready);

-- Instantiation of Axi Bus Interface S01_AXI
axi_matrix_multiply_v1_0_S01_AXI_inst : entity work.axi_matrix_multiply_v1_0_S01_AXI(arch_imp)
generic map (
    WIDTH      => WIDTH,
    SIZE       => SIZE,
    C_S_AXI_ID_WIDTH      => C_S01_AXI_ID_WIDTH,
    C_S_AXI_DATA_WIDTH    => C_S01_AXI_DATA_WIDTH,
    C_S_AXI_ADDR_WIDTH    => C_S01_AXI_ADDR_WIDTH,
    C_S_AXI_AWUSER_WIDTH  => C_S01_AXI_AWUSER_WIDTH,
    C_S_AXI_ARUSER_WIDTH  => C_S01_AXI_ARUSER_WIDTH,
    C_S_AXI_WUSER_WIDTH   => C_S01_AXI_WUSER_WIDTH,
    C_S_AXI_RUSER_WIDTH   => C_S01_AXI_RUSER_WIDTH,
    C_S_AXI_BUSER_WIDTH   => C_S01_AXI_BUSER_WIDTH)
port map (
    -- Users to add ports here
    mem_addr_o      => mem_addr_s,
    mem_data_o      => mem_data_s,
    mem_wr_o        => mem_wr_s,

    a_axi_data_i    => a_axi_data_s,
    b_axi_data_i    => b_axi_data_s,
    c_axi_data_i    => c_axi_data_s,
    -- User ports ends
    S_AXI_ACLK      => s01_axi_aclk,
    S_AXI_ARESETN   => s01_axi_aresetn,
    S_AXI_AWID      => s01_axi_awid,
    S_AXI_AWADDR    => s01_axi_awaddr,
    S_AXI_AWLEN     => s01_axi_awlen,
    S_AXI_AWSIZE    => s01_axi_awsiz,
    S_AXI_AWBURST   => s01_axi_awburst,

```

```

S_AXI_AWLOCK      => s01_axi_awlock,
S_AXI_AWCACHE     => s01_axi_awcache,
S_AXI_AWPROT      => s01_axi_awprot,
S_AXI_AWQOS       => s01_axi_awqos,
S_AXI_AWREGION    => s01_axi_awregion,
S_AXI_AWUSER      => s01_axi_awuser,
S_AXI_AWVALID     => s01_axi_awvalid,
S_AXI_AWREADY     => s01_axi_awready,
S_AXI_WDATA       => s01_axi_wdata,
S_AXI_WSTRB       => s01_axi_wstrb,
S_AXI_WLAST       => s01_axi_wlast,
S_AXI_WUSER       => s01_axi_wuser,
S_AXI_WVALID      => s01_axi_wvalid,
S_AXI_WREADY      => s01_axi_wready,
S_AXI_BID         => s01_axi_bid,
S_AXI_BRESP       => s01_axi_bresp,
S_AXI_BUSER       => s01_axi_buser,
S_AXI_BVALID      => s01_axi_bvalid,
S_AXI_BREADY      => s01_axi_bready,
S_AXI_ARID        => s01_axi_arid,
S_AXI_ARADDR      => s01_axi_araddr,
S_AXI_ARLEN       => s01_axi_arlen,
S_AXI_ARSIZE      => s01_axi_arsize,
S_AXI_ARBURST     => s01_axi_arburst,
S_AXI_ARLOCK      => s01_axi_arlock,
S_AXI_ARCACHE     => s01_axi_arcache,
S_AXI_ARPROT      => s01_axi_arprot,
S_AXI_ARQOS       => s01_axi_arqos,
S_AXI_ARREGION    => s01_axi_arregion,
S_AXI_ARUSER      => s01_axi_aruser,
S_AXI_ARVALID     => s01_axi_arvalid,
S_AXI_ARREADY     => s01_axi_arready,
S_AXI_RID         => s01_axi_rid,
S_AXI_RDATA       => s01_axi_rdata,
S_AXI_RRESP       => s01_axi_rresp,
S_AXI_RLAST       => s01_axi_rlast,
S_AXI_RUSER       => s01_axi_ruser,
S_AXI_RVALID      => s01_axi_rvalid,
S_AXI_RREADY      => s01_axi_rready);

```

-- Add user logic here

```
reset_s <= not s00_axi_aresetn;
```

-- Memory subsystem

memory_subsystem: **entity** work.mem_subsystem(struct)

generic map (

 WIDTH => WIDTH,

 SIZE => SIZE

)

port map (

 clk => s00_axi_aclk,

 reset => reset_s,

-- Interface to the AXI controllers

reg_data_i => reg_data_s,

n_wr_i => n_wr_s,

m_wr_i => m_wr_s,

p_wr_i => p_wr_s,

cmd_wr_i => cmd_wr_s,

```

n_axi_o      => n_axi_s,
m_axi_o      => m_axi_s,
p_axi_o      => p_axi_s,
cmd_axi_o    => cmd_axi_s,
status_axi_o => status_axi_s,

mem_addr_i   => mem_addr_s,
mem_data_i   => mem_data_s,
mem_wr_i     => mem_wr_s,

a_axi_data_o => a_axi_data_s,
b_axi_data_o => b_axi_data_s,
c_axi_data_o => c_axi_data_s,

-- Interface to the matrix multiply module
n_o          => n_s,
m_o          => m_s,
p_o          => p_s,
start_o      => start_s,
ready_i      => ready_s,

a_addr_i     => a_addr_s,
a_wr_i       => a_wr_s,
a_data_o     => a_data_s,

b_addr_i     => b_addr_s,
b_wr_i       => b_wr_s,
b_data_o     => b_data_s,

c_addr_i     => c_addr_s,
c_wr_i       => c_wr_s,
c_data_i     => c_data_s);

-- Matrix multiplier module
matrix_multiplier: entity work.matrix_mult(two_seg_arch)
generic map (
    WIDTH      => WIDTH,
    SIZE       => SIZE
)
port map (
    ----- Clocking and reset interface -----
    clk         => s00_axi_aclk,
    reset       => reset_s,
    ----- Input data interface -----
    -- Matrix A memory interface
    a_addr_o    => a_addr_s,
    a_data_i    => a_data_s,
    a_wr_o     => a_wr_s,
    -- Matrix B memory interface
    b_addr_o    => b_addr_s,
    b_data_i    => b_data_s,
    b_wr_o     => b_wr_s,
    -- Matrix dimensions definition interface
    n_in        => n_s,
    p_in        => m_s,
    m_in        => p_s,
    ----- Output data interface -----
    -- Matrix C memory interface

```

```

c_addr_o      => c_addr_s,
c_data_o      => c_data_s,
c_wr_o        => c_wr_s,
----- Command interface -----
start         => start_s,
----- Status interface -----
ready         => ready_s);
-- User logic ends
end arch_imp;

```

Ovim je process projektovanja IP jezgra „Naive” množača sa AXI interfejsom završen. Pre nego što predemo na process pakovanja svih izvornih fajlova koji će činiti IP jezgro „Naive” množača, potrebno je verifikovati korektan način rada IP jezgra. Sledeći testbenč može se koristiti u tu svrhu.

Prikazani testbenč instancionira IP jezgro „Naive“ množača sa AXI interfejsom i koristeći stimulus generator proces generiše potrebne transakcije na AXI-Lite i AXI-Full interfejsima kako bi konfigurisao IP jezgro na potreban način, upisao elemente matrica koje je potrebno pomnožiti i na kraju pročitao rezultat množenja.

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

use work.utils_pkg.all;

entity axi_matrix_mult_tb is
end entity;

architecture beh of axi_matrix_mult_tb is
    constant DATA_WIDTH_c : integer := 8;
    constant SIZE_c        : integer := 3;
    constant N_c            : integer := 3;
    constant M_c            : integer := 3;
    constant P_c            : integer := 3;

    type mem_t is array (0 to SIZE_c*SIZE_c-1) of integer;

    constant MEM_A_CONTENT_c: mem_t := (1, 2, 3, 4, 5, 6, 7, 8, 9, others => 0);
    constant MEM_B_CONTENT_c: mem_t := (10, 11, 12, 13, 14, 15, 16, 17, 18, others => 0);

    signal clk_s: std_logic;
    signal reset_s: std_logic;

    -- Matrix multiplier core's address map
    constant N_REG_ADDR_C : integer := 0;
    constant M_REG_ADDR_C : integer := 4;
    constant P_REG_ADDR_C : integer := 8;
    constant CMD_REG_ADDR_C : integer := 12;
    constant STATUS_REG_ADDR_C : integer := 16;

    -- Matrix multiplier internal memory map
    constant MEMORY_A_OFFSET_C : integer := 0;
    constant MEMORY_B_OFFSET_C : integer := 256*4;

```



```

constant MEMORY_C_OFFSET_C      : integer := 512*4;

----- AXI Interfaces signals -----
-- Parameters of Axi-Lite Slave Bus Interface S00_AXI
constant C_S00_AXI_DATA_WIDTH_c : integer := 32;
constant C_S00_AXI_ADDR_WIDTH_c : integer := 5;

-- Parameters of Axi-Full Slave Bus Interface S01_AXI
constant C_S01_AXI_ID_WIDTH_c    : integer := 1;
constant C_S01_AXI_DATA_WIDTH_c  : integer := 32;
constant C_S01_AXI_ADDR_WIDTH_c  : integer := 12;
constant C_S01_AXI_AWUSER_WIDTH_c : integer := 1;
constant C_S01_AXI_ARUSER_WIDTH_c : integer := 1;
constant C_S01_AXI_WUSER_WIDTH_c : integer := 1;
constant C_S01_AXI_RUSER_WIDTH_c : integer := 1;
constant C_S01_AXI_BUSER_WIDTH_c : integer := 1;

-- Ports of Axi-Lite Slave Bus Interface S00_AXI
signal s00_axi_aclk_s      : std_logic := '0';
signal s00_axi_aresetn_s   : std_logic := '1';
signal s00_axi_awaddr_s    : std_logic_vector(C_S00_AXI_ADDR_WIDTH_c-1 downto 0)
                                     := (others => '0');
signal s00_axi_awprot_s    : std_logic_vector(2 downto 0) := (others => '0');
signal s00_axi_awvalid_s   : std_logic := '0';
signal s00_axi_awready_s   : std_logic := '0';
signal s00_axi_wdata_s     : std_logic_vector(C_S00_AXI_DATA_WIDTH_c-1 downto 0)
                                     := (others => '0');
signal s00_axi_wstrb_s     : std_logic_vector((C_S00_AXI_DATA_WIDTH_c/8)-1 downto 0)
                                     := (others => '0');
signal s00_axi_wvalid_s    : std_logic := '0';
signal s00_axi_wready_s    : std_logic := '0';
signal s00_axi_bresp_s     : std_logic_vector(1 downto 0) := (others => '0');
signal s00_axi_bvalid_s    : std_logic := '0';
signal s00_axi_bready_s    : std_logic := '0';
signal s00_axi_araddr_s    : std_logic_vector(C_S00_AXI_ADDR_WIDTH_c-1 downto 0)
                                     := (others => '0');
signal s00_axi_arprot_s    : std_logic_vector(2 downto 0) := (others => '0');
signal s00_axi_arvalid_s   : std_logic := '0';
signal s00_axi_arready_s   : std_logic := '0';
signal s00_axi_rdata_s     : std_logic_vector(C_S00_AXI_DATA_WIDTH_c-1 downto 0) := (others
=> '0');
signal s00_axi_rresp_s     : std_logic_vector(1 downto 0) := (others => '0');
signal s00_axi_rvalid_s    : std_logic := '0';
signal s00_axi_rready_s    : std_logic := '0';

-- Ports of Axi-Full Slave Bus Interface S01_AXI
signal s01_axi_aclk_s      : std_logic := '0';
signal s01_axi_aresetn_s   : std_logic := '1';
signal s01_axi_awid_s      : std_logic_vector(C_S01_AXI_ID_WIDTH_c-1 downto 0)
                                     := (others => '0');
signal s01_axi_awaddr_s    : std_logic_vector(C_S01_AXI_ADDR_WIDTH_c-1 downto 0)
                                     := (others => '0');
signal s01_axi_awlen_s     : std_logic_vector(7 downto 0) := (others => '0');
signal s01_axi_awsz_s      : std_logic_vector(2 downto 0) := (others => '0');
signal s01_axi_awburst_s   : std_logic_vector(1 downto 0) := (others => '0');
signal s01_axi_awlock_s    : std_logic := '0';
signal s01_axi_awcache_s   : std_logic_vector(3 downto 0) := (others => '0');
signal s01_axi_awprot_s    : std_logic_vector(2 downto 0) := (others => '0');
signal s01_axi_awqos_s     : std_logic_vector(3 downto 0) := (others => '0');

```

```

signal s01_axi_awregion_s      : std_logic_vector(3 downto 0) := (others => '0');
signal s01_axi_awuser_s       : std_logic_vector(C_S01_AXI_AWUSER_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_awvalid_s      : std_logic := '0';
signal s01_axi_awready_s      : std_logic := '0';
signal s01_axi_wdata_s        : std_logic_vector(C_S01_AXI_DATA_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_wstrb_s        : std_logic_vector((C_S01_AXI_DATA_WIDTH_c/8)-1 downto 0)
                                := (others => '0');

signal s01_axi_wlast_s        : std_logic := '0';
signal s01_axi_wuser_s        : std_logic_vector(C_S01_AXI_WUSER_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_wvalid_s       : std_logic := '0';
signal s01_axi_wready_s       : std_logic := '0';
signal s01_axi_bid_s          : std_logic_vector(C_S01_AXI_ID_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_bresp_s        : std_logic_vector(1 downto 0) := (others => '0');
signal s01_axi_buser_s        : std_logic_vector(C_S01_AXI_BUSER_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_bvalid_s       : std_logic := '0';
signal s01_axi_bready_s       : std_logic := '0';
signal s01_axi_arid_s         : std_logic_vector(C_S01_AXI_ID_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_araddr_s       : std_logic_vector(C_S01_AXI_ADDR_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_arlen_s        : std_logic_vector(7 downto 0) := (others => '0');
signal s01_axi_arsize_s       : std_logic_vector(2 downto 0) := (others => '0');
signal s01_axi_arburst_s      : std_logic_vector(1 downto 0) := (others => '0');
signal s01_axi_arlock_s       : std_logic := '0';
signal s01_axi_arcache_s      : std_logic_vector(3 downto 0) := (others => '0');
signal s01_axi_arprot_s       : std_logic_vector(2 downto 0) := (others => '0');
signal s01_axi_arqos_s        : std_logic_vector(3 downto 0) := (others => '0');
signal s01_axi_arregion_s     : std_logic_vector(3 downto 0) := (others => '0');
signal s01_axi_aruser_s       : std_logic_vector(C_S01_AXI_ARUSER_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_arvalid_s      : std_logic := '0';
signal s01_axi_arready_s      : std_logic := '0';
signal s01_axi_rid_s          : std_logic_vector(C_S01_AXI_ID_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_rdata_s        : std_logic_vector(C_S01_AXI_DATA_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_rresp_s        : std_logic_vector(1 downto 0) := (others => '0');
signal s01_axi_rlast_s        : std_logic := '0';
signal s01_axi_ruser_s        : std_logic_vector(C_S01_AXI_RUSER_WIDTH_c-1 downto 0)
                                := (others => '0');

signal s01_axi_rvalid_s       : std_logic := '0';
signal s01_axi_rready_s       : std_logic := '0';
begin
  clk_gen: process
  begin
    clk_s <= '0', '1' after 100 ns;
    wait for 200 ns;
  end process;

  stimulus_generator: process
  variable axi_read_data_v      : std_logic_vector(31 downto 0);
  variable transfer_size_v      : integer;
  begin
    -- reset AXI-lite interface. Reset will be 10 clock cycles wide

```

```

s00_axi_aresetn_s <= '0';
-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;
-- release reset
s00_axi_aresetn_s <= '1';
wait until falling_edge(clk_s);

-----
--          Initialize the Matrix Multiplier core          --
-----

report "Loading the matrix dimensions information into the Matrix Multiplier core!";
-- Set the value for the first dimension (parameter N) of matrix A and C
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(N_REG_ADDR_C, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '1';
s00_axi_wdata_s <= conv_std_logic_vector(N_c, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s <= '1';
s00_axi_wstrb_s <= "1111";
s00_axi_bready_s <= '1';
wait until s00_axi_awready_s = '1';
wait until s00_axi_wready_s = '0';
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(0, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '0';
s00_axi_wdata_s <= conv_std_logic_vector(0, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s <= '0';
s00_axi_wstrb_s <= "0000";
wait until s00_axi_bvalid_s = '0';
wait until falling_edge(clk_s);
s00_axi_bready_s <= '0';
wait until falling_edge(clk_s);

-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;

-- Set the value for the second dimension of matrix A and the first dimension of matrix B (parameter M)
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(M_REG_ADDR_C, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '1';
s00_axi_wdata_s <= conv_std_logic_vector(M_c, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s <= '1';
s00_axi_wstrb_s <= "1111";
s00_axi_bready_s <= '1';
wait until s00_axi_awready_s = '1';
wait until s00_axi_wready_s = '0';
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(0, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '0';
s00_axi_wdata_s <= conv_std_logic_vector(0, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s <= '0';
s00_axi_wstrb_s <= "0000";
wait until s00_axi_bvalid_s = '0';
wait until falling_edge(clk_s);
s00_axi_bready_s <= '0';
wait until falling_edge(clk_s);

```

```

-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;

-- Set the value for the second dimension of matrix B and the first dimension of matrix C (parameter P)
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(P_REG_ADDR_C, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '1';
s00_axi_wdata_s <= conv_std_logic_vector(P_c, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s <= '1';
s00_axi_wstrb_s <= "1111";
s00_axi_bready_s <= '1';
wait until s00_axi_awready_s = '1';
wait until s00_axi_bready_s = '0';
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(0, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '0';
s00_axi_wdata_s <= conv_std_logic_vector(0, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s <= '0';
s00_axi_wstrb_s <= "0000";
wait until s00_axi_bvalid_s = '0';
wait until falling_edge(clk_s);
s00_axi_bready_s <= '0';
wait until falling_edge(clk_s);

-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;

-----
-- Load the element values for matrix A into the Matrix Multiplier core --
-----

report "Loading matrix A element values into the core!";
-- reset AXI4 interface. Reset will be 10 clock cycles wide
s01_axi_aresetn_s <= '0';
-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;
-- release reset
s01_axi_aresetn_s <= '1';
wait until falling_edge(clk_s);

-- Write some data using AXI4 interface and burst mode
transfer_size_v := N_c*M_c;
wait until falling_edge(clk_s);
s01_axi_awaddr_s <= conv_std_logic_vector(MEMORY_A_OFFSET_C, s01_axi_awaddr_s'length);
-- Set the number of data that will be transferred in one burst
s01_axi_awlen_s <= conv_std_logic_vector(transfer_size_v-1, 8);
s01_axi_awsizs_s <= "010"; -- Size of each transfer will be 4 bytes
s01_axi_awburst_s <= "01"; -- Burst type will be "INCR"
s01_axi_awvalid_s <= '1';
wait until s01_axi_awready_s = '1';
wait until s01_axi_bready_s = '0';
wait until falling_edge(clk_s);

```

```

s01_axi_wdata_s  <= conv_std_logic_vector(MEM_A_CONTENT_c(0), s01_axi_wdata_s'length);
s01_axi_wvalid_s  <= '1';
s01_axi_wstrb_s  <= "1111";
s01_axi_wlast_s  <= '0';
s01_axi_bready_s  <= '1';
wait until s01_axi_wready_s = '1';
wait until falling_edge(clk_s);
wait until falling_edge(clk_s);
for data_counter in 1 to transfer_size_v-2 loop
    s01_axi_wdata_s  <= conv_std_logic_vector(MEM_A_CONTENT_c(data_counter),
                                                s01_axi_wdata_s'length);

    s01_axi_wvalid_s <= '1';
    s01_axi_wstrb_s  <= "1111";
    s01_axi_wlast_s  <= '0';
    s01_axi_bready_s <= '1';
    wait until falling_edge(clk_s);
end loop;
s01_axi_wdata_s  <= conv_std_logic_vector(MEM_A_CONTENT_c(transfer_size_v-1),
                                                01_axi_wdata_s'length);

s01_axi_wvalid_s  <= '1';
s01_axi_wstrb_s  <= "1111";
s01_axi_wlast_s  <= '1';
s01_axi_bready_s  <= '1';
wait until falling_edge(clk_s);
s01_axi_wdata_s  <= conv_std_logic_vector(0, s01_axi_wdata_s'length);
s01_axi_wvalid_s  <= '0';
s01_axi_wstrb_s  <= "0000";
s01_axi_wlast_s  <= '0';
s01_axi_awaddr_s  <= conv_std_logic_vector(0, s01_axi_awaddr_s'length);
s01_axi_awlen_s   <= (others => '0');
s01_axi_awburst_s <= "00";
s01_axi_awvalid_s <= '0';
if (s01_axi_bvalid_s = '1') then
    wait until s01_axi_bvalid_s = '0';
else
    wait until s01_axi_bvalid_s = '1';
    wait until s01_axi_bvalid_s = '0';
end if;
wait until falling_edge(clk_s);
s01_axi_bready_s <= '0';

```

```
-- Load the element values for matrix B into the Matrix Multiplier core --
```

```

report "Loading matrix B element values into the core!";
-- reset AXI4 interface. Reset will be 10 clock cycles wide
s01_axi_aresetn_s <= '0';
-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;
-- release reset
s01_axi_aresetn_s <= '1';
wait until falling_edge(clk_s);

-- Write some data using AXI4 interface and burst mode
transfer_size_v := M_c*P_c;
wait until falling_edge(clk_s);

```

```

s01_axi_awaddr_s <= conv_std_logic_vector(MEMORY_B_OFFSET_C, s01_axi_awaddr_s'length);
-- Set the number of data that will be transferred in one burst
s01_axi_awlen_s <= conv_std_logic_vector(transfer_size_v-1, 8);
s01_axi_awsz_s <= "010"; -- Size of each transfer will be 4 bytes
s01_axi_awburst_s <= "01"; -- Burst type will be "INCR"
s01_axi_awvalid_s <= '1';
wait until s01_axi_awready_s = '1';
wait until s01_axi_awready_s = '0';
wait until falling_edge(clk_s);

s01_axi_wdata_s <= conv_std_logic_vector(MEM_B_CONTENT_c(0), s01_axi_wdata_s'length);
s01_axi_wvalid_s <= '1';
s01_axi_wstrb_s <= "1111";
s01_axi_wlast_s <= '0';
s01_axi_bready_s <= '1';
wait until s01_axi_wready_s = '1';
wait until falling_edge(clk_s);
wait until falling_edge(clk_s);
for data_counter in 1 to transfer_size_v-2 loop
    s01_axi_wdata_s <= conv_std_logic_vector(MEM_B_CONTENT_c(data_counter),
                                                s01_axi_wdata_s'length);

    s01_axi_wvalid_s <= '1';
    s01_axi_wstrb_s <= "1111";
    s01_axi_wlast_s <= '0';
    s01_axi_bready_s <= '1';
    wait until falling_edge(clk_s);
end loop;
s01_axi_wdata_s <= conv_std_logic_vector(MEM_B_CONTENT_c(transfer_size_v-1),
                                                s01_axi_wdata_s'length);

s01_axi_wvalid_s <= '1';
s01_axi_wstrb_s <= "1111";
s01_axi_wlast_s <= '1';
s01_axi_bready_s <= '1';
wait until falling_edge(clk_s);
s01_axi_wdata_s <= conv_std_logic_vector(0, s01_axi_wdata_s'length);
s01_axi_wvalid_s <= '0';
s01_axi_wstrb_s <= "0000";
s01_axi_wlast_s <= '0';
s01_axi_awaddr_s <= conv_std_logic_vector(0, s01_axi_awaddr_s'length);
s01_axi_awlen_s <= (others => '0');
s01_axi_awburst_s <= "00";
s01_axi_awvalid_s <= '0';
if (s01_axi_bvalid_s = '1') then
    wait until s01_axi_bvalid_s = '0';
else
    wait until s01_axi_bvalid_s = '1';
    wait until s01_axi_bvalid_s = '0';
end if;
wait until falling_edge(clk_s);
s01_axi_bready_s <= '0';

-----
--                                     Start the Matrix Multiplier core                                     --
-----

report "Starting the matric multiplication process!";
-- Set the value start bit (bit 0 in the CMD register) to 1
wait until falling_edge(clk_s);
s00_axi_awaddr_s <= conv_std_logic_vector(CMD_REG_ADDR_C, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '1';

```

```

s00_axi_wdata_s  <= conv_std_logic_vector(1, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s  <= '1';
s00_axi_wstrb_s  <= "1111";
s00_axi_bready_s  <= '1';
wait until s00_axi_awready_s = '1';
wait until s00_axi_awready_s = '0';
wait until falling_edge(clk_s);
s00_axi_awaddr_s  <= conv_std_logic_vector(0, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '0';
s00_axi_wdata_s  <= conv_std_logic_vector(0, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s  <= '0';
s00_axi_wstrb_s  <= "0000";
wait until s00_axi_bvalid_s = '0';
wait until falling_edge(clk_s);
s00_axi_bready_s  <= '0';
wait until falling_edge(clk_s);

-- wait for 5 falling edges of AXI-lite clock signal
for i in 1 to 5 loop
    wait until falling_edge(clk_s);
end loop;

report "Clearing the start bit!";
-- Set the value start bit (bit 0 in the CMD register) to 1
wait until falling_edge(clk_s);
s00_axi_awaddr_s  <= conv_std_logic_vector(CMD_REG_ADDR_C, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '1';
s00_axi_wdata_s  <= conv_std_logic_vector(0, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s  <= '1';
s00_axi_wstrb_s  <= "1111";
s00_axi_bready_s  <= '1';
wait until s00_axi_awready_s = '1';
wait until s00_axi_awready_s = '0';
wait until falling_edge(clk_s);
s00_axi_awaddr_s  <= conv_std_logic_vector(0, C_S00_AXI_ADDR_WIDTH_c);
s00_axi_awvalid_s <= '0';
s00_axi_wdata_s  <= conv_std_logic_vector(0, C_S00_AXI_DATA_WIDTH_c);
s00_axi_wvalid_s  <= '0';
s00_axi_wstrb_s  <= "0000";
wait until s00_axi_bvalid_s = '0';
wait until falling_edge(clk_s);
s00_axi_bready_s  <= '0';
wait until falling_edge(clk_s);

-----
--      Wait until Matrix Multiplier core finishes matrix multiplication      --
-----

report "Waiting for the matric multiplication process to complete!";
loop
    -- Read the content of the Status register
    wait until falling_edge(clk_s);
    s00_axi_araddr_s <= conv_std_logic_vector(STATUS_REG_ADDR_C, 5);
    s00_axi_arvalid_s <= '1';
    s00_axi_rready_s <= '1';
    wait until s00_axi_arready_s = '1';
    axi_read_data_v := s00_axi_rdata_s;
    wait until s00_axi_arready_s = '0';
    wait until falling_edge(clk_s);

```

```

s00_axi_araddr_s <= conv_std_logic_vector(0, 5);
s00_axi_arvalid_s <= '0';
s00_axi_rready_s <= '0';

-- Check is the 1st bit of the Status register set to one
if (axi_read_data_v(0) = '1') then
  -- Matrix multiplication process completed
  exit;
else
  wait for 1000 ns;
end if;
end loop;

-----
--          Read the elements of matrix C from the Matrix Multiplier core      --
-----

report "Reading the results of the matrix multiplication process!";
-- Read some data using AXI4 interface
transfer_size_v := N_c * P_c;
wait until falling_edge(clk_s);
s01_axi_araddr_s <= conv_std_logic_vector(MEMORY_C_OFFSET_C, s01_axi_araddr_s'length);
-- Set the number of data that will be transfered in one burst
s01_axi_arlen_s <= conv_std_logic_vector(transfer_size_v-1, 8);
s01_axi_arsize_s <= "010"; -- Size of each transfer will be 4 bytes
s01_axi_arburst_s <= "01"; -- Burst type will be "INCR"
s01_axi_arvalid_s <= '1';
s01_axi_rready_s <= '1';
wait until s01_axi_arready_s = '1';
wait until s01_axi_arready_s = '0';
wait until falling_edge(clk_s);
s01_axi_araddr_s <= conv_std_logic_vector(0, s01_axi_araddr_s'length);
s01_axi_arlen_s <= (others => '0');
s01_axi_arburst_s <= "00";
s01_axi_arvalid_s <= '0';
for data_counter in 1 to transfer_size_v loop
  wait until s01_axi_rvalid_s = '1';
  wait until s01_axi_rvalid_s = '0';
  wait until falling_edge(clk_s);
end loop;
wait until falling_edge(clk_s);
s01_axi_rready_s <= '0';

-- End of the test
wait;
end process;

-----
--          DUT          --
-----

axi_matrix_multiplier: entity work.axi_matrix_multiply_v1_0(arch_imp)
generic map(
  WIDTH      => DATA_WIDTH_c,
  SIZE       => SIZE_c)
port map (
  -- Ports of Axi Slave Bus Interface S00_AXI
  s00_axi_aclk      => clk_s,
  s00_axi_aresetn   => s00_axi_aresetn_s,
  s00_axi_awaddr    => s00_axi_awaddr_s,
  s00_axi_awprot    => s00_axi_awprot_s,

```



```

s00_axi_awvalid => s00_axi_awvalid_s,
s00_axi_awready => s00_axi_awready_s,
s00_axi_wdata   => s00_axi_wdata_s,
s00_axi_wstrb   => s00_axi_wstrb_s,
s00_axi_wvalid  => s00_axi_wvalid_s,
s00_axi_wready  => s00_axi_wready_s,
s00_axi_bresp   => s00_axi_bresp_s,
s00_axi_bvalid  => s00_axi_bvalid_s,
s00_axi_bready  => s00_axi_bready_s,
s00_axi_araddr  => s00_axi_araddr_s,
s00_axi_arprot  => s00_axi_arprot_s,
s00_axi_arvalid => s00_axi_arvalid_s,
s00_axi_arready => s00_axi_arready_s,
s00_axi_rdata   => s00_axi_rdata_s,
s00_axi_rresp   => s00_axi_rresp_s,
s00_axi_rvalid  => s00_axi_rvalid_s,
s00_axi_rready  => s00_axi_rready_s,

```

-- Ports of Axi Slave Bus Interface S01_AXI

```

s01_axi_aclk      => clk_s,
s01_axi_aresetn   => s01_axi_aresetn_s,
s01_axi_awid      => s01_axi_awid_s,
s01_axi_awaddr    => s01_axi_awaddr_s,
s01_axi_awlen     => s01_axi_awlen_s,
s01_axi_awsz      => s01_axi_awsz_s,
s01_axi_awburst   => s01_axi_awburst_s,
s01_axi_awlock    => s01_axi_awlock_s,
s01_axi_awcache   => s01_axi_awcache_s,
s01_axi_awprot    => s01_axi_awprot_s,
s01_axi_awqos     => s01_axi_awqos_s,
s01_axi_awregion  => s01_axi_awregion_s,
s01_axi_awuser    => s01_axi_awuser_s,
s01_axi_awvalid   => s01_axi_awvalid_s,
s01_axi_awready   => s01_axi_awready_s,
s01_axi_wdata     => s01_axi_wdata_s,
s01_axi_wstrb     => s01_axi_wstrb_s,
s01_axi_wlast     => s01_axi_wlast_s,
s01_axi_wuser     => s01_axi_wuser_s,
s01_axi_wvalid    => s01_axi_wvalid_s,
s01_axi_wready    => s01_axi_wready_s,
s01_axi_bid       => s01_axi_bid_s,
s01_axi_bresp     => s01_axi_bresp_s,
s01_axi_buser     => s01_axi_buser_s,
s01_axi_bvalid    => s01_axi_bvalid_s,
s01_axi_bready    => s01_axi_bready_s,
s01_axi_arid      => s01_axi_arid_s,
s01_axi_araddr    => s01_axi_araddr_s,
s01_axi_arlen     => s01_axi_arlen_s,
s01_axi_arsize    => s01_axi_arsize_s,
s01_axi_arburst   => s01_axi_arburst_s,
s01_axi_arlock    => s01_axi_arlock_s,
s01_axi_arcache   => s01_axi_arcache_s,
s01_axi_arprot    => s01_axi_arprot_s,
s01_axi_arqos     => s01_axi_arqos_s,
s01_axi_arregion  => s01_axi_arregion_s,
s01_axi_aruser    => s01_axi_aruser_s,
s01_axi_arvalid   => s01_axi_arvalid_s,
s01_axi_arready   => s01_axi_arready_s,
s01_axi_rid       => s01_axi_rid_s,

```

```

s01_axi_rdata    => s01_axi_rdata_s,
s01_axi_rresp    => s01_axi_rresp_s,
s01_axi_rlast    => s01_axi_rlast_s,
s01_axi_ruser    => s01_axi_ruser_s,
s01_axi_rvalid   => s01_axi_rvalid_s,
s01_axi_ready    => s01_axi_ready_s);
end architecture beh;

```

Na slici 6.21 prikazani su talasni oblici karakterističnih signala *AXI-Lite* i *AXI-Full* magistrala, koji predstavljaju rezultat simulacije verifikacionog orkuženja i modela IP jezgra „*Naive*“ množača matrica sa AXI interfejsom, u slučaju množenja kvadratnih matrica *A* i *B*, deminenzija 3x3. Matrice *A* i *B* definisane su, kao i u svim dosadašnjim slučajevima, na sledeći način

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 & 12 \\ 13 & 14 & 15 \\ 16 & 17 & 18 \end{bmatrix}.$$

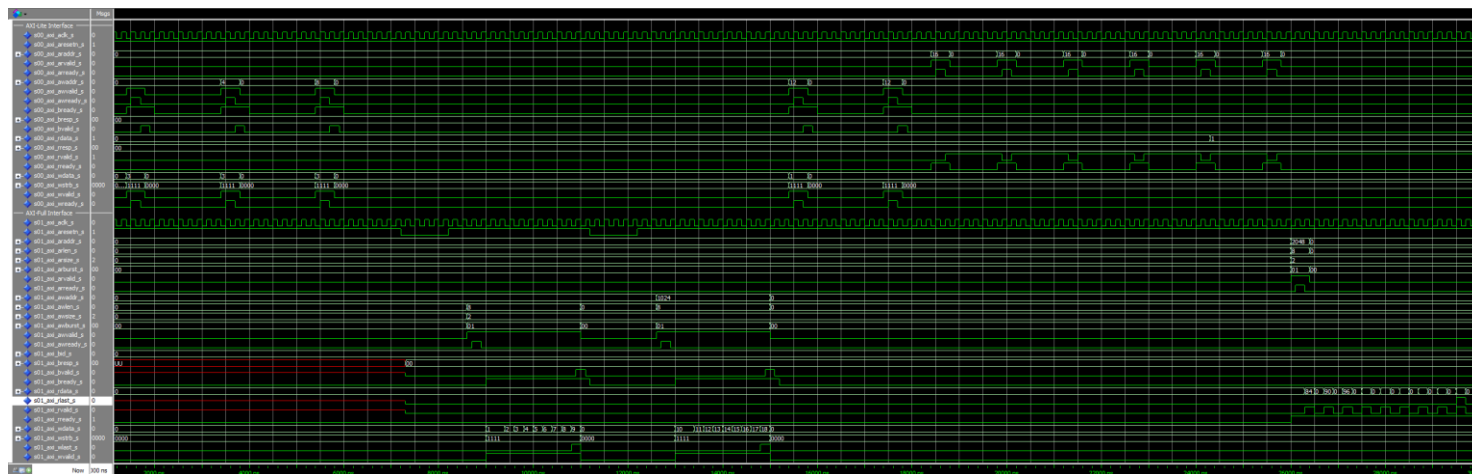
Rezultat množenja ove dve matrice trebalo bi da bude sledeće matrica

$$C = A \cdot B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \cdot \begin{bmatrix} 10 & 11 & 12 \\ 13 & 14 & 15 \\ 16 & 17 & 18 \end{bmatrix} = \begin{bmatrix} 84 & 90 & 96 \\ 201 & 216 & 231 \\ 318 & 342 & 366 \end{bmatrix}.$$

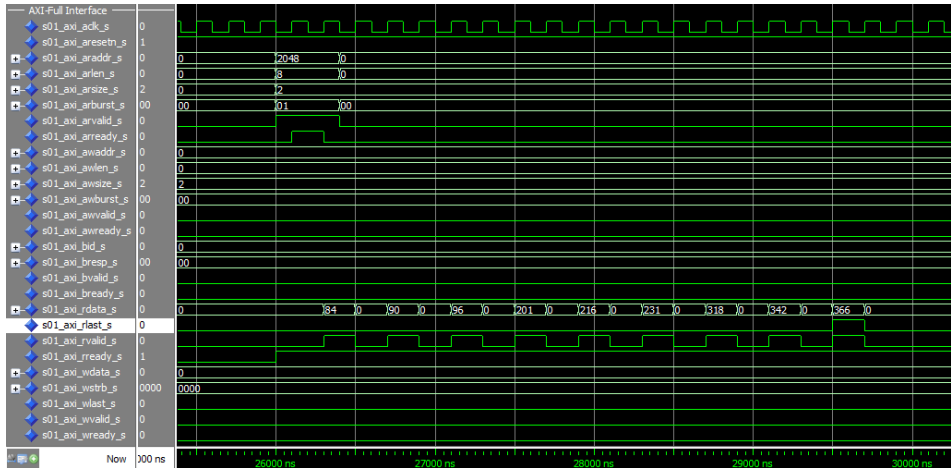
Na slici 6.21 mogu se videti *AXI-Lite* transakcije na početku simulacije pomoću kojih se IP jezgru „*Naive*“ množača matrica prosleđuju vrednosti parametara n_in , m_in i p_in . Nakon ovih transakcija slede dve *AXI-Full burst* transakcije prenosa bloka podataka pomoću kojih se u IP jezgro „*Naive*“ množača matrica upisuju vrednosti elemenata matrica *A* i *B*. Nakon toga sledi još jedna *AXI-Lite* transakcija, pomoću koje se startuje rad množača matrica, podizanjem unutrašnjeg *start* signala na jedinicu. Nakon nje, pomoću dodatne *AXI-Lite* transakcije, spušta se unutrašnji *start* signal na nulu, kako množač matrica nakon završetka množenja tekuće dve matrice ne bi ponovo započeo proces množenja. Nakon ove transakcije sledi odgovarajući broj periodično generisanih *AXI-Lite* transakcija čitanja podatka, pomoću kojih se proverava status unutrašnjeg *ready* signala. Kada vrednost *ready* signala postane jednaka jedinici, to je indikacija da je množač matrica završio množenje dve matrice i da se množe preuzeti rezultat množenja. Korišćenjem jedne *AXI-Full burst* transakcije čitanja bloka podataka, iz IP jezgra „*Naive*“ množača matrica čitaju se vrednosti elemenata matrice *C*, nakon čega se proces simulacije završava.

Kako na slici 6.21 nije moguće videti sve vrednosti elemenata matrice *C*, koje se isčitavaju iz IP jezgra, slika 6.22 prikazuje uvećan taj deo talasnih oblika. Inspekcijom pročitanih vrednosti sadržaja unutrašnje memorije u kojoj je smeštena matrica *C* na

AXI-Full interfejsu sa slike 6.22 (vidite signal *s01_axi_rdata_s*), možemo se uveriti da projektovano IP jezgro korektno implementira „Naive“ algoritam množenja dve matrice.



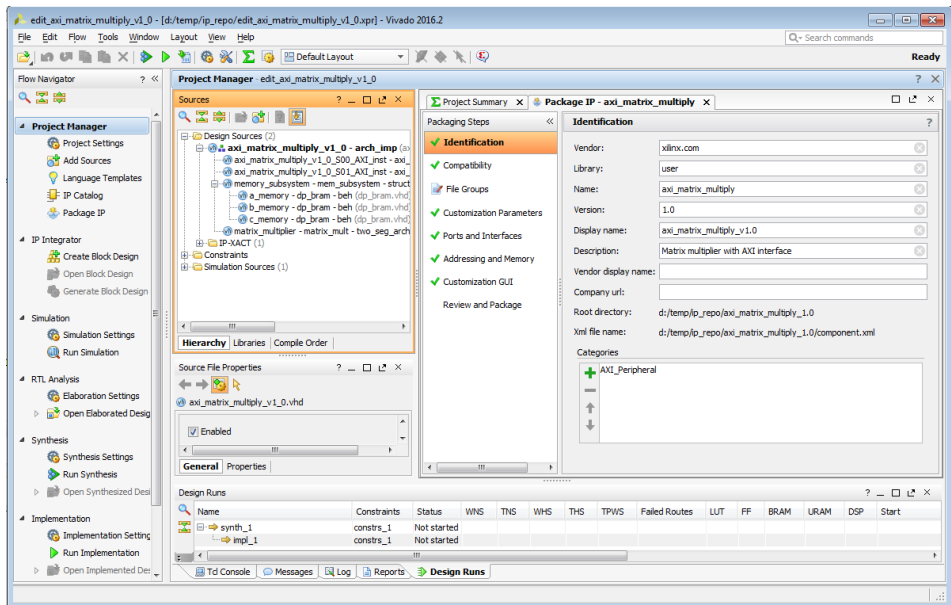
Slika 6.21. Rezultat simulacije rada IP jezgra „Naive“ množača matrica sa AXI interfejsom na primeru množenja dve matrice dimenzija 3×3



Slika 6.22. Uvećani deo talasnih oblika koji predstavljaju rezultat množenja dve matrice, dobijem simulacijom IP jezgra „Naive“ množača matrica sa AXI interfejsom

Završno pakovanje IP jezgra „Naive“ množača matrica

Pre nego što pređemo na završno pakovanje IP jezgra „Naive“ množača matrica, potrebno je izvršiti navedene modifikacije na automatski generisanim VHDL modelima AXI-Lite i AXI-Full kontrolera (fajlovi *axi_matrix_multiply_v1_0_S00_AXI.vhd* i *axi_matrix_multiply_v1_0_S01_AXI.vhd*), kao i na VHDL modelu celokupnog sistema (*axi_matrix_multiply_v1_0.vhd* fajl). Takođe, potrebno je u privremeni projekat uključiti i VHDL modele memorijskog podsistema (*mem_subsystem.vhd*), dvoprístupne memorije (*dp_bram.vhd*), kao i VHDL model samog „Naive“ množača matrica (*matrix_mult.vhd*). Pored ovih VHDL modela u privremeni projekat potrebno je uključiti i *utils_pkg.vhd* VHDL paket. Ovi VHDL dizajn fajlovi uključuju se u privremeni projekat pritiskom na dugme „Add Sources“ u okviru „Project Manager“ grupe unutar Vivado glavnog prozora. Izborom opcije „Add or create design sources“ na sledećem prozoru, a zatim opcije „Add Files“ na sledećem prozoru možemo odabrati dizajn fajlove koje je potrebno uključiti u projekat. Nakon uključivanja potrebnih fajlova, izgled glavnog prozora Vivado alata trebao bi da odgovara onom sa slike 6.23.



Slika 6.23. Izgled glavnog prozora Vivado alata nakon uključivanja dodatnih VHDL dizajn fajlova u privremeni projekat IP jezgra „Naive“ množača matrica

Sada konačno možemo završiti proces pakovanja IP jezgra „Naive“ množača matrica sa AXI interfejsom. Da bi se proces pakovanja kompletirao moramo proći kroz korake pakovanja (*Packaging Steps*), koji su prikazani unutar „*Package IP – axi_matrix_multiply*“ kartice.

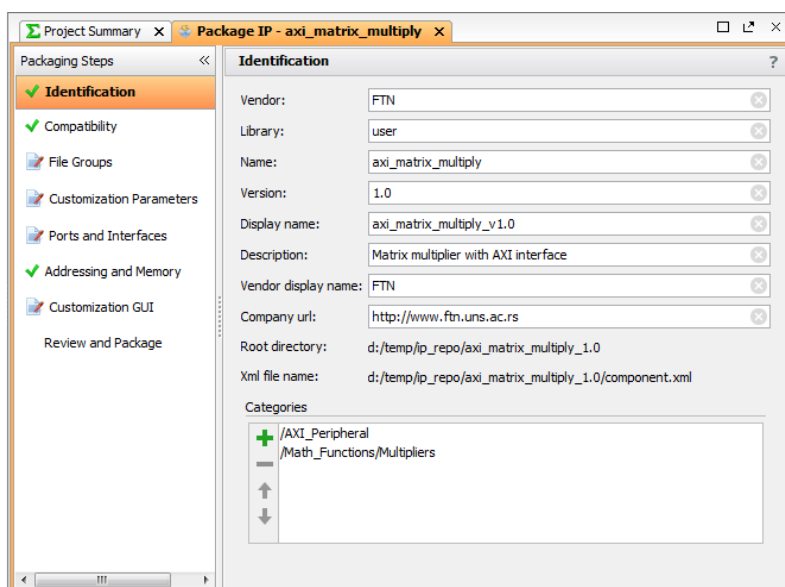
Korak 1: Identifikacija

U okviru ovog koraka potrebno je popuniti odgovarajući broj polja, prikazan na slici 6.23, čiji sadržaj omogućava da se korisniku zapakovanog IP jezgra pruže osnovne informacije o jezgru. Polja čiji sadržaj treba definisati su:

- *Vendor* – sadržaj ovog polja predstavlja ime kompanije koja je razvila IP jezgro
- *Library* – ime biblioteke u koju treba smestiti zapakovano IP jezgro
- *Name* – ime IP jezgra koje pakujemo
- *Version* – verzija IP jezgra koje pakujemo
- *Display name* – predstavlja ime IP jezgra koje će biti prikazivano unutar ostalih Vivado alata

- *Description* – predstavlja kratak opis funkcionalnosti IP jezgra koje pakujemo. Ovaj opis će se pojaviti kada selektujemo IP jezgro unutar IP kataloga Vivado Design Suite alata
- *Vendor display name* – ime proizvođača IP jezgra koje će biti prikazivano unutar ostalih Vivado Design Suite alata
- *Company url* – adresa web sajta kompanije koja je razvila IP jezgro
- *Categories* – prozor koji sadrži kategorije, koje postoje unutar Vivado IP kataloga, u koje želimo da smestimo IP jezgro koje pakujemo. Pritiskom na zeleno plus dugme pojavljuje se novi prozor u okviru kojega možemo izabrati kategorije u koje želimo da smestimo naše IP jezgro. U našem slučaju, pored kategorije *AXI_Peripheral*, potrebno je dodati i kategoriju *Math_Functions/Multipliers*.

U našem slučaju podešavanja ovih polja treba da odgovaraju onim prikazanim na slici 6.24.

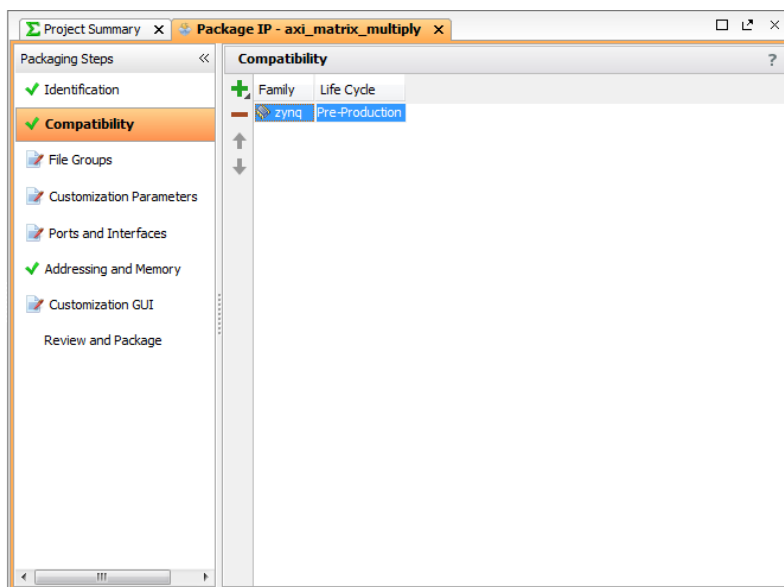


Slika 6.24. Potrebni sadržaj polja unutar Identification koraka prilikom pakovanja IP jezgra „Naive“ množača matrica

Korak 2: Kompatibilnost

Pritiskom na korak *Compatibility* prelazimo na sledeći korak u pakovanju IP jezgra. U ovom koraku potrebno je izabrati Xilinx FPGA familije pomoću kojih se može

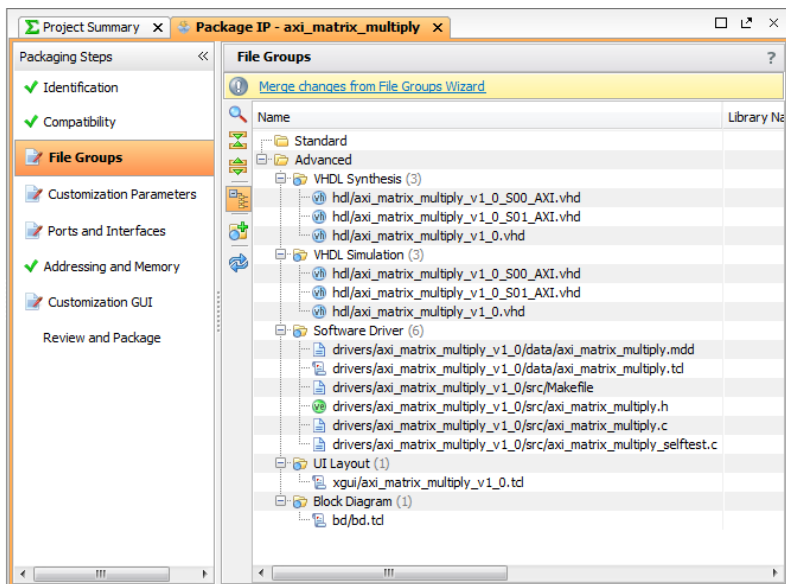
implementirati IP jezgro. Neka jezgra se mogu implementirati bez ikakvih ograničenja pomoću različitih FPGA familija, čak i pomoću FPGA komponenti različitih proizvođača, dok je kod nekih IP jezgara ovaj skup FPGA familija sa kojima su kompatibilna ograničen. U našem slučaju nema nekih posebnih ograničenja po pitanju FPGA familija pomoću kojih je moguće implementirati IP jezgro „Naive“ množača matrica, jer prilikom njegovog razvoja nismo koristili nikakve specifične hardverske resursa koji su prisutni samo u nekim FPGA familijama. U tom smislu mogli bismo selektovati sve Xilinx FPGA familije, ali u našem slučaju dovoljno je ako u listu bude uključena samo Zynq familija, kao što je prikazano na slici 6.25.



Slika 6.25. Potrebni sadržaj polja unutar Compatibility koraka prilikom pakovanja IP jezgra „Naive“ množača matrica

Korak 3: Definisanje grupa fajlova koji treba da budu zapakovani u IP jezgro

Pritiskom na korak „File Groups“ prelazimo na sledeći korak u pakovanju IP jezgra. U okviru ovog koraka potrebno je definisati sve dizajn i ostale tipove fajlova koje želimo da zapakujemo unutar IP jezgra. Svaki fajl koji želimo da zapakujemo moramo da smestimo u jednu od mogućih grupa. Osnovne grupe fajlova, sa njihovim inicijalnim sadržajem prikazane su na slici 6.26.



Slika 6.26. Inicijalni sadržaj polja unutar „File Groups“ koraka prilikom pakovanja IP jezgra „Naive“ množača matrica

Sa slike 6.26 možemo videti da je IP Packager čarobnjak već uključio izvestan broj fajlova u odgovarajuće najvažnije grupe:

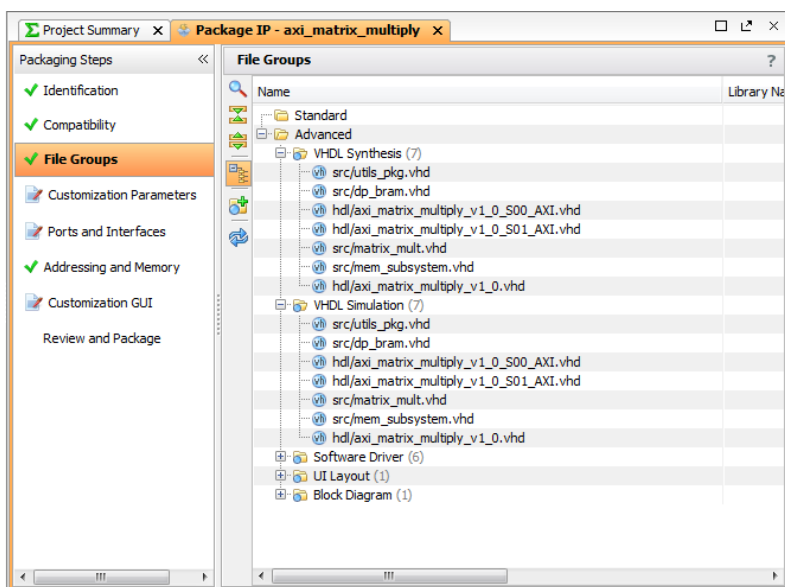
- „VHDL Synthesis“ – u ovoj grupi nalaze se fajlovi koje je potrebno koristiti prilikom sinteze IP jezgra
- „VHDL Simulation“ - u ovoj grupi nalaze se fajlovi koje je potrebno koristiti prilikom simulacije IP jezgra
- „Software Driver“ – u ovoj grupi nalaze se fajlovi koji čine softverski drajver za IP jezgro
- „UI Layout“ – u ovoj grupi nalaze se fajlovi koji definišu izgled korisničkog prozora koji se pojavljuje prilikom kastomizacije IP jezgra u trenutku kada korisnik želi da kreira jednu instancu IP jezgra
- „Block Diagram“ – u ovoj grupi nalaze se fajlovi koji generišu blok dijagram koji opisuje strukturu IP jezgra

Pored ovih grupa fajlova postoji još mnogo drugih. Pristiskom na dugme  moguće je sa liste ponuđenih grupa fajlova odabrati one koje su nam neophodne.

Najvažnija grupa svakako je „VHDL Synthesis“ grupa, te ćemo se mi na ovom mestu koncentrisati samo na njen sadržaj. Zainteresovani čitalac upućuje se na dodatnu

literaturu kompanije Xilinx u kojoj se mogu pronaći dodatne informacije o ostalim grupama.

Na slici 6.26. može se primetiti da je IP Packager čarobnjak identifikovao da u okviru privremenog projekta postoje dodatni dizajn fajlovi koji čine IP jezgro „Naive“ množača matrica, koji su bili dodati ručno od strane projektanta (reč je o *mem_subsystem.vhd*, *dp_bram.vhd*, *matrix_mult.vhd* i *utils_pkg.vhd* fajlovima). Čarobnjak je primetio da ovi fajlovi nisu dodati u „VHDL Synthesis“ grupu i ponudio nam je opciju da on to uradi umesto nas („Merge changes from File Groups Wizard“ akcija na vrhu „File Groups“ kartice). Pritiskom na ovu akciju, IP Packager čarobnjak automatski uključuje ove dizajn fajlove u odgovarajuće grupe, što se može videti na slici 6.27.



Slika 6.27. Konačni sadržaj polja unutar „File Groups“ koraka prilikom pakovanja IP jezgra „Naive“ množača matrica, nakon uključivanja dodatnih dizajn fajlova

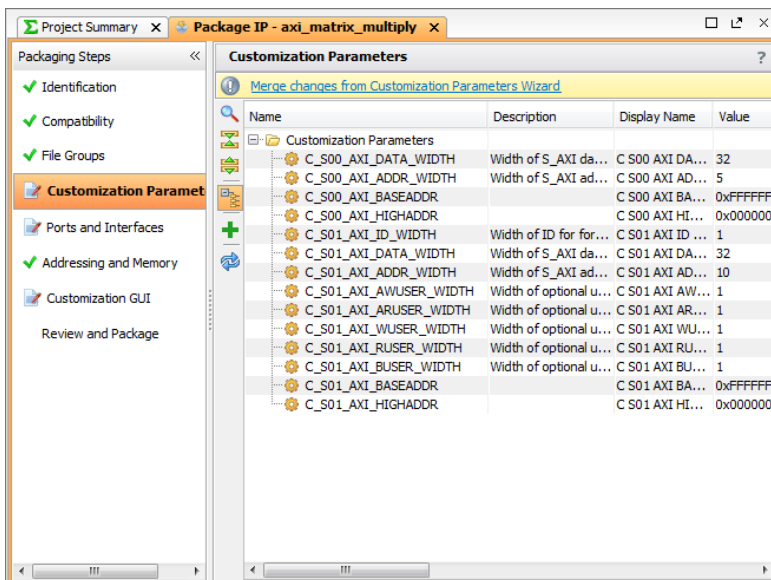
NAPOMENA: Primetimo takođe da se znak pored „File Groups“ koraka promenio u odnosu na sliku 6.26. Tekući znak označava da u „File Groups“ koraku čarobnjak nema nikakve sugestije i da možemo nastaviti dalje sa procesom pakovanja. Imajući ovo u vidu sa slike 6.27 možemo videti da u koracima „Customization Parameters“, „Ports and Interfaces“ i „Customization GUI“ čarobnjak ima izvesne sugestije na koje moramo odgovoriti, pre nego što se proces pakovanja može završiti.

Korak 4: Definisanje konfiguracionih parametara IP jezgra

U okviru „*Customization Parameters*“ koraka možemo definisati konfiguracione parametre IP jezgra koje pakujemo. Ovi konfiguracioni parametri omogućavaju da korisnik IP jezgra konfiguriše IP jezgro prema zahtevima njegove aplikacije. Konfiguracioni parametri zapravo predstavljaju vrednosti generičkih konstanti koje su definisane u okviru *entity* deklaracije glavnog VHDL fajla IP jezgra. U našem slučaju glavni VHDL fajl je *axi_matrix_multiply_v1_0.vhd* fajl. Inspekcijom njegove *entity* deklaracije možemo zaključiti da ona sadrži sledeće generičke konstante

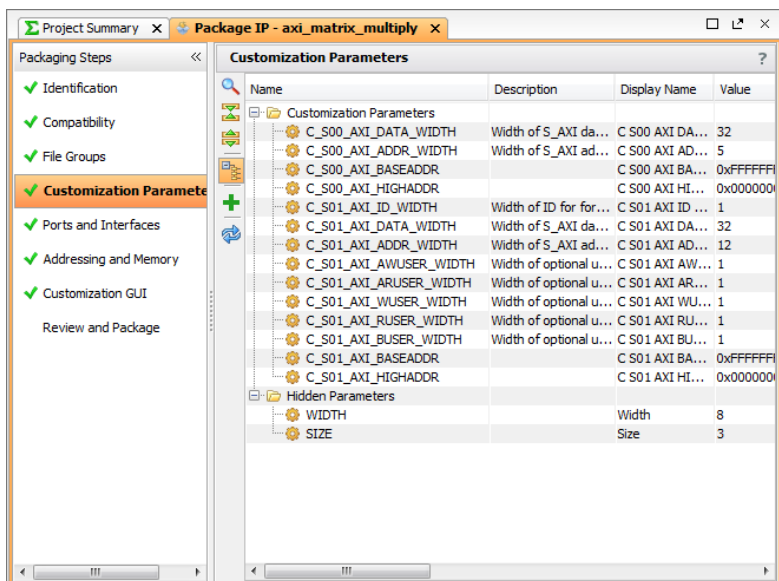
```
generic (  
  -- Users to add parameters here  
  WIDTH      : integer := 8;  
  SIZE       : integer := 3;  
  -- User parameters ends  
  -- Do not modify the parameters beyond this line  
  -- Parameters of Axi Slave Bus Interface S00_AXI  
  C_S00_AXI_DATA_WIDTH      : integer := 32;  
  C_S00_AXI_ADDR_WIDTH      : integer := 5;  
  -- Parameters of Axi Slave Bus Interface S01_AXI  
  C_S01_AXI_ID_WIDTH        : integer := 1;  
  C_S01_AXI_DATA_WIDTH      : integer := 32;  
  C_S01_AXI_ADDR_WIDTH      : integer := 12;  
  C_S01_AXI_AWUSER_WIDTH    : integer := 1;  
  C_S01_AXI_ARUSER_WIDTH    : integer := 1;  
  C_S01_AXI_WUSER_WIDTH     : integer := 1;  
  C_S01_AXI_RUSER_WIDTH     : integer := 1;  
  C_S01_AXI_BUSER_WIDTH     : integer := 1  
);
```

Pored automatski generisanih generičkih konstanti, postoje i dve, *WIDTH* i *SIZE*, koje smo mi dodali prilikom modifikacije *axi_matrix_multiply_v1_0.vhd* dizajn fajla. *WIDTH* generička konstanta definiše broj bitova koji se koristi za reprezentaciju vrednosti elemenata matrica *A* i *B*, dok *SIZE* generička konstanta definiše maksimalni broj elemenata po bilo kojoj dimenziji matrica *A*, *B* i *C*. Inicijalni sadržaj „*Customization Parameters*“ kartice prikazan je na slici 6.28.



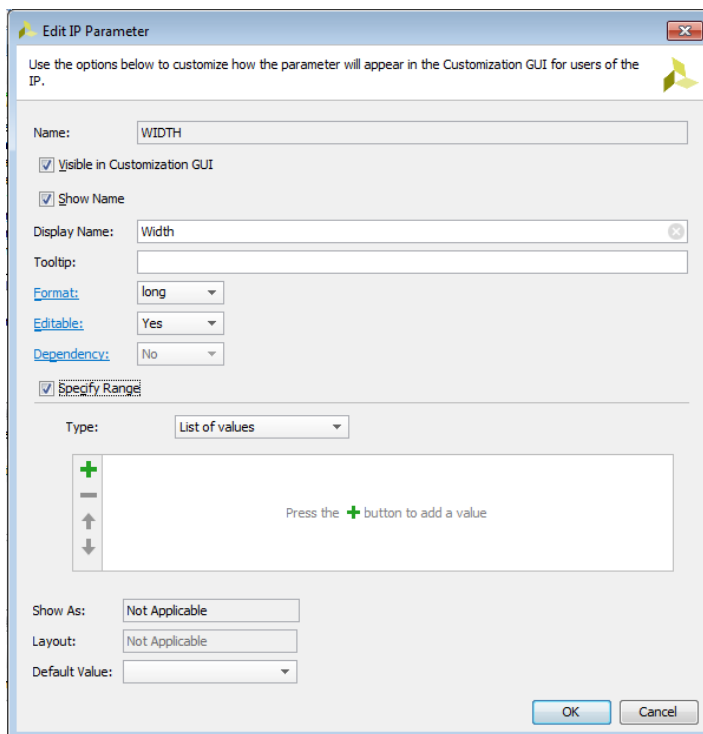
Slika 6.28. Inicijalni sadržaj „Customization Parameters“ kartice prilikom pakovanja IP jezgra „Naive“ množača matrica

Kao i u koraku 3, i u ovom slučaju IP Packager čarobnjak je primetio da u listi konfiguracionih parametara nedostaju parametri *WIDTH* i *SIZE*, i ponudio nam se da ih on automatski uključi (akcija „Merge changes from Customization Parameters Wizard“). Pritiskom na ovu akciju, sadržaj „Customization Parameters“ kartice se menja i trebao bi da odgovara sadržaju sa slike 6.29.



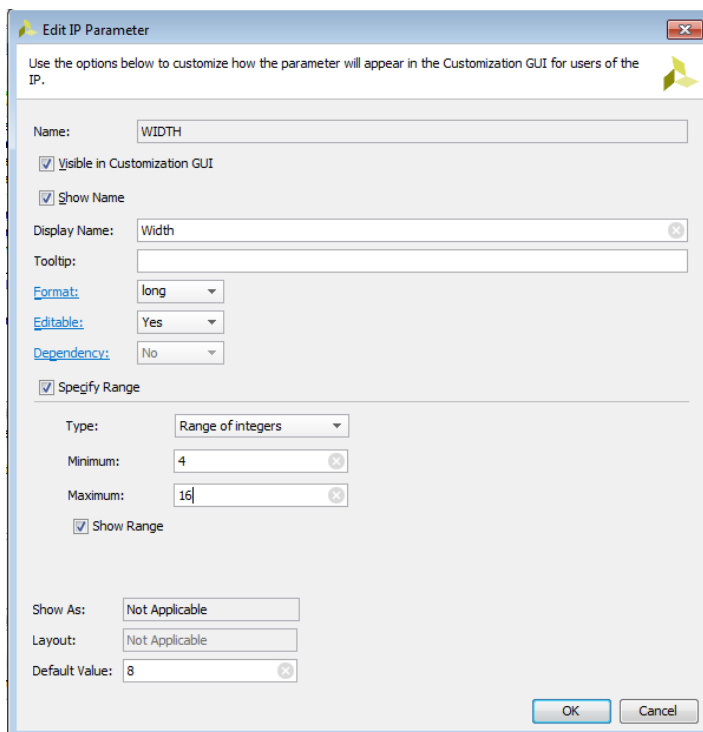
Slika 6.29. Sadržaj „Customization Parameters“ kartice prilikom pakovanja IP jezgra „Naive“ množača matrica, nakon dodavanja konfiguracionih parametara WIDTH i SIZE

Sa slike 6.29 možemo videti da je čarobnjak konfiguracione parametre *WIDTH* i *SIZE* definisao kao skrivene parametre za korisnika (smestio ih je u „Hidden Parameters“ grupu). To znači da korisnik prilikom konfigurisanja IP jezgra neće imati mogućnost da menja njihove vrednosti, već će uvek biti korišćene podrazumevane vrednosti, onako kako su definisane unutar *generic* mape prilikom definisanja *entity* deklaracije. U našem slučaju parametar *WIDTH* imao bi vrednost 8, a *SIZE* vrednost 3. Kako mi želimo da dozvolimo korisniku da, uz izvesna ograničenja, definiše vrednosti ovih parametara prema sopstvenim potrebama, moramo ova dva parametra prebaciti iz „Hidden Parameters“ grupe u „Customization Parameters“ grupu. Ovo se može izvesti na sledeći način. Selektovanjem odgovarajućeg parametra, pritiskom na desno dugme miša i izborom „Edit Parameter“ opcije otvara se novi prozor, u kojem je moguće promeniti karakteristike odabranog konfiguracionog parametra. Na primer, ukoliko izaberemo parametar *WIDTH* i izaberemo „Edit Parameter“ opciju na način opisan malopre, otvoriće se „Edit IP Parameter“ prozor prikazan na slici 6.30. Inicijalno je većina opcija nedostupna, jer *WIDTH* parametar trenutno pripada „Hidden Parameters“ grupi. Aktiviranjem opcije „Visible in Customization GUI“, prevodimo *WIDTH* parametar u „Customization Parameters“ grupu, čime se većina opcija u okviru „Edit IP Parameter“ prozora će se aktivirati, kao što je prikazano na slici 6.30.



Slika 6.30. Sadržaj „Edit IP Parameter“ prozora za parametar *WIDTH*

Pored toga što smo *WIDTH* parametar preveli u grupu vidljivih parametara za korisnika IP jezgra, sada je trenutak i da definišemo opseg dozvoljenih vrednosti ovog parametra. Aktiviranjem opcije „Specify Range“, izborom opcije „Range of Integers“ izgled „Edit IP Parameter“ prozora se menja. Sada imamo polja unutar kojih možemo definisati minimalnu i maksimalnu vrednost *WIDTH* konfiguracionog parametra. Kako *WIDTH* parametar predstavlja broj bitova koji se koriste za reprezentaciju vrednosti elemenata matrica *A*, *B*, i indirektno matrice *C*, njegov opseg dozvoljenih vrednosti možemo izabrati prilično proizvoljno. U našem primeru dozvoljeni opseg vrednosti parametra *WIDTH* treba da bude jednak onom prikazanom na slici 6.31.



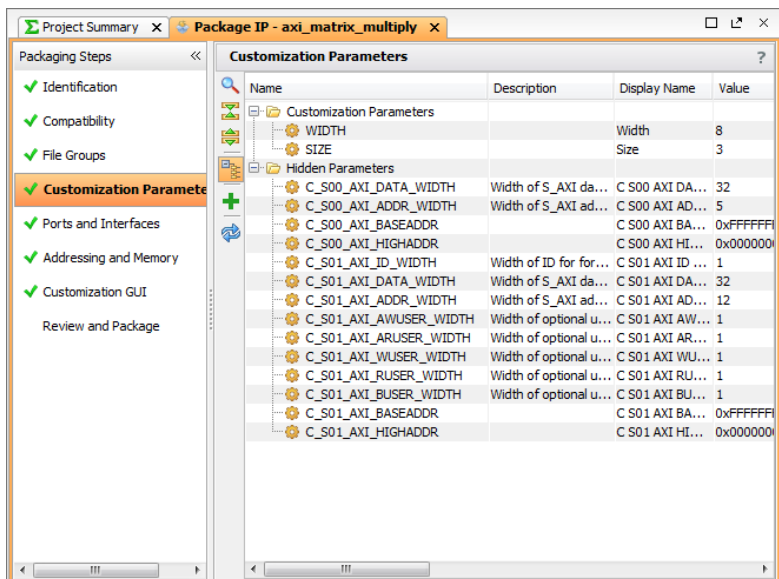
Slika 6.31. Konačni sadržaj „Edit IP Parameter“ prozora za parametar *WIDTH*

Pritiskom na OK dugme vraćamo se u glavni prozor Vivado Design Suite alata, u kojem se sada može videti da je *WIDTH* parametar prebačen u „Customization Parameters“ grupu.

Slično je potrebno uraditi i sa parametrom *SIZE*, pri čemu u slučaju parametra *SIZE* opseg dozvoljenih vrednosti treba da bude postavljen na opseg od 2 do 8.

NAPOMENA: Maskimalni broj elemenata u bilo kojoj dimenziji matrica *A*, *B* i *C* limitiran je na 8, jer se na taj način ograničava maksimalni broj elemenata koje mogu imati matrice *A*, *B* i *C* na 256. Ovo je važno jer smo prilikom projektovanja adresnog dekodera, unutar memorijskog podsistema, pretpostavili da memorije *A*, *B* i *C* neće imati više od 256 elemenata.

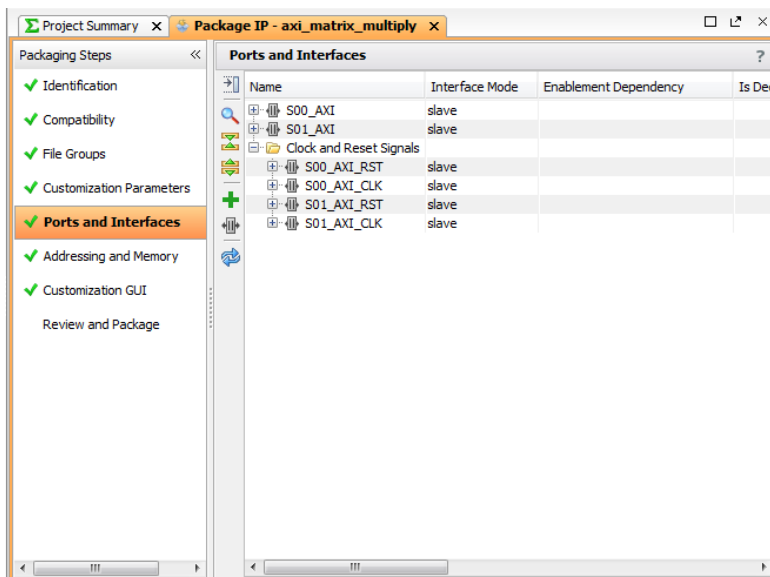
Kako su vrednosti automatski generisanih konfiguracionih parametara fiksne i korisnik ne bi trebao da ih menja, potrebno je prebaciti sve automatski generisane konfiguracione parametre iz „Customization Parameters“ grupe u „Hidden Parameters“ grupu. Nakon svih navedenih modifikacija konačni sadržaj „Customization Parameters“ kartice trebalo bi da odgovara onom sa slike 6.32.



Slika 6.32. Konačni sadržaj „Customization Parameters“ kartice prilikom pakovanja IP jezgra „Naive“ množača matrica

Korak 5: definisanje portova i interfejsa IP jezgra

U ovom koraku mogu se definisati celokupni interfejsi (na primer AXI interfejsi), kao i individualni portovi koji čine kompletan interfejs IP jezgra koje se pakuje. Kako u našem primeru nismo ručno dodavali nikakve nove interfejse ili portove (osim AXI-Lite i AXI-Full interfejsa koje smo definisali prilikom kreiranja IP jezgra pomoću čarobnjaka), IP Packager čarobnjak nema nikakvu dodatnu sugestiju. Sadržaj „Ports and Interfaces“ kartice prikazan je na slici 6.33.



Slika 6.33. Sadržaj „Ports and Interfaces“ kartice prilikom pakovanja IP jezgra „Naive“ množača matrica

Sa slike 6.33 možemo videti da je IP Packager čarobnjak pravilno identifikovao prisustvo dva AXI interfejsa, AXI-Lite interfejsa označenog kao *S00_AXI*, i AXI-Full interfejsa označenog kao *S01_AXI*. Ovo su i jedini interfejsi koje IP jezgro „Naive“ množača matrica ima. Pored ova dva interfejsa, čarobnjak je identifikovao po dva ulazna porta za AXI klok i reset signale.

Korak 6: definisanje memorijske mape IP jezgra

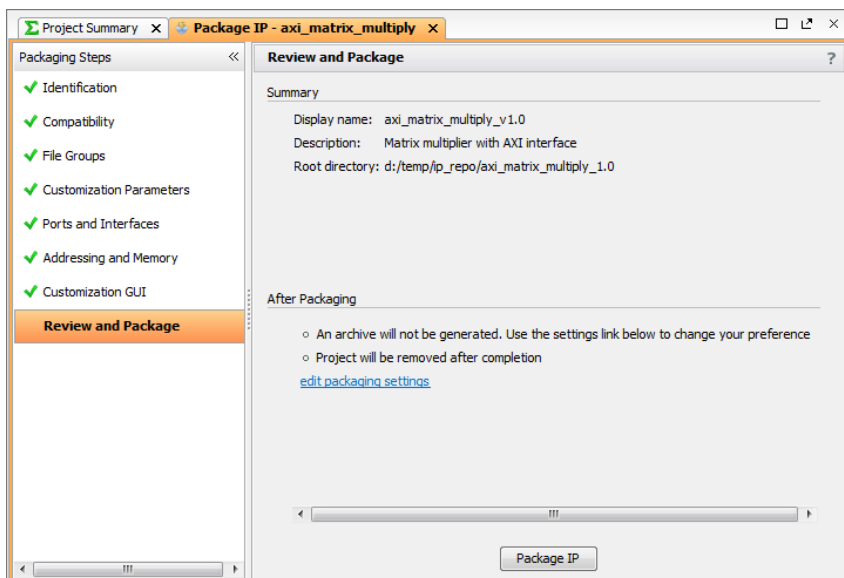
U ovom koraku mogu se definisati detalji unutrašnje memorijske mape IP jezgra. U našem slučaju nije potrebno izvršiti nikakve modifikacije, tako se ovaj korak može preskočiti.

Korak 7: definisanje izgleda grafičkog korisničkog interfejsa za konfigurisanje IP jezgra

U ovom koraku mogu se napraviti izmene u vizuelnom izgledu konfiguracionog prozora koji će se prikazati korisniku IP jezgra. U našem slučaju nećemo praviti nikakve izmene inicijalnog izgleda grafičkog korisničkog interfejsa, pa možemo preći na završni korak pakovanja IP jezgra.

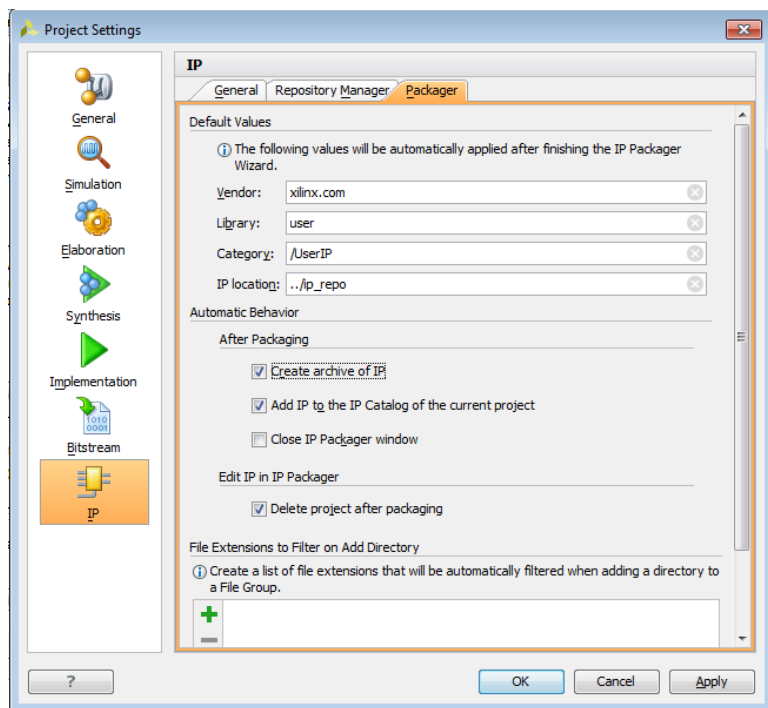
Korak 8: pakovanje IP jezgra

Nakon što smo, tokom prethodnih sedam koraka, pružili sve potrebne informacije IP Packager čarobnjaku, možemo konačno zapakovati IP jezgro. Ovo se postiže pritiskom na „*Review and Package*“ akciju. Izgled „*Review and Package*“ kartice prikazan je na slici 6.34.



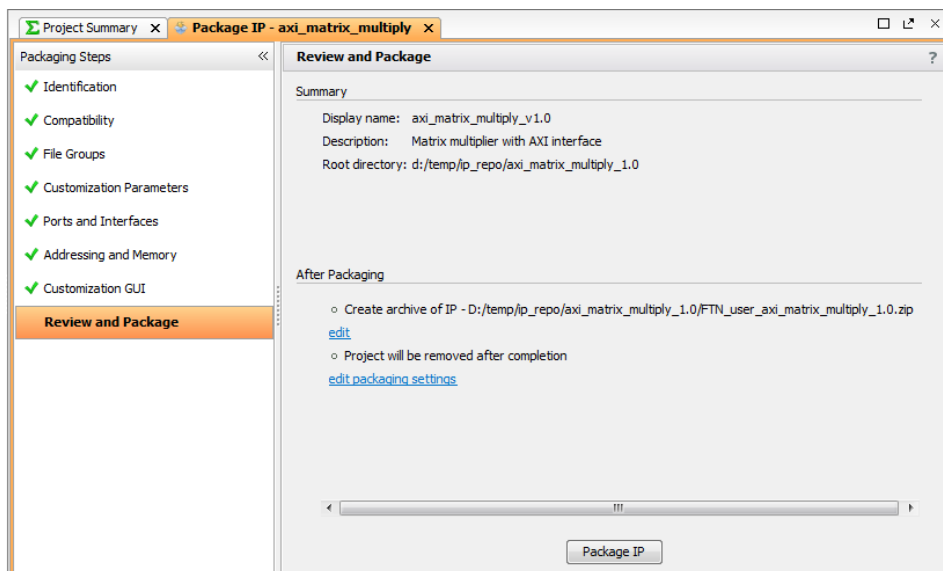
Slika 6.34. Izgled „*Review and Package*“ kartice prilikom pakovanja IP jezgra „*Naive*“ množača matrica

Obratite pažnju da su inicijalna podešavanja „*Review and Package*“ kartice takva da prilikom pakovanja neće biti generisana arhiva koja sadrži zapakovano IP jezgro „*Naive*“ množača matrica („*An archive will not be generated.*“). Kako je nama ova arhiva neophodna, potrebno je promeniti inicijalna podešavanja Vivado Design Suite alata. Ovo se postiže pritiskom na opciju „*edit packaging settings*“, nakon koje se otvara prozor sa slike 6.35.



Slika 6.35. Izgled Packager kartice sa potrebnim podešavanjima

U okviru *Packager* kartice „*Project Settings*“ prozora mogu se podesiti opcije pakovanja IP jezgara. Kako mi želimo da IP Packager prilikom pakovanja jezgra generiše arhivu, potrebno je uključiti opciju „*Create archive of IP*“. Nakon što smo zadovoljni, pritiskom na dugme OK vraćamo se u IP Packager prozor, u kojem možemo videti da se sadržaj „*Review and Package*“ kartice promenio, i sada ima izgled kao na slici 6.36.

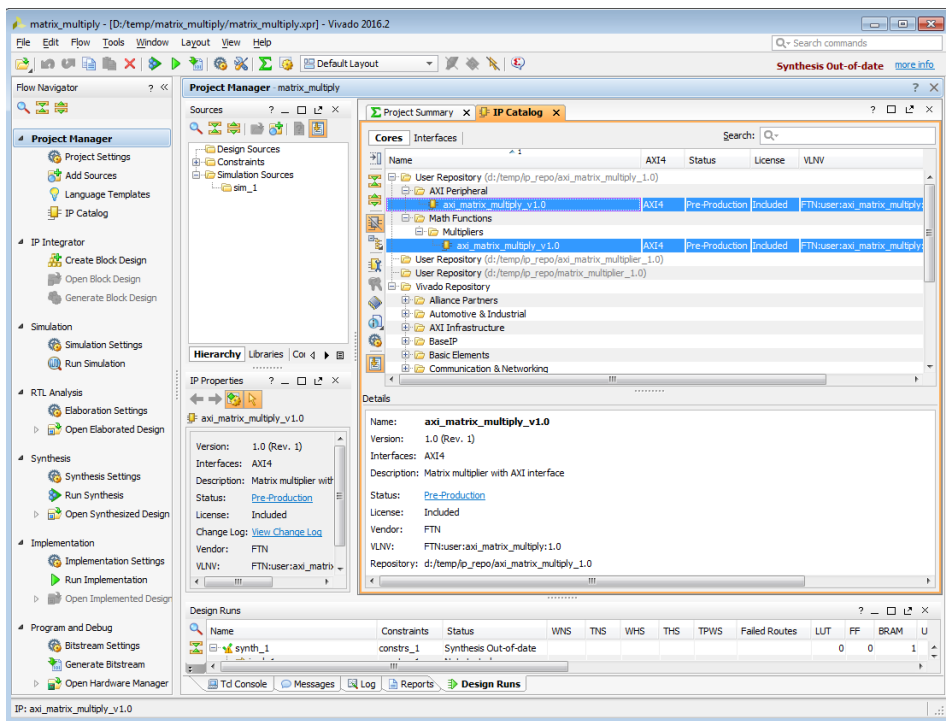


Slika 6.36. Konačni izgled „Review and Package“ kartice prilikom pakovanja IP jezgra „Naive“ množača matrica, nakon promene opcija pakovanja

Sa slike 6.36 sada možemo videti da će prilikom pakovanja IP jezgra „Naive“ množača matrica sa AXI interfejsom biti kreirana arhiva, pod imenom *FTN_user_axi_matrix_multiply_1.0.zip*, kao i lokacija na disku gde će ona biti smeštena.

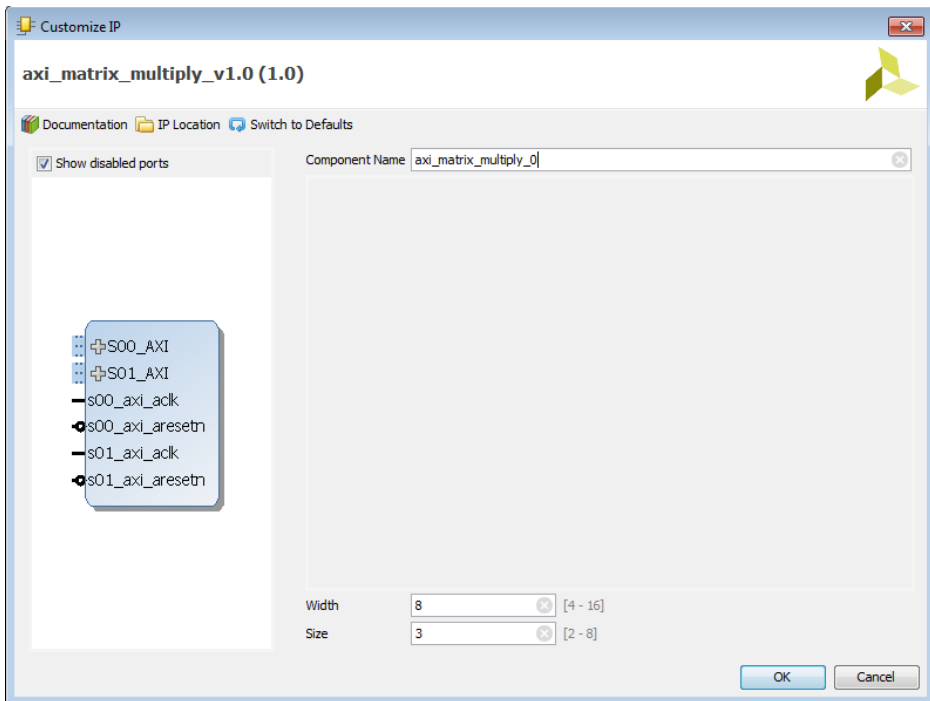
Pritiskom na dugme „Package IP“ završava se postupak generisanja IP jezgra „Naive“ množača matrica sa AXI interfejsom, zatvara se privremeni Vivado projekat i vraćamo se u glavni Vivado prozor *matrix_multiply* projekta.

Pritiskom na opciju „IP Catalog“ iz „Project Manager“ grupe otvara se „IP Catalog“ kartica, prikazana na slici 6.37, koja sadrži spisak svih IP jezgara koja se trenutno nalaze unutar Vivado Design Suite IP repozitorijuma.



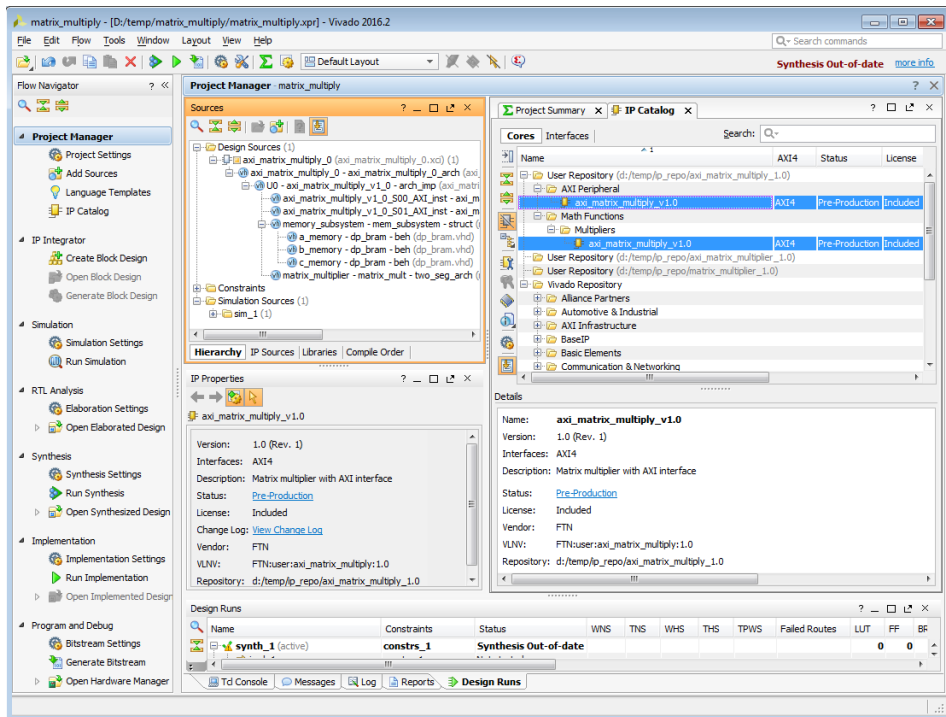
Slika 6.37. Sadržaj „IP Catalog“ kartice sa IP jezgrom „Naive“ množača matrica uključenim u repozitorijum raspoloživih IP jezgara

Sa slike vidimo da se IP jezgro „Naive“ množača matrica nalazi unutar „AXI Peripheral“ i „Math Functions/Multipliers“ grupa IP jezgara, kao što smo i definisali prilikom pakovanja IP jezgra. Pritiskom na *axi_matrix_multiply_v1.0* IP jezgro dobijamo osnovne informacije o njemu, koje se takođe mogu videti na slici 6.37. Vidimo da osnovne informacije o IP jezgru „Naive“ množača matrica odgovaraju onim koje smo definisali prilikom pakovanja IP jezgra. Dvoklikom na *axi_matrix_multiply_v1.0* IP jezgro pokreće se postupak generisanja instance IP jezgra, koja će zatim biti uključena u tekući projekat. Prvi korak prilikom generisanja instance IP jezgra jeste da se ona konfiguriše prema potrebama ciljne aplikacije. U tu svru otvara se „Customize IP“ prozor, prikazan na slici 6.38.



Slika 6.38. Sadržaj „Customize IP“ prozora za IP jezgro „Naive“ množača matrica

Sa slike 6.38 možemo videti da u slučaju konfigurisanja parametara IP jezgra „Naive“ množača matrica možemo podesiti vrednosti za samo dva konfiguraciona parametra, *WIDTH* i *SIZE*. Ovo je upravo ono što smo definisali prilikom pakovanja IP jezgra „Naive“ množača matrica. Pored definisanja vrednosti za ova dva parametra, možemo definisati i ime instance IP jezgra koja će biti generisana (pomoću polja „Component Name“). Kada smo zadovoljni sa konfiguracijom IP jezgra, pritiskom na dugme OK, a zatim na dugme *Generate* u sledećem prozoru, Vivado Design Suite alat generiše instancu odabranog IP jezgra i uključuje je u tekući projekat, što se može proveriti inspekcijom sadržaja „Design Sources“ grupe unutar *Sources* prozora u okviru „Project Manager“ prozora. Unutar navedene grupe sada se nalazi instanca IP jezgra „Naive“ množača matrica, pod nazivom *axi_matrix_multiply_0*, što odgovara imenu koje smo odabrali za ovu instancu prilikom njenog generisanja (vidite sadržaj „Component Name“ polja sa slike 6.38). Otvaranjem unutrašnjeg sadržaja ove instance IP jezgra „Naive“ množača matrica, pritiskom na znak plus pored njenog imena, možemo videti unutrašnju hijerarhiju IP jezgra „Naive“ množača matrica sa AXI interfejsom sa svim dizajn fajlovima koji čine njegov model, kao što je prikazano na slici 6.39. Obratite pažnju da su dizajn fajlovi koji čine IP jezgro „Naive“ množača matrica upravo oni koje smo definisali u postupku njegovog pakovanja.



Slika 6.39. Sadržaj Vivado projekta u koji je uključena instanca IP jezgra „Naive“ množača matrica, zajedno sa prikazom njene unutrašnje strukture

Zadaci za vežbu

Zadatak 6.1.

Izvršiti projektovanje IP jezgra koje će implementirati algoritam za računanje kvadratnog korena zadatog broja, prikazan u zadatku 4.1. Koristiti modul za hardversku implementaciju algoritma za računanje kvadratnog korena zadatog broja koji je razvijen u zadatku 4.1, a projektovati potreban memorijski podsistem, kao i potrebne AXI interfejsse kako bi se dobilo potpuno funkcionalno IP jezgro koje se može integrisati u AXI bazirane sisteme na čipu.

Napisati verifikaciono okruženje pomoću kojega je moguće verifikovati korektan rad razvijenog IP jezgra.

Razvijeno jezgro zapakovati pomoću IP packager alata.

Zadatak 6.2.

Izvršiti projektovanje IP jezgra koje će implementirati algoritam za nalaženje pozicije zadatog elementa unutar prethodno sortirano niza brojeva, prikazan u zadatku 4.4. Koristiti modul za hardversku implementaciju algoritma za nalaženje pozicije zadatog elementa unutar prethodno sortirano niza brojeva, koji je razvijen u zadatku 4.4, a projektovati potreban memorijski podsistem, kao i potrebne AXI interfejsse kako bi se dobilo potpuno funkcionalno IP jezgro koje se može integrisati u AXI bazirane sisteme na čipu.

Napisati verifikaciono okruženje pomoću kojega je moguće verifikovati korektan rad razvijenog IP jezgra.

Razvijeno jezgro zapakovati pomoću IP packager alata.

Zadatak 6.3.

Izvršiti projektovanje IP jezgra koje će implementirati algoritam za određivanje indeksa elementa niza brojeva koji je najbliži srednjoj vrednosti članova niza, prikazan u zadatku 4.5. Koristiti modul za hardversku implementaciju algoritma za određivanje indeksa elementa niza brojeva koji je najbliži srednjoj vrednosti članova niza, koji je razvijen u zadatku 4.5, a projektovati potreban memorijski podsistem, kao i potrebne AXI interfejsse kako bi se dobilo potpuno funkcionalno IP jezgro koje se može integrisati u AXI bazirane sisteme na čipu.

Napisati verifikaciono okruženje pomoću kojega je moguće verifikovati korektan rad razvijenog IP jezgra.

Razvijeno jezgro zapakovati pomoću IP packager alata.

Zadatak 6.4.

Izvršiti projektovanje IP jezgra koje će implementirati „*Selection Sort*“ algoritam za sortiranje niza brojeva, prikazan u zadatku 4.6. Koristiti modul za hardversku implementaciju „*Selection Sort*“ algoritma za sortiranje niza brojeva, koji je razvijen u zadatku 4.6, a projektovati potreban memorijski podsistem, kao i potrebne AXI interfejsse kako bi se dobilo potpuno funkcionalno IP jezgro koje se može integrisati u AXI bazirane sisteme na čipu.

Napisati verifikaciono okruženje pomoću kojega je moguće verifikovati korektan rad razvijenog IP jezgra.

Razvijeno jezgro zapakovati pomoću IP packager alata.